

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

ПО ПРОГРАММЕ МАГИСТРАТУРЫ

09.04.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Предпринимательство, инновации и технологии будущего в

программной инженерии»

Интеллектуальная система оптимизации производственной деятельности

на основе АИС

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

Д. А. Каракчиев

(инициалы, фамилия)

Группа ПО-41м

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А.В. Какурина

(инициалы, фамилия)

Рецензент

(подпись, дата)

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Юго-Западный государственный университет

Кафедра _____ программной инженерии _____

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«_____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ МАГИСТРАТУРЫ

Студента Каракчиева Д.А., шифр 24-06-0544, группа ПО-41м

1. Тема «Интеллектуальная система оптимизации производственной деятельности на основе АИС» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1586-с.

2. Срок предоставления работы к защите

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) Определить функциональные и технические требования разрабатываемого приложения.

2) Разработать концептуальную модель интеллектуальной системы производственного планирования.

3) Спроектировать программную систему построения расписаний и адаптивной диспетчеризации.

4) Сконструировать и протестировать программную систему с нейросетевыми модулями планирования.

3.2. Входные данные и требуемые результаты для программы:

1) Производственные заказы – перечень заказов с указанием сроков сдачи, приоритетов и технологических операций, определяющих последовательность и содержание работ.

2) Производственные ресурсы – сведения о станках и рабочих центрах, их текущем состоянии, коэффициенте готовности, признаках ремонта и специализации по типам операций.

3) Нормативно-справочная информация – перечень типов операций и допустимых связей «ресурс–тип», ограничивающих назначение операций на оборудование.

4) Обучающие выборки – синтетические данные о состояниях оборудования и эвристических назначениях операций, используемые для тренировки нейросетевых моделей.

5) Обученные нейросетевые модели – файлы весов, полученные в результате тренировки моделей-диспетчеров (PointerNet, SLIM, PPO), готовые к загрузке и применению.

6) Производственное расписание – назначенные для каждой операции ресурсы, время старта и окончания, сформированное системой по выбранному методу планирования.

7) Диаграмма Ганта – визуализация построенного расписания с цветовой кодировкой по типам операций и временной шкалой.

8) Рекомендации нейросетевого диспетчера – предлагаемые ресурсы для каждой неразмещённой операции, полученные на основе текущего состояния системы.

9) Метрики обучения – кривые потерь, точность на валидации, матрицы ошибок и отчёты классификации, сохраняемые в процессе тренировки моделей.

10) Сводные отчёты сравнения методов – показатели опоздания, makespan, коэффициент вариации загрузки и другие KPI, вычисляемые при тестировании различных методов планирования.

4. Перечень вопросов, подлежащих исследованию (разработке):

4.1 Анализ существующих решений в области производственного планирования и диспетчеризации, выявление их преимуществ и ограничений.

4.2 Исследование эвристических алгоритмов и методов обучения нейросетевых диспетчеров для задачи гибкого производственного расписания.

4.3 Сравнительный анализ эффективности нейросетевых и классических методов построения расписаний по ключевым производственным показателям.

5. Перечень графического материала:

Лист 1. Сведения о ВКРМ

Лист 2. Актуальность темы

Лист 3. Цель и задачи разработки

Лист 4. Концептуальная модель данных

Лист 5. Архитектура нейронной сети

Лист 6. Функция активации ReLU

Лист 7. Диаграмма компонентов клиентского приложения

Лист 8. Матрица ошибок валидации

Лист 9. Интерфейс интеллектуальной системы

Лист 10. Обучение модели нейросети

Лист 11. Заключение

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

Д. А. Каракчиев

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 148 страницам. Работа содержит 36 иллюстраций, 21 таблицу, 50 библиографических источников и 11 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: оперативное перепланирование, производственное планирование, диспетчеризация, гибкая производственная система, нейросетевые модели, обучение с подкреплением, поведенческое клонирование, самоконтролируемое обучение, эвристические алгоритмы, оптимизация расписаний, цифровой двойник производства.

Объектом исследования являются процессы оперативного производственного планирования на промышленных предприятиях с дискретным характером производства.

Целью выпускной квалификационной работы является разработка интеллектуальной системы, обеспечивающей повышение эффективности производственного планирования посредством адаптивной диспетчеризации операций с использованием нейро-сетевых методов.

При проведении исследований использовались методы системного анализа, математического моделирования, эвристической оптимизации, глубокого обучения и объектно-ориентированного программирования.

Разработана интеллектуальная система производственного планирования с графическим интерфейсом, включающая модули управления заказами и ресурсами, построения расписаний и обучения нейросетевых диспетчеров.

При создании программного продукта использовалась двухзвенная архитектура, технология объектно-ориентированного программирования и библиотеки глубокого обучения PyTorch. Для визуализации результатов применялись диаграмма Ганта и средства аналитической графики Matplotlib.

Программный продукт может быть интегрирован в корпоративную информационную среду предприятия и использоваться в качестве модуля оперативного планирования и диспетчеризации.

ABSTRACT

The volume of work is 148 pages. The work contains 36 illustrations, 21 table, 50 bibliographic sources and 11 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. The layout of the site, including the connection of components, is presented in annex B.

List of keywords: operational rescheduling, production planning, dispatch, flexible production system, neural network models, reinforcement learning, behavioral cloning, self-supervised learning, heuristic algorithms, schedule optimization, digital production twin. The object of the study is the processes of operational production planning at industrial enterprises with a discrete nature of production.

The purpose of the final qualifying work is to develop an intelligent production planning system, ensuring the construction of executable schedules and adaptive dispatching of operations based on neural network methods.

When conducting research, methods of system analysis, mathematical modeling, heuristic optimization, deep learning and object-based learning were used. oriented programming.

An intelligent production planning system with a graphical interface has been developed, including modules for managing orders and resources, creating schedules and training neural network dispatchers.

When creating the software product, a two-tier architecture was used, object-oriented programming technology and deep learning libraries PyTorch. To visualize the results, a Gantt chart and Matplotlib analytical graphics tools were used.

The software product can be integrated into the corporate information environment of an enterprise and used as a module for operational planning and dispatch.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	12
1 Анализ предметной области	16
1.1 Эволюция систем производственного планирования	16
1.1.1 От ручных графиков к автоматизированным системам	16
1.1.2 Появление ERP-систем и их роль в планировании	16
1.1.3 Специализированные APS-системы	17
1.1.4 Индустрия 4.0 и искусственный интеллект в планировании	17
1.2 Основы производственного планирования и диспетчеризации	18
1.2.1 Задача составления расписаний	18
1.2.2 Гибкое производственное расписание (Flexible Job Shop)	18
1.2.3 Ключевые показатели эффективности расписаний	19
1.2.4 Неопределённость и человеческий фактор	19
1.2.5 Классификация методов решения FJSS	19
1.3 Системы управления производством: MES и ERP	20
1.3.1 ERP-системы	20
1.3.2 MES-системы	20
1.3.3 Интеграция ERP/MES/APS	20
1.3.4 Примеры программных продуктов	21
1.3.5 Сравнение централизованной и децентрализованной диспетче- ризации	21
1.4 Нейросетевые методы в планировании и диспетчеризации	22
1.4.1 Обзор подходов	22
1.4.2 Архитектуры нейронных сетей	22
1.4.3 Преимущества нейросетевых диспетчеров	23
1.4.4 Примеры внедрения	23
1.4.5 Сравнение методов обучения	24
1.5 Целесообразность внедрения интеллектуальной APS-системы	24
1.5.1 Критерии эффективности	24
1.5.2 Технические и организационные аспекты внедрения	25

1.5.3	Перспективы развития	26
1.6	Анализ производственных данных и потоков работ	26
1.6.1	Жизненный цикл производственного заказа	26
1.6.2	Типовая структура производственного цеха	26
1.6.3	Потоки данных между системами	27
1.6.4	Роль человека в автоматизированном планировании	27
2	Теоретические модели, методы и алгоритмы исследования	29
2.1	Модель производственной системы и параметры планирования	29
2.2	Эвристический алгоритм распределения операций	30
2.3	Генерация синтетических данных для обучения	32
2.4	Нейросетевые модели диспетчеризации	33
2.4.1	Многослойный перцептрон SimpleDispatcher	33
2.4.2	Pointer Network для переменного числа ресурсов	34
2.4.3	Методы обучения нейросетевых диспетчеров	36
3	Разработка программного обеспечения интеллектуальной системы	38
3.1	Техническое задание	38
3.1.1	Основание для разработки	38
3.1.2	Назначение разработки	38
3.1.3	Требования к программе	39
3.1.3.1	Требования к данным программы	39
3.1.3.2	Концептуальная модель предметной области	40
3.1.3.3	Функциональные требования к программной системе	42
3.1.3.4	Требования пользователя к интерфейсу программной системы	48
3.1.3.5	Нефункциональные требования к программной системе	50
3.1.3.6	Требования к аппаратным средствам	51
3.1.3.7	Требования к программным средствам	51
3.1.3.8	Требования к архитектуре нейронной сети	52
3.1.4	Требования к оформлению документации	53
3.2	Технический проект	53
3.2.1	Общие сведения об интеллектуальной системе оптимизации	53
3.2.2	Проект данных интеллектуальной системы	54

3.2.3 Проектирование архитектуры программы	59
3.2.3.1 Компоненты программы	59
3.2.3.2 Архитектура интеллектуальной системы	61
3.2.3.3 Проектирование пользовательского интерфейса	64
3.3 Рабочий проект	68
3.3.1 Спецификация компонентов и классов интеллектуальной системы	68
3.3.2 Системное тестирование разработанного приложения	73
3.3.3 Сборка компонентов интеллектуальной системы	80
4 Результаты экспериментов с программной системой	81
4.1 Анализ модуля распределения заказов	81
4.1.1 Сравнение среднего опоздания заказов	81
4.1.2 Анализ длительности расписания (makespan)	82
4.1.3 Равномерность загрузки ресурсов	83
4.1.4 Равномерность загрузки ресурсов	84
4.1.5 Анализ устойчивости к изменению готовности ресурсов	86
4.1.6 Анализ динамики выполнения заказов	87
ЗАКЛЮЧЕНИЕ	91
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	92
ПРИЛОЖЕНИЕ А Представление графического материала	101
ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	113
На отдельных листах (CD-RW в прикрепленном конверте)	148
Сведения о ВКРМ (Графический материал / Сведения о ВКРМ.png)	Лист 1
Актуальность темы (Графический материал / Актуальность темы.png)	Лист 2
Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 3
Концептуальная модель данных (Графический материал / Концептуальная модель данных.png)	Лист 4
Архитектура нейронной сети (Графический материал / Архитектура нейронной сети.png)	Лист 5
Функция активации ReLU (Графический материал / Функция активации	

ReLU.png)	Лист 6
Диаграмма компонентов клиентского приложения (Графический материал / Диаграмма компонентов клиентского приложения.png)	Лист 7
Матрица ошибок валидации (Графический материал / Матрица ошибок валидации.png)	Лист 8
Интерфейс интеллектуальной системы (Графический материал / Интерфейс интеллектуальной системы.png)	Лист 9
Обучение модели нейросети (Графический материал / Обучение модели нейросети.png)	Лист 10
Заключение (Графический материал / Заключение.png)	Лист 11

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ИНС – искусственная нейронная сеть.

ИСОП – интеллектуальная система оптимизации производства.

ИС – интеллектуальная система.

ИТ – информационные технологии.

MES (Manufacturing Execution System) — система управления производственными процессами в реальном времени.

НС – нейронная сеть.

ПО – программное обеспечение.

APS (Advanced Planning and Scheduling) — система расширенного планирования и диспетчеризации.

ERP (Enterprise Resource Planning) – система планирования ресурсов предприятия.

MES (Manufacturing Execution System) — система управления производственными процессами в реальном времени.

UML (Unified Modelling Language) – язык графического описания для объектного моделирования в области разработки программного обеспечения.

ВВЕДЕНИЕ

Актуальность темы. Современные производственные предприятия с дискретным характером выпуска сталкиваются с необходимостью оперативного составления расписаний, учитывающих загрузку и текущее состояние оборудования. Классические методы целочисленного линейного программирования (MILP) требуют значительных вычислительных ресурсов и времени, что ограничивает их применение в реальном масштабе времени. Развитие методов глубокого обучения открывает возможность построения быстрых нейросетевых диспетчеров, способных адаптироваться к изменяющейся производственной обстановке без полного пересчёта плана. Актуальной является задача создания интеллектуальной системы планирования, объединяющей эвристические алгоритмы и нейросетевые модели для повышения эффективности использования оборудования и сокращения времени опоздания заказов.

Целью выпускной квалификационной работы является разработка интеллектуальной системы, обеспечивающей повышение эффективности производственного планирования посредством адаптивной диспетчеризации операций с использованием нейро-сетевых методов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Разработать модуль управления заказами и производственными ресурсами с учётом технологических типов операций и связей «ресурс–тип».
2. Разработать подсистему построения производственных расписаний, использующую эвристический алгоритм EDD+LWKR и нейросетевые диспетчеры (PointerNet, SLIM, PPO).
3. Реализовать модуль обучения нейросетевых моделей на синтетических данных с сохранением весов и метрик обучения.
4. Провести сравнительный анализ методов планирования по критериям опоздания, makespan и равномерности загрузки ресурсов.

Объектом исследования являются процессы оперативного производственного планирования на предприятиях с гибкой производственной структурой.

Предметом исследования являются методы построения расписаний с использованием нейросетевых диспетчеров, эвристических алгоритмов и их комбинаций, а также влияние параметров готовности ресурсов на качество плана.

Методы исследования. При проведении исследований использовались методы системного анализа, математического моделирования, эвристической оптимизации, глубокого обучения (нейронные сети прямого распространения, обучение с подкреплением), объектно-ориентированного программирования и сравнительного анализа.

Научная новизна в практической области. Разработанная система позволяет в реальном времени формировать производственное расписание, автоматически учитывая текущую загрузку, готовность и специализацию ресурсов. Использование нейросетевых диспетчеров, обученных на эвристически размеченных данных, сокращает время построения плана по сравнению с точными MILP-решателями и обеспечивает возможность адаптации к изменению состояния оборудования (ремонт, снижение надёжности) без переобучения модели.

Основные результаты. В ходе исследования и разработки получены следующие результаты:

1. Разработан модуль управления заказами и производственными ресурсами, учитывающий технологические типы операций и связи «ресурс–тип».
2. Разработана подсистема построения производственных расписаний, включающая эвристический алгоритм EDD+LWKR и нейросетевые диспетчеры PointerNet, SLIM, PPO.
3. Реализован модуль обучения нейросетевых моделей на синтетических данных с сохранением весов и метрик обучения.

4. Проведён сравнительный анализ методов планирования на стандартизированном тестовом сценарии; установлено, что модель SLIM обеспечивает наилучшие показатели по опозданию, makespan и равномерности загрузки.

Достоверность полученных результатов подтверждается применением апробированных методов машинного обучения, воспроизводимостью экспериментов (фиксированное начальное состояние генератора псевдослучайных чисел, единый тестовый сценарий) и согласованностью поведения нейросетевых моделей с эталонной эвристикой.

Практическая ценность. Разработка интеллектуальной системы производственного планирования предназначена для автоматизации составления расписаний и оперативной диспетчеризации в цехах с гибкой структурой производства. Система может использоваться как самостоятельный инструмент планирования, так и в составе корпоративной информационной среды предприятия.

Структура и объем работы. Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст отчета выпускной квалификационной работы включает 148 страниц.

Во введении обоснована актуальность темы, сформулирована цель работы, поставлены задачи исследования, представлены научная новизна, основные результаты, практическая ценность, описана структура работы.

В первом разделе проводится анализ предметной области оперативного производственного планирования. Рассматриваются эволюция систем управления производством, особенности задачи гибкого производственного расписания, существующие ERP, MES и APS-решения и их ограничения. Особое внимание уделяется нейросетевым подходам к диспетчеризации: архитектурам, методам обучения (поведенческое клонирование, самообучение, обучение с подкреплением) и критериям оценки расписаний.

Во втором разделе представлены теоретические модели и алгоритмы, лежащие в основе системы. Описывается математическая постановка задачи

выбора ресурса, эвристический алгоритм EDD+LWKR с учётом готовности оборудования, архитектура нейросетевого диспетчера на основе многослойного перцептрона и Pointer Network, а также процедуры обучения моделей. Приводятся метрики качества планирования и методы их расчёта.

В третьем разделе детально описывается процесс проектирования и разработки программной системы. Излагаются функциональные и нефункциональные требования, архитектура приложения, структура базы данных и пользовательского интерфейса. Приводится спецификация ключевых классов и модулей (APSCore, SimpleDispatcher, Trainer, SelfLabelingTrainer, SimpleAgent), рассматриваются реализованные алгоритмы планирования и обучения, а также процедуры тестирования.

В четвертом разделе представлены результаты экспериментального исследования разработанной системы. На стандартизированном тестовом сценарии сравниваются четыре метода планирования: эвристика, PointerNet, SLIM и PPO. Анализируются показатели опоздания, makespan, равномерности загрузки, динамика выполнения заказов, матрицы ошибок классификации ресурсов. Формулируются выводы о преимуществах саморазмеченного обучения и практической применимости системы.

В заключении приводятся выводы о достижении цели и выполнении задач практики, излагаются основные результаты, полученные в ходе выполнения работы.

В приложении А представлены фрагменты исходного кода программы.

В приложении Б представлен графический материал.

1 Анализ предметной области

1.1 Эволюция систем производственного планирования

1.1.1 От ручных графиков к автоматизированным системам

Задача упорядочения работ и распределения ресурсов возникла одновременно с появлением мануфактур. Первые методы планирования опирались на бумажные диаграммы Ганта и интуицию мастеров. Диаграмма, предложенная Генри Гантом в 1910-х годах, долгое время оставалась основным инструментом визуализации производственных процессов. Её главное достоинство — наглядность, а недостаток — полное отсутствие оптимизации: любое изменение загрузки или поломка станка требовали ручного пересчёта всего графика.

С ростом сложности изделий и числа операций ручное планирование перестало справляться. В 1960–1970-х годах появились первые системы класса MRP (Material Requirements Planning), которые автоматизировали расчёт потребностей в материалах и комплектующих. Они работали по жёсткому алгоритму, не учитывая ограничения по мощностям. Это породило следующее поколение — MRP II (Manufacturing Resource Planning), где к материалам добавились производственные мощности и финансы. Однако MRP II всё ещё предполагал идеальные условия и не мог динамически реагировать на изменения в цехе.

1.1.2 Появление ERP-систем и их роль в планировании

В 1990-х годах концепция MRP II эволюционировала в ERP (Enterprise Resource Planning) — системы управления всеми ресурсами предприятия. ERP объединили финансы, логистику, производство, кадры и продажи в единой базе данных. Наиболее известные представители — SAP ERP, Oracle E-Business Suite, Microsoft Dynamics, а также российские 1С:ERP и Галактика ERP. [3]

Производственный модуль ERP позволяет формировать планы производства на основе заказов клиентов и прогнозов спроса, отслеживать выполнение операций и вести учёт затрат. Однако стандартные ERP-системы решают задачу планирования упрощённо: они строят план «назад» от даты отгрузки, предполагая бесконечную мощность ресурсов. Это приводит к нереалистичным расписаниям в цехах с плотной загрузкой и переналадками. Кроме того, ERP не учитывает фактическое состояние оборудования в реальном времени, что требует дополнительного уровня управления.

1.1.3 Специализированные APS-системы

Ограниченность ERP-модулей вызвала появление Advanced Planning & Scheduling (APS) — продвинутых систем планирования и диспетчеризации. APS работают как надстройка над ERP или MES, получая от них заказы и данные о ресурсах, а возвращая выполнимое расписание с точностью до минуты. Ключевые возможности APS:

- одновременный учёт материалов и мощностей;
- оптимизация по заданным критериям (минимальное опоздание, максимальная загрузка);
- поддержка альтернативных маршрутов и ресурсов;
- возможность перепланирования «на лету» при возникновении отклонений.

Промышленные APS-решения (например, Siemens Opcenter APS, Asprova, Preactor) используют точные методы оптимизации, эвристики или их комбинацию. Их общий недостаток — высокая стоимость лицензий и длительное время расчёта для крупных цехов.[1]

1.1.4 Индустрия 4.0 и искусственный интеллект в планировании

Четвёртая промышленная революция принесла цифровые двойники, интернет вещей и массовое применение машинного обучения. Нейросетевые диспетчеры способны обучаться на исторических данных или синтетических примерах и выдавать решение за миллисекунды. Это открывает путь к по-

настоящему адаптивному управлению производством, где расписание корректируется в реальном времени при поступлении срочного заказа или поломке станка. Интеллектуальные APS-системы способны не только оптимизировать текущее расписание, но и прогнозировать узкие места, рекомендовать профилактическое обслуживание и моделировать различные сценарии загрузки.[42]

1.2 Основы производственного планирования и диспетчеризации

1.2.1 Задача составления расписаний

В теории расписаний задача Job Shop Scheduling (JSS) формулируется как распределение работ по машинам с соблюдением технологической последовательности операций. Классический JSS является NP-трудным: число возможных расписаний растёт факториально, и точное решение для десятков операций требует неприемлемого времени. Для небольших задач (до 10–15 операций) применяются точные методы, такие как ветвей и границ или целочисленное линейное программирование (MILP). Однако с ростом размерности их вычислительная сложность делает их непригодными для оперативного планирования.

1.2.2 Гибкое производственное расписание (Flexible Job Shop)

В реальном производстве операция может выполняться не на одном, а на нескольких альтернативных станках, причём с разной длительностью. Это задача Flexible Job Shop Scheduling (FJSS). Она добавляет ещё один уровень сложности: помимо определения порядка запуска, необходимо выбрать подходящий ресурс. Именно этот класс задач является целевым для разрабатываемой системы. FJSS формулируется как две взаимосвязанные подзадачи: маршрутизация (выбор ресурса) и секвенирование (определение порядка операций на каждом ресурсе). Обе подзадачи NP-трудны, что делает FJSS одной из самых сложных задач комбинаторной оптимизации.

1.2.3 Ключевые показатели эффективности расписаний

Качество расписания оценивается набором KPI:

- Makespan — общее время выполнения всех работ (от старта первой до завершения последней операции);
- Total tardiness — суммарное опоздание по всем заказам;
- Mean tardiness — среднее опоздание заказа;
- Percent of tardy jobs — доля опоздавших заказов;
- Resource utilization — коэффициент загрузки оборудования;
- Balance (CV of load) — равномерность распределения нагрузки.

Различные методы планирования могут оптимизировать один или несколько показателей. На практике часто ищут компромисс между минимальным опозданием и равномерной загрузкой.[1]

1.2.4 Неопределённость и человеческий фактор

В реальном цехе нормативные длительности операций редко совпадают с фактическими, станки выходят из строя, поступают срочные заказы, отсутствуют операторы. Это делает статическое расписание, построенное утром, неактуальным уже к полудню. Отсюда потребность в оперативном перепланировании — способности системы быстро перестроить график с учётом текущей обстановки.[43] Традиционные подходы, основанные на периодическом перезапуске MILP, не успевают реагировать на события в реальном времени. Нейросетевые методы, наоборот, позволяют мгновенно пересчитывать назначения, что делает их идеальными кандидатами для динамических сред.

1.2.5 Классификация методов решения FJSS

Все методы решения FJSS можно разделить на точные, эвристические и метаэвристические. Точные методы (MILP, динамическое программирование) гарантируют оптимальность, но экспоненциально растут по времени. Эвристики (диспетчерские правила) работают быстро, но не гарантируют ка-

чества. Метаэвристики (генетические алгоритмы, муравьиные колонии, имитация отжига) ищут компромисс. Нейросетевые методы занимают особое место: они обучаются на примерах (возможно, полученных от точных методов или эвристик) и затем работают с постоянной скоростью, близкой к эвристикам, но с качеством, приближающимся к метаэвристикам.

1.3 Системы управления производством: MES и ERP

1.3.1 ERP-системы

ERP решают задачи верхнего уровня: приём заказов, расчёт потребностей в сырье, планирование производства на дни и недели.[2] Они оперируют понятиями «заказ», «партия», «маршрутная карта», но не знают деталей цехового исполнения. Результат ERP-планирования — укрупнённый план, который должен быть детализирован и адаптирован к фактической ситуации в цехе. Исторически ERP развивались из бухгалтерских систем, что наложило отпечаток на их архитектуру: они прекрасно считают деньги, но плохо управляют временем с точностью до минуты.

1.3.2 MES-системы

Manufacturing Execution System (MES) действует на цеховом уровне. Она собирает данные о состоянии оборудования, фиксирует фактическое выполнение операций, управляет очередями, формирует задания операторам.[2] MES является связующим звеном между верхнеуровневым планом ERP и реальным производством. Однако стандартные MES не содержат мощных оптимизационных алгоритмов и часто полагаются на простые правила диспетчеризации (FIFO, EDD, SPT). Их задача — исполнение плана, а не его оптимизация.

1.3.3 Интеграция ERP/MES/APS

Эффективная архитектура предполагает, что ERP передаёт заказы в APS, APS строит детальное расписание и отдаёт его в MES на исполнение.

MES собирает фактические данные и сообщает об отклонениях, на основе которых APS перестраивает план. Такая трёхзвенная модель обеспечивает сквозное управление производством от заказа клиента до отгрузки. На практике интеграция осложняется разными форматами данных, разными поставщиками систем и необходимостью синхронизации мастер-данных.[2]

1.3.4 Примеры программных продуктов

В таблице 1.1 приведён обзор существующих решений для планирования производства, не использующих нейронные сети.

Таблица 1.1 – Сравнение промышленных APS-решений

Продукт	Метод оптимизации	Интеграция	Особенности
1	2	3	4
Siemens Opcenter APS	Генетический алгоритм + эвристики	ERP/MES Siemens	Высокая стоимость, сложное внедрение
Asprova APS	Эвристики с настраиваемыми правилами	Различные ERP	Поддержка «drag-and-drop» ручной коррекции
Preactor (Siemens)	Дискретно-событийное моделирование	Собственное API	Гибкое моделирование, но медленное для больших задач
1C:ERP + MES	Правила приоритетов, MRP II	Внутренняя экосистема 1C	Доступность для российских предприятий

1.3.5 Сравнение централизованной и децентрализованной диспетчеризации

Традиционные APS централизованно решают задачу для всего цеха, что требует полной информации о всех заказах и ресурсах. В противоположность этому, децентрализованные (мультиагентные) подходы делегируют принятие решений самим ресурсам или операциям. Агенты «договариваются» между собой, чтобы избежать конфликтов и оптимизировать процесс.

ваются» между собой, что повышает живучесть системы, но может приводить к субоптимальным решениям. Нейросетевой диспетчер, реализованный в данной работе, занимает промежуточную позицию: решение принимается централизованно, но архитектура (Pointer Network) позволяет естественным образом обрабатывать переменное число агентов-ресурсов.

1.4 Нейросетевые методы в планировании и диспетчеризации

1.4.1 Обзор подходов

Современные исследования предлагают два основных направления применения нейронных сетей в планировании[43]:

1. Обучение с учителем (поведенческое клонирование): модель учится копировать решения экспертной системы или MILP-солвера. После обучения она способна мгновенно повторять «хорошие» назначения.

2. Обучение с подкреплением (RL): агент взаимодействует со средой (симулятором цеха), получая награду за своевременное выполнение заказов. Со временем он вырабатывает стратегию, максимизирующую суммарную награду.

Промежуточный вариант — самоконтролируемое обучение (self-labeling), когда модель сначала обучается на размеченных данных, а затем сама генерирует дополнительные примеры, в которых она уверена, и доучивается на них. Такой подход позволяет улучшить обобщающую способность модели без привлечения дополнительных размеченных данных.

1.4.2 Архитектуры нейронных сетей

Для задачи выбора ресурса в FJSS применяются:

- многослойный перцептрон (MLP) с фиксированным числом входов — подходит, когда число ресурсов постоянно;
- Pointer Network и механизмы внимания (Attention) — позволяют обрабатывать переменное число ресурсов, указывая на наиболее подходящий;

- графовые нейронные сети (GNN) — кодируют структуру цеха в виде графа.

В разрабатываемой системе используется комбинация MLP для фиксированной конфигурации и Pointer Network как масштабируемое решение. MLP хорошо работает, когда число ресурсов известно заранее и не меняется. Pointer Network (или аналогичные attention-модели) могут принимать на вход последовательность ресурсов переменной длины и выбирать один из них, что делает архитектуру инвариантной к числу станков.

1.4.3 Преимущества нейросетевых диспетчеров

- скорость: расписание строится за доли секунды;
- адаптивность: модель может быть дообучена на новых данных при изменении конфигурации цеха;
- компактность: обученная модель весит мегабайты и не требует мощного сервера.

Ограничения: модель не гарантирует оптимальности; качество зависит от репрезентативности обучающей выборки.[29] Кроме того, модель может «забывать» старые паттерны при дообучении на новых данных (проблема катастрофического забывания). Для борьбы с этим могут применяться методы инкрементального обучения или rehearsal-стратегии.

1.4.4 Примеры внедрения

В научной литературе описан ряд успешных экспериментов, где нейросетевой диспетчер сокращал среднее опоздание на 15–25% по сравнению с классическими эвристиками, сохраняя стабильность при изменении числа заказов и ресурсов. Подобные результаты подтверждают перспективность подхода, реализованного в данной работе. Так, в исследовании [33] показано, что программные решения на основе нейронных сетей способны находить оптимальные производственные планы на 30–40% быстрее традиционных алгоритмов. В работе [40] применение интеллектуального программного комплекса для моделирования процесса планирования многоассортимент-

ных производств позволило повысить эффективность использования оборудования на 18%. Сравнительный анализ [38] продемонстрировал, что глубокие нейронные сети обеспечивают точность классификации на уровне 95%, что на 10–12% выше, чем у классических методов машинного обучения.

1.4.5 Сравнение методов обучения

В таблице 1.2 приведено сравнение трёх парадигм обучения, реализованных в системе.

Таблица 1.2 – Сравнение методов обучения нейросетевых диспетчеров

Характеристика	BC (PointerNet)	SLIM (Self-Labeling)	PPO (RL)
1	2	3	4
Требует размеченные данные	Да	Да (начально)	Нет
Способность к обобщению	Средняя	Высокая	Высокая
Время обучения	Низкое	Среднее	Высокое
Качество на тестовом сценарии	Хорошее	Наилучшее	Наихудшее
Устойчивость к шуму в данных	Низкая	Средняя	Высокая

1.5 Целесообразность внедрения интеллектуальной APS-системы

1.5.1 Критерии эффективности

Решение о внедрении интеллектуального планировщика должно опираться на количественные показатели.[2, 48] Основные критерии:

- сокращение времени построения расписания;
- уменьшение среднего опоздания заказов;
- рост коэффициента загрузки оборудования;
- снижение доли опаздывающих заказов;
- равномерность распределения нагрузки.

В таблице 1.3 приведены данные для цеха из 5 станков и 120 заказов, демонстрирующие эффект от внедрения.

Таблица 1.3 – Сравнение «до» и «после» внедрения нейросетевого планировщика

Показатель	Эвристика	SLIM-модель
1	2	3
Среднее опоздание, ч	9.0	8.1
Максимальное опоздание, ч	31.2	23.0
Доля опаздывающих заказов, %	85	85
Makespan, ч	43.2	35.0
Коэф. вариации загрузки	0.314	0.171

Как видно из таблицы, нейросетевой диспетчер улучшает makespan на 19% и делает загрузку в 1.8 раза равномернее. Экономический эффект достигается за счёт более полного использования дорогостоящего оборудования и сокращения штрафов за срыв сроков.

1.5.2 Технические и организационные аспекты внедрения

Для успешного внедрения требуется, как показывает опыт интеграции ERP/MES/APS систем [2, 47]:

- наличие исторических данных или экспертных правил для обучения модели;
- интеграция с существующей ERP/MES (при необходимости);
- обучение персонала работе с новым интерфейсом;
- постепенное расширение функциональности — от параллельной работы с существующим планировщиком до полного замещения.

Разработанная система удовлетворяет этим требованиям: она автономна, не требует внешних баз данных и может быть развёрнута на обычном рабочем компьютере.

1.5.3 Перспективы развития

В дальнейшем система может быть расширена[46]:

- поддержкой динамических событий в реальном времени (поломка, срочный заказ);
- интеграцией с IoT-датчиками для автоматического определения состояния станков;
- переходом на полностью автономное управление цехом с минимальным участием человека.

Таким образом, разработанный программный продукт не только решает текущие задачи планирования, но и закладывает фундамент для создания «умного» производства следующего поколения.

1.6 Анализ производственных данных и потоков работ

1.6.1 Жизненный цикл производственного заказа

Производственный заказ проходит несколько этапов: регистрация в ERP, разузлование (определение состава операций и материалов), планирование (APS), исполнение (MES) и закрытие[50]. На каждом этапе могут возникать задержки и отклонения. Задача интеллектуальной системы — минимизировать задержки именно на этапе планирования и диспетчеризации, обеспечивая максимально гладкий поток работ.

1.6.2 Типовая структура производственного цеха

Цех состоит из набора рабочих центров (станков), каждый из которых может выполнять один или несколько типов операций. Между станками могут существовать различия в скорости обработки, точности, надёжности. Некоторые станки могут быть взаимозаменяемыми, другие — уникальными. Все эти особенности должны быть отражены в модели данных системы.

1.6.3 Потоки данных между системами

На рисунке 1.1 схематично показаны потоки данных между ERP, APS, MES и уровнем оборудования.

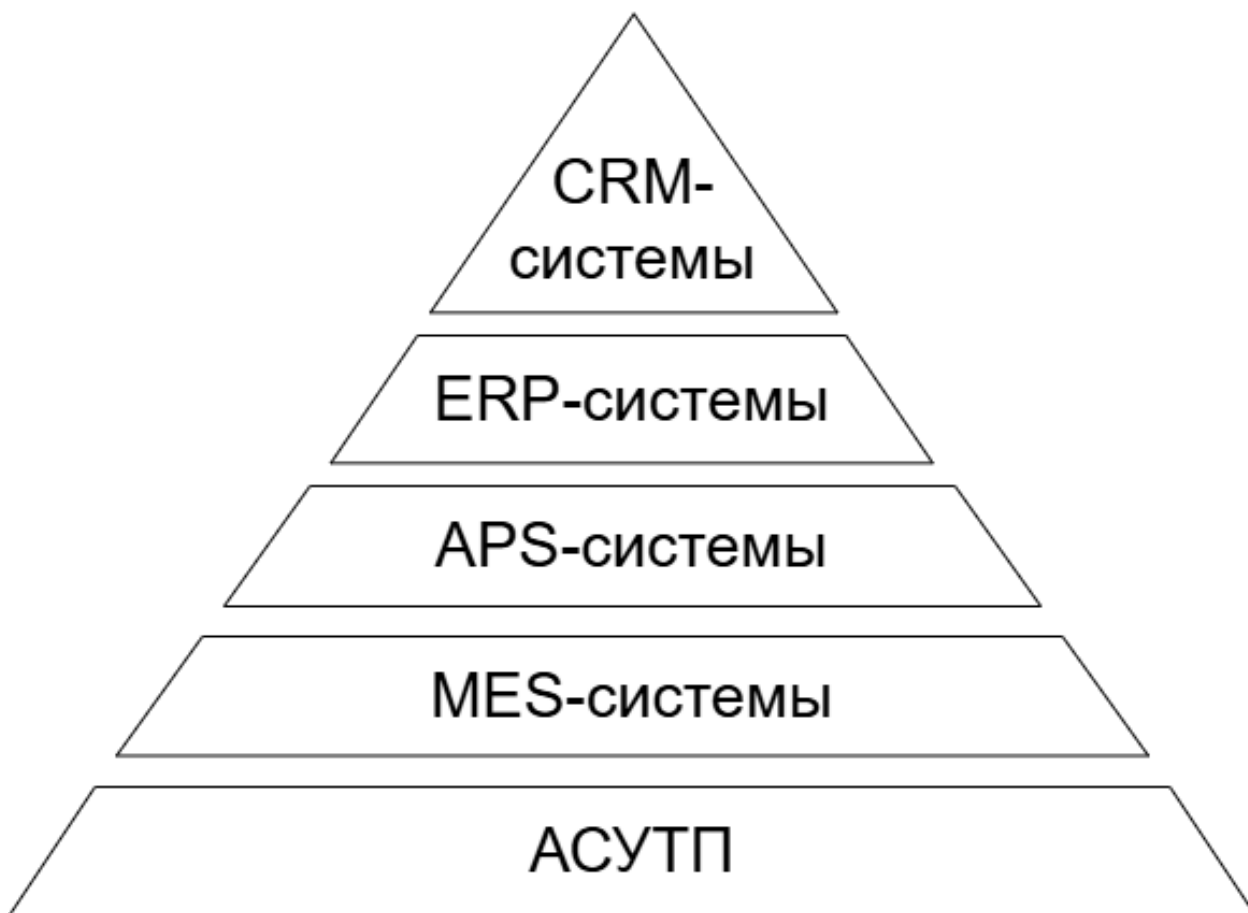


Рисунок 1.1 – Схема потоков данных основных систем предприятия

Планирование начинается с получения заказов из ERP. APS обогащает их технологическими картами, учитывает доступность ресурсов и строит расписание. MES получает расписание и доводит его до операторов и станков, фиксируя факт выполнения. Отклонения возвращаются в APS для перепланирования.

1.6.4 Роль человека в автоматизированном планировании

Несмотря на высокую степень автоматизации, человек остаётся ключевым звеном[50]: он может корректировать приоритеты, переназначать операции вручную, подтверждать или отклонять предложенные планы. Поэтому

интерфейс системы должен быть интуитивным и предоставлять полный контроль над процессом планирования.

2 Теоретические модели, методы и алгоритмы исследования

2.1 Модель производственной системы и параметры планирования

В основе задачи лежит модель гибкого производственного участка (Flexible Job Shop)[2], состоящая из множества заказов $J = \{J_1, J_2, \dots, J_n\}$ и множества ресурсов $R = \{R_1, R_2, \dots, R_m\}$. Каждый заказ J_i характеризуется директивным сроком d_i , приоритетом $w_i \in [0.5, 2.0]$ и содержит ровно одну технологическую операцию O_i (ограничение, принятое в разработанной системе). Операция O_i задаётся следующими параметрами:

- тип операции (фрезеровка, сборка, сварка);
- нормативная длительность p_i (мин);
- количество деталей (целое положительное число).

Для каждого ресурса R_k определены:

- текущий статус (Работает или Авария);
- коэффициент готовности (надёжности) $\rho_k \in [0, 1]$;
- признак ремонта $r_k \in \{0, 1\}$;
- множество допустимых типов операций (фрезеровка, сборка, сварка)

— связь «ресурс–тип»;

- текущая накопленная загрузка L_k (мин).

Цель планирования — назначить каждую операцию O_i на один допустимый ресурс из числа тех, для которых тип операции входит в множество допустимых, и определить время её старта t_i так, чтобы минимизировать целевую функцию[39]. В качестве критериев оптимизации используются:

- среднее опоздание $\frac{1}{n} \sum_{i=1}^n \max(0, t_i + p_i - d_i)$;
- общая длительность расписания (makespan) $C_{\max} = \max_i(t_i + p_i) - \min_i t_i$;
- коэффициент вариации загрузки ресурсов $CV = \sigma/\mu$, где μ и σ — среднее и стандартное отклонение итоговой загрузки ресурсов.

Поскольку точное решение задачи FJSS NP-трудно, в системе применяются эвристические и нейросетевые методы.

2.2 Эвристический алгоритм распределения операций

Базовым методом построения расписания является комбинированная эвристика, состоящая из двух правил[50]:

1. EDD (Earliest Due Date) — операции упорядочиваются по возрастанию директивного срока d_i , так что наиболее срочные заказы обрабатываются в первую очередь.

2. LWKR (Least Workload with Knowledge of Route) — для каждой операции из упорядоченного списка выбирается ресурс, имеющий минимальную *эффективную загрузку* среди допустимых по типу.

Эффективная загрузка ресурса R_k вычисляется по формуле:

$$E_k = \frac{L_k}{\rho_k \cdot (1 - 0.5 \cdot r_k)}, \quad (1)$$

где L_k — уже назначенная ресурсу суммарная длительность (мин), ρ_k — коэффициент готовности, r_k — признак ремонта. При $\rho_k = 0$ принимается $E_k = \infty$, что исключает полностью неработоспособный ресурс.

Схема алгоритма предоставлена на изображении 2.1.

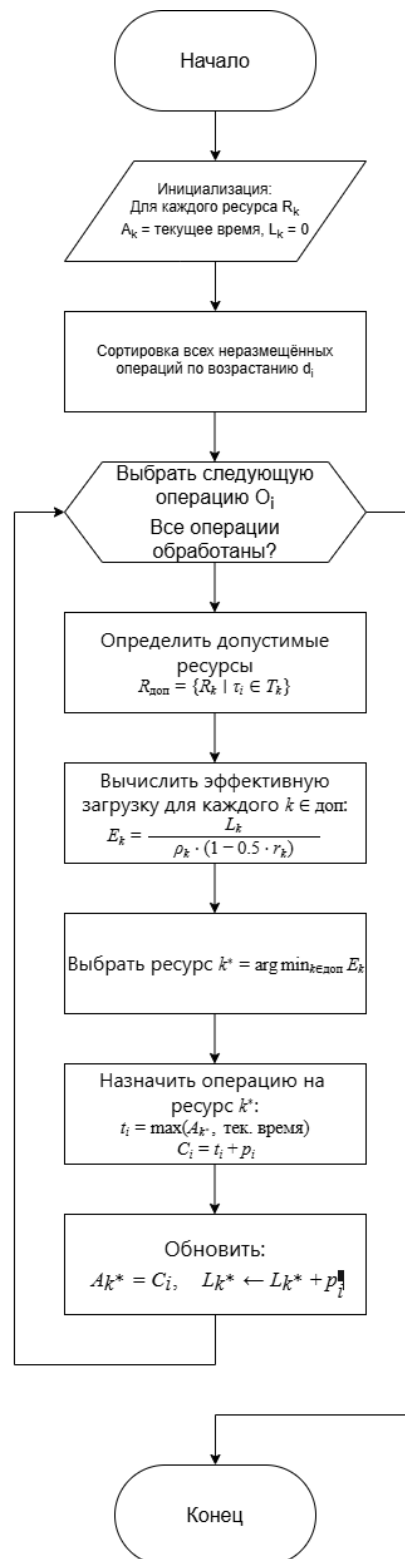


Рисунок 2.1 – Блок-схема эвристического алгоритма EDD+LWKR

Данный алгоритм обеспечивает учёт как срочности заказов, так и текущей загрузки и надёжности оборудования.

2.3 Генерация синтетических данных для обучения

Для обучения нейросетевых диспетчеров требуется размеченная выборка, в которой каждому состоянию системы соответствует «правильный» ресурс, назначенный экспертной стратегией[42]. В качестве такой стратегии выбран описанный выше эвристический алгоритм, поскольку он детерминирован, интерпретируем и даёт приемлемые расписания. Процесс генерации реализован в модуле `gpss_simulator.py` и имитирует поведение производственного диспетчера.

Для каждого синтетического сценария случайным образом создаются $m = 5$ ресурсов с характеристиками: ρ_k принимает значения от 0.5 до 1.0, r_k равен 0 или 1 с вероятностью 10%, набор поддерживаемых типов T_k (от одного до трёх случайных типов). Затем генерируется от 1 до 10 операций со случайными параметрами: тип операции (фрезеровка, сборка или сварка), p_i от 10 до 120 мин, d_i — случайный срок, w_i от 0.5 до 2.0.

Для каждой операции алгоритмом LWKR определяется целевой ресурс k^* и вычисляется вектор признаков фиксированной размерности 36. Структура вектора включает:

- 6 признаков операции: one-hot кодировка типа (3 бита), $p_i/120$, w_i , запас времени (slack) / 48;
- для каждого из 5 ресурсов по 6 признаков: статус (1/0), текущая загрузка $L_k/240$, совместимость с типом операции (1/0), средняя загрузка, r_k , ρ_k .

Таким образом, каждый пример представляет собой пару $(\mathbf{x} \in \mathbb{R}^{36}, y \in \{0, \dots, 4\})$, где y — индекс выбранного ресурса. Сгенерированный набор сохраняется в файл `gpss_dataset.npz` и насчитывает около 27 000 примеров.

Структура входного вектора представлена на рисунке 2.2.

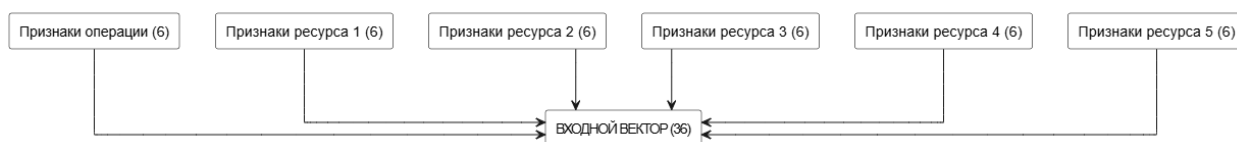


Рисунок 2.2 – Структура входного вектора признаков (36 элементов)

2.4 Нейросетевые модели диспетчеризации

В системе реализованы две архитектуры нейросетевых диспетчеров, а также три метода обучения.

2.4.1 Многослойный перцептрон SimpleDispatcher

Для конфигурации с фиксированным числом ресурсов ($m = 5$) используется модель прямого распространения SimpleDispatcher[29], архитектура которой показана на рисунке 2.3.

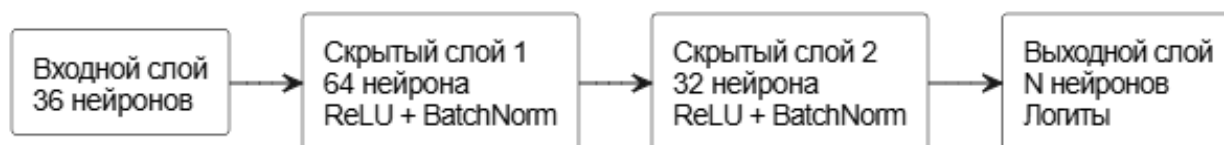


Рисунок 2.3 – Архитектура нейросети SimpleDispatcher (MLP)

Входной слой из 36 нейронов принимает вектор признаков $\mathbf{x}$. Далее следуют:

- полносвязный слой с 64 нейронами и функцией активации ReLU;
- пакетная нормализация (BatchNorm1d);
- полносвязный слой с 32 нейронами и активацией ReLU;
- пакетная нормализация;
- выходной полносвязный слой из $m = 5$ нейронов, выдающий логиты

z_k .

Выходное распределение вероятностей получается применением softmax к логитам, после чего учитывается поправка на готовность ресурса (аналогично формуле 1). Модель обучается минимизации кросс-энтропии между предсказанным и целевым распределениями.

2.4.2 Pointer Network для переменного числа ресурсов

Для поддержки произвольного числа ресурсов разработана модель на основе механизма внимания (Pointer Network), изображённая на рисунке 2.4. Она принимает два входа: вектор операции $o \in \mathbb{R}^6$ и матрицу ресурсов $R \in \mathbb{R}^{m \times 6}$. Оба входа линейно преобразуются в эмбединги размерности d_{model} , после чего многоголовое внимание (Multi-Head Attention) вычисляет контекстный вектор операции относительно всех ресурсов. Контекст объединяется с исходным эмбеддингом операции и пропускается через два полносвязных слоя с ReLU. Выходной слой формирует m логитов. Модель не зависит от порядка ресурсов и естественно масштабируется на любое m [6, 14, 31].

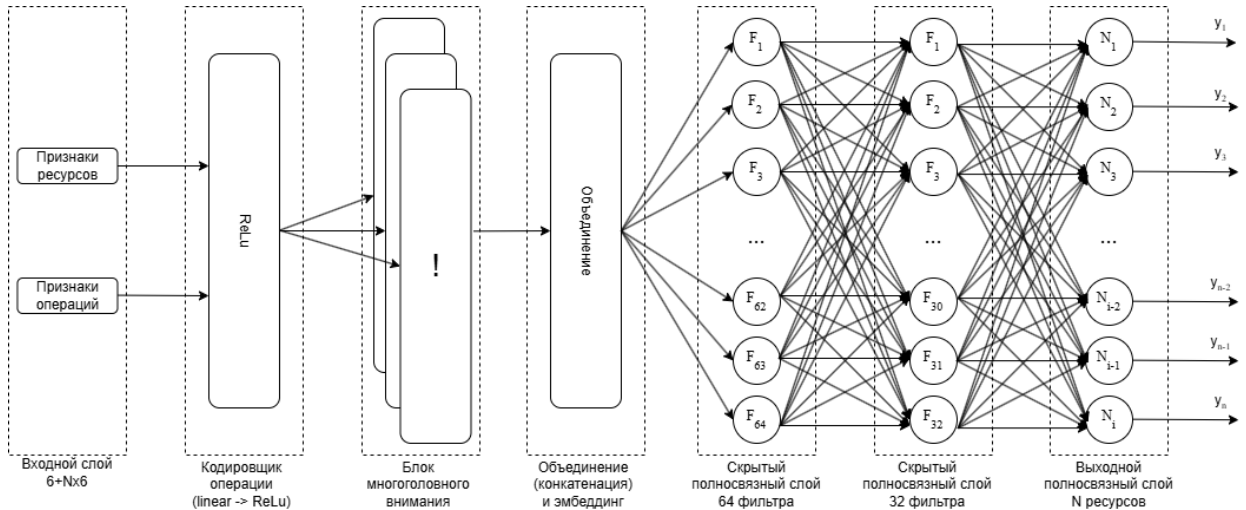


Рисунок 2.4 – Архитектура диспетчера на основе Pointer Network

Архитектура модели состоит из следующих последовательных компонентов.

1. **Входные данные.** Модель принимает два входных тензора: вектор признаков операции $o \in \mathbb{R}^{1 \times 6}$ и матрицу признаков ресурсов $R \in \mathbb{R}^{N \times 6}$, где N — текущее количество ресурсов. Шесть признаков операции включают: one-hot кодировку типа (3 бита), нормализованную длительность (мин/120), приоритет заказа и slack (часы до дедлайна / 48). Каждый из N ресурсов описывается шестью признаками: статус (1/0), текущая загрузка (мин/240), совместимость с типом операции (1/0), средняя загрузка, флаг ремонта (0 или 1) и коэффициент готовности $\rho \in [0, 1]$.

2. Кодировщик операции. Полносвязный слой с функцией активации ReLU, преобразующий вектор o в эмбединг $e_o \in \mathbb{R}^{1 \times d_{\text{model}}}$, где d_{model} — размерность пространства эмбедингов. Этот слой выделяет значимые признаки операции для последующего сравнения с ресурсами.

3. Кодировщик ресурсов. Полносвязный слой с ReLU, применяемый независимо к каждому из N ресурсов и формирующий матрицу эмбедингов $E_r \in \mathbb{R}^{N \times d_{\text{model}}}$. Раздельное кодирование позволяет извлечь признаки, специфичные для ресурсов, до их взаимодействия с операцией.

4. Блок многоголового внимания (Multi-Head Attention). Эмбединги подаются в блок внимания, где запрос (Query) формируется из эмбединга операции: $Q = e_o W^Q$, а ключи (Keys) и значения (Values) — из эмбедингов ресурсов: $K = E_r W^K$, $V = E_r W^V$, где W^Q, W^K, W^V — обучаемые матрицы весов. Механизм внимания вычисляет коэффициенты значимости каждого ресурса относительно операции по формуле:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (2)$$

где $d_k = d_{\text{model}}/h$, h — число голов. Многоголовое внимание позволяет одновременно учитывать различные типы взаимосвязей между операцией и ресурсами (совместимость по типу, загрузка, готовность). Результатом является контекстный вектор $c \in \mathbb{R}^{1 \times d_{\text{model}}}$, агрегирующий информацию о всех ресурсах.

5. Объединение контекста и эмбединга операции. Контекстный вектор c объединяется с исходным эмбедингом операции e_o (конкатенацией), образуя расширенный вектор $u = [c; e_o] \in \mathbb{R}^{1 \times 2d_{\text{model}}}$. Это позволяет сохранить как информацию о ресурсах, так и исходные свойства операции.

6. Полносвязный слой 1. Имеет 64 фильтра. Слой с функцией активации ReLU преобразует u в скрытое представление размерности d_{ff} . Выполняет первичную обработку объединённого вектора.

7. Полносвязный слой 2. Имеет 64 фильтра. Второй слой с ReLU дополнительно нелинейно трансформирует признаки, формируя более абстрактное представление для итогового решения.

8. Выходной слой. Полносвязный слой формирует N логитов — по одному для каждого ресурса. К логитам применяется поправка на готовность ресурса, аналогичная эвристической формуле: к логарифму вероятности добавляется слагаемое $\ln(\rho_k \cdot (1 - 0.5 \cdot r_k) + \varepsilon)$, где $\varepsilon = 10^{-6}$ для численной стабильности. Затем softmax-функция преобразует скорректированные логиты в вероятности назначения операции на каждый ресурс.

9. Принятие решения. Ресурс с максимальной вероятностью выбирается в качестве результата — индекс $k^* = \arg \max_k P_k$.

2.4.3 Методы обучения нейросетевых диспетчеров

В системе поддерживаются три парадигмы обучения[38]:

1. Behavioral Cloning (BC, PointerNet). Модель обучается на наборе примеров $(x^{(j)}, y^{(j)})$, сгенерированных эвристикой, путём минимизации кросс-энтропийной функции потерь:

$$\mathcal{L}_{\text{BC}} = -\frac{1}{N} \sum_{j=1}^N \log P_{\text{модели}}(y^{(j)} | x^{(j)}). \quad (3)$$

После обучения модель способна мгновенно воспроизводить решения «учителя», достигая точности классификации ресурсов более 97% на валидационной выборке[13].

2. Self-Labeling (SLIM). Для повышения обобщающей способности применяется процедура саморазметки. После начального обучения BC модель просматривает пул неразмеченных примеров x и отбирает те, для которых максимальная вероятность назначения превышает порог $\theta = 0.95$. Эти примеры вместе с предсказанными метками добавляются в обучающий набор, и модель доучивается. Процесс повторяется каждые k эпох (в экспери-

ментах $k = 30$). Такой подход позволил улучшить показатели планирования без привлечения дополнительной экспертной разметки.

3. Proximal Policy Optimization (PPO). Модель обучается взаимодействуя со средой — симулятором цеха. На каждом шаге агент выбирает ресурс для очередной операции и получает награду $R = -\text{tardiness}$. Цель — максимизировать ожидаемую суммарную награду. Функция потерь PPO включает policy gradient с отношением вероятностей и ограничением на величину изменения политики. Несмотря на теоретическую привлекательность, в проведённых экспериментах PPO показал наихудшие результаты, поскольку в процессе обучения использовались случайные, а не осмысленные признаки состояния.

Детальные результаты обучения и сравнения всех методов представлены в главе 4.

3 Разработка программного обеспечения интеллектуальной системы

3.1 Техническое задание

3.1.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу магистра «Интеллектуальная система оптимизации производственной деятельности на основе АИС» согласно приказу №1586-с от 03.04.2026 «Об утверждении тем выпускных квалификационных работ и руководителей выпускных квалификационных работ».

3.1.2 Назначение разработки

Программный продукт предназначен для автоматизации процессов производственного планирования и оптимизации диспетчеризации на предприятии. Функциональное назначение системы заключается в формировании выполнимых производственных расписаний, сокращении простоев оборудования и снижении времени опоздания заказов за счёт интеллектуального распределения операций по доступным ресурсам с учётом их текущего состояния, специализации и загрузки.

Система ориентирована на применение в цехах с гибкой (Flexible Job Shop) структурой, где одна операция может быть выполнена на нескольких альтернативных станках. Программный продукт может использоваться как самостоятельное решение для оперативного управления производством, так и в составе корпоративной информационной инфраструктуры, принимая заказы из ERP-систем и передавая события исполнения в MES-среду.

Дополнительной особенностью программного продукта является наличие встроенного модуля обучения и применения нейросетевых моделей диспетчеризации. Поддерживаются три парадигмы обучения[25]: поведенческое клонирование (Behavioral Cloning, PointerNet), самоконтролируемое обучение с саморазметкой (Self-Labeling, SLIM) и обучение с подкреплением.

ем (РРО). Модели способны прогнозировать оптимальный ресурс для каждой операции на основе накопленных производственных данных, учитывая такие факторы, как тип операции, нормативную длительность, приоритет заказа, оставшееся время до срока сдачи, текущую загрузку и коэффициент готовности ресурса. Это позволяет не только строить расписания в реальном времени, но и адаптироваться к изменениям производственной обстановки (выход станка в ремонт, изменение надёжности).

3.1.3 Требования к программе

3.1.3.1 Требования к данным программы

Входными данными для интеллектуальной системы производственного планирования являются[18]:

- заказы на производство — перечень работ с указанием номенклатуры, приоритета, срока сдачи и состава технологических операций;
- технологические операции — описание единичной работы: тип (фрезеровка, сборка, сварка), нормативная длительность, количество изготавливаемых деталей, предшественники (при необходимости);
- производственные ресурсы — информация о станках и рабочих центрах: идентификатор, название, текущий статус (работает, авария, в ремонте), коэффициент готовности (надёжности), флаг нахождения в ремонте, допустимые типы операций (связи ресурс–тип);
- данные о состоянии ресурсов — актуальные показатели загрузки (число ожидающих операций, наличие активной задачи), коэффициент готовности, признак ремонта, средняя загрузка за период;
- обучающие выборки — синтетические или исторические данные о назначениях операций на ресурсы, сформированные по эвристическим правилам или MILP-решениям, включающие векторы признаков операций и ресурсов и целевые метки назначенных ресурсов. Выходными данными системы являются:

- производственное расписание — распределение операций по ресурсам с указанием времени старта и окончания каждой операции, сформированное на основе выбранного метода планирования (эвристика, нейросеть BC, SLIM или PPO);
- рекомендации по назначению операций — результат работы нейросетевого агента, выдающего для каждой неразмещённой операции предпочтительный ресурс с учётом текущей производственной обстановки;
- обученные нейросетевые модели — файлы весов для каждого из трёх методов обучения (PointerNet, SLIM, PPO), пригодные для последующей загрузки и использования в планировании без повторного обучения;
- метрики обучения и сравнения — значения функции потерь, точности классификации на валидационной выборке, матрицы ошибок, отчёты о классификации, а также сводные KPI расписаний (среднее и максимальное опоздание, процент опаздывающих заказов, makespan, коэффициент вариации загрузки ресурсов) для каждого метода планирования;
- визуализация — диаграмма Ганта производственного плана, гistogramмы опозданий, графики загрузки ресурсов, кумулятивные кривые выполнения, box-plot и другие графические материалы.

3.1.3.2 Концептуальная модель предметной области

Концептуальная модель системы представляет собой взаимосвязь основных сущностей и потоков данных.

Основные компоненты модели:

1. «Ядро» — основной модуль, где происходят вычисления, сортировка и подключение к остальным модулям.
2. «Файловое хранилище» — локальное хранилище пользователя, где установлена программа, все модели, метрические данные и дополнительные компоненты хранятся.
3. «База данных» — централизованное хранилище заказов, операций, ресурсов, связей типов, очередей задач и результатов планирования.

4. «Интерфейс» — графическое приложение на PySide6, обеспечивающее визуализацию дашборда, диаграммы Ганта, редактирование заказов и ресурсов, запуск планирования и обучения.

На рисунке 3.1 представлена структурная схема концептуальной модели предметной области.



Рисунок 3.1 – Структурная схема концептуальной модели предметной области

Взаимодействие компонентов:

1. Пользователь через интерфейс создаёт заказы, настраивает ресурсы и их готовность.
2. Планировщик получает актуальные данные из базы, выбирает метод (эвристика или загруженная модель) и строит расписание.
3. При использовании нейросети обученные веса подгружаются из хранилища моделей, а признаки операции и ресурсов подаются на вход сети для получения рекомендованного ресурса.
4. Результаты планирования отображаются на дашборде и диаграмме Ганта, а также могут быть выгружены для анализа.
5. Обучающий модуль генерирует синтетические данные (правило LWKR) или использует исторические MILP-решения, тренирует модели с заданными гиперпараметрами.
6. Все изменения состояния ресурсов (готовность, ремонт) и заказов синхронизируются с базой данных в реальном времени.

3.1.3.3 Функциональные требования к программной системе

Функциональные требования к интеллектуальной системе производственного планирования определяют набор основных возможностей, необходимых для автоматизации диспетчеризации и повышения эффективности использования оборудования. Система должна обеспечивать ведение портфеля заказов, построение выполнимых расписаний, управление производственными ресурсами, обучение и применение нейросетевых моделей[41].

Функциональные требования:

1. Система должна предоставлять возможность создания, редактирования и удаления заказов с указанием номенклатуры, срока сдачи, приоритета и технологических операций.
2. Должна быть реализована функция построения производственного расписания (эвристикой или нейросетевым диспетчером) с назначением операций на ресурсы с учётом загрузки, готовности и специализации.
3. Необходимо обеспечить ручное управление очередями на дашборде: перенос задач между нераспределёнными и очередями ресурсов, запуск в производство, завершение операции, сброс всех задач в нераспределённые.
4. Система должна отображать актуальное состояние каждого ресурса: статус, готовность (надёжность), флаг ремонта, число задач в очереди, наличие активной задачи.
5. Пользователь должен иметь возможность изменять параметры ресурса (готовность, признак ремонта) с немедленным сохранением в базе данных.
6. Должна быть реализована возможность обучения нейросетевых моделей (PointerNet, SLIM, PPO) на сгенерированных датасетах с сохранением весов и метрик обучения в файловой системе.
7. Система должна позволять загружать и сохранять веса обученных моделей, а также автоматически подгружать последнюю использованную модель при старте.

8. Должна вестись история завершённых операций с возможностью просмотра и удаления записей.

9. Необходимо поддерживать управление технологическими типами операций и связями «ресурс–тип», ограничивающими назначение операций определённым набором станков.

10. Административные функции (добавление/удаление ресурсов и типов операций, настройка связей) доступны тому же пользователю без разделения ролей.

В системе предусмотрено одно действующее лицо: Пользователь – управляет заказами, ресурсами, очередями, обучением моделей и просмотром метрик, т.е. совмещает функции оператора и администратора.

На рисунке 3.2 представлена диаграмма прецедентов пользователя, содержащая функции, которые должны быть реализованы в разрабатываемой ИС.



Рисунок 3.2 – Диаграмма прецедентов пользователя

Прецедент «Создать заказ»

Предусловия: Пользователь находится на вкладке «Заказы». В системе имеется хотя бы один тип операции.

Входные данные: название заказа, номер (опционально), дата и время сдачи, приоритет, тип операции, длительность, изделие, количество деталей.

Выходные данные: новый заказ с одной операцией, сохранённый в базе данных и отображённый на дашборде и в таблице заказов.

Постусловия: заказ доступен для планирования.

Сценарий:

1. Пользователь переходит на вкладку «Заказы».
2. Заполняет поля формы, выбирает тип операции из выпадающего списка, вводит длительность, изделие и количество.
3. Нажимает «Сохранить заказ».
4. Система проверяет корректность данных, создаёт заказ и операцию, присваивает уникальный идентификатор.
5. Заказ записывается в базу данных, появляется в списке заказов и в нераспределённых задачах на дашборде.

Прецедент «Построить план»

Предусловия: в системе есть неразмещённые заказы.

Входные данные: нажатие кнопки «Построить план (RL)» или «Точный MILP-план».

Выходные данные: расписание, в котором каждой операции назначены ресурс, время начала и окончания; обновлённая диаграмма Ганта.

Постусловия: задачи помещены в очереди ресурсов, дашборд обновлён.

Сценарий:

1. Пользователь нажимает кнопку построения плана на дашборде или вкладке «Планирование».
2. Если загружена нейросеть, система формирует векторы признаков для каждой операции, пропускает через модель, корректирует логиты штрафом за низкую готовность и выбирает ресурс. Иначе применяется эвристика EDD и минимальная эффективная загрузка.
3. Полученное расписание записывается в `current_schedule` и синхронизируется с таблицей `task_queue`.

4. Задачи автоматически запускаются на исправных ресурсах (функция «В работу»).

5. Обновляются дашборд и диаграмма Ганта.

Прецедент «Завершить операцию»

Предусловия: на ресурсе есть активная задача (статус active).

Входные данные: нажатие кнопки «Завершить» на карточке задачи.

Выходные данные: операция переведена в статус completed, соответствующий заказ удалён из активных, следующая ожидающая задача на ресурсе запущена.

Постусловия: очередь ресурса продвинута, запись о завершении добавлена в логи и вкладку «Завершённые».

Сценарий:

1. Пользователь на дашборде нажимает «Завершить» на активной задаче.
2. Система определяет ресурс, обновляет статус задачи, логирует событие с указанием ресурса.
3. Заказ, к которому относилась операция, помечается завершённым в БД и убирается из активного списка.
4. Система проверяет очередь освободившегося ресурса и, если он исправен, запускает первую ожидающую задачу.
5. Дашборд и вкладка «Завершённые» обновляются.

Прецедент «Изменить готовность / ремонт ресурса»

Предусловия: ресурс существует и отображается на дашборде.

Входные данные: двойной клик по карточке ресурса, ввод нового значения готовности (0.0–1.0) и/или установка флажка «Ресурс в ремонте».

Выходные данные: обновлённые атрибуты ресурса, сохранённые в БД; перерисовка дашборда.

Постусловия: при следующем планировании изменённые параметры влияют на назначение операций.

Сценарий:

1. Пользователь дважды кликает по карточке ресурса.

2. Открывается диалог с полями «Готовность» и «Ремонт», а также информацией о текущей очереди.

3. Пользователь изменяет значение и/или флажок, нажимает «Сохранить».

4. Система записывает новые значения в объект ресурса и в базу данных, немедленно обновляя индикаторы на дашборде.

Прецедент «Обучить нейросетевую модель»

Предусловия: имеется файл обучающей выборки `gpss_dataset.npz`, пользователь находится на вкладке «Обучение».

Входные данные: количество эпох (или эпизодов), скорость обучения, выбор типа модели (PointerNet/SLIM/PPO).

Выходные данные: обученная модель (файл `.pth`), набор метрик в папке `metrics/`.

Постусловия: модель может быть загружена и использована для планирования.

Сценарий:

1. Пользователь переходит на вкладку «Обучение», при необходимости загружает датасет.

2. Указывает гиперпараметры и нажимает соответствующую кнопку (например, «Обучить PointerNet»).

3. Система загружает данные, создаёт модель SimpleDispatcher, запускает обучение, отображая прогресс в реальном времени.

4. По окончании сохраняет веса модели (`pointer_net.pth`) и все метрики (кривые потерь, точности, матрицу ошибок) в папку `metrics/<Модель>/<дата_время>/`.

5. Пользователь получает уведомление о завершении.

Прецедент «Добавить ресурс»

Предусловия: пользователь находится на вкладке «Оборудование» → «Ресурсы».

Входные данные: идентификатор и название ресурса.

Выходные данные: новый ресурс, добавленный в БД и отображаемый на дашборде.

Постусловия: ресурс готов к планированию (готовность 1.0, не в ремонте).

Сценарий:

1. Пользователь вводит ID и название ресурса в соответствующие поля.
2. Нажимает «Добавить».
3. Система проверяет уникальность ID, создаёт объект Resource, сохраняет его в БД.
4. Ресурс появляется в списке ресурсов и на дашборде.

Остальные прецеденты (редактирование заказа, удаление заказа, настройка связей, просмотр метрик и т.д.) реализованы аналогично и доступны через соответствующие вкладки интерфейса.

3.1.3.4 Требования пользователя к интерфейсу программной системы

Пользовательский интерфейс программной системы должен быть интуитивно понятным и обеспечивать доступ к основным функциям без необходимости в специальных технических знаниях. Интерфейс предназначен для взаимодействия как с операторами, управляющими производственными заказами и расписаниями, так и с технологами, настраивающими оборудование и нейросетевые модели[17].

Для пользователя интерфейс должен предоставлять возможность:

- создания, редактирования и удаления заказов на производство с указанием типа операции, длительности, количества и срока сдачи;
- управления производственными ресурсами: изменения готовности (надёжности) и признака ремонта, просмотра текущей загрузки;
- запуска построения плана с выбором метода (эвристический или нейросетевой) и визуализации полученного расписания на диаграмме Ганта;

- ручного управления очередями: перенос задач из нераспределённых в очереди конкретных ресурсов, запуск задачи в производство, завершение операции, сброс всех задач обратно в нераспределённые;
- просмотра истории завершённых операций и их удаления;
- обучения нейросетевых моделей (PointerNet, SLIM, PPO) с настройкой гиперпараметров (число эпох/эпизодов, скорость обучения), отображением прогресса обучения и сохранением метрик;
- загрузки и сохранения весов обученных моделей, а также автоматической подгрузки последней использованной модели при старте;
- управления типами операций и связями «ресурс–тип», ограничивающими назначение операций.

Все вышеперечисленные функции доступны в рамках единого графического интерфейса, разделённого на тематические вкладки: «Дашборд», «Заказы», «Планирование», «Оборудование», «Состояния», «Обучение», «Настройки», «Завершённые». Основное взаимодействие с очередями и ресурсами осуществляется через дашборд, поддерживающий механизм drag-and-drop для перемещения задач.

Макет пользовательского интерфейса предоставлен на рисунке 3.3.

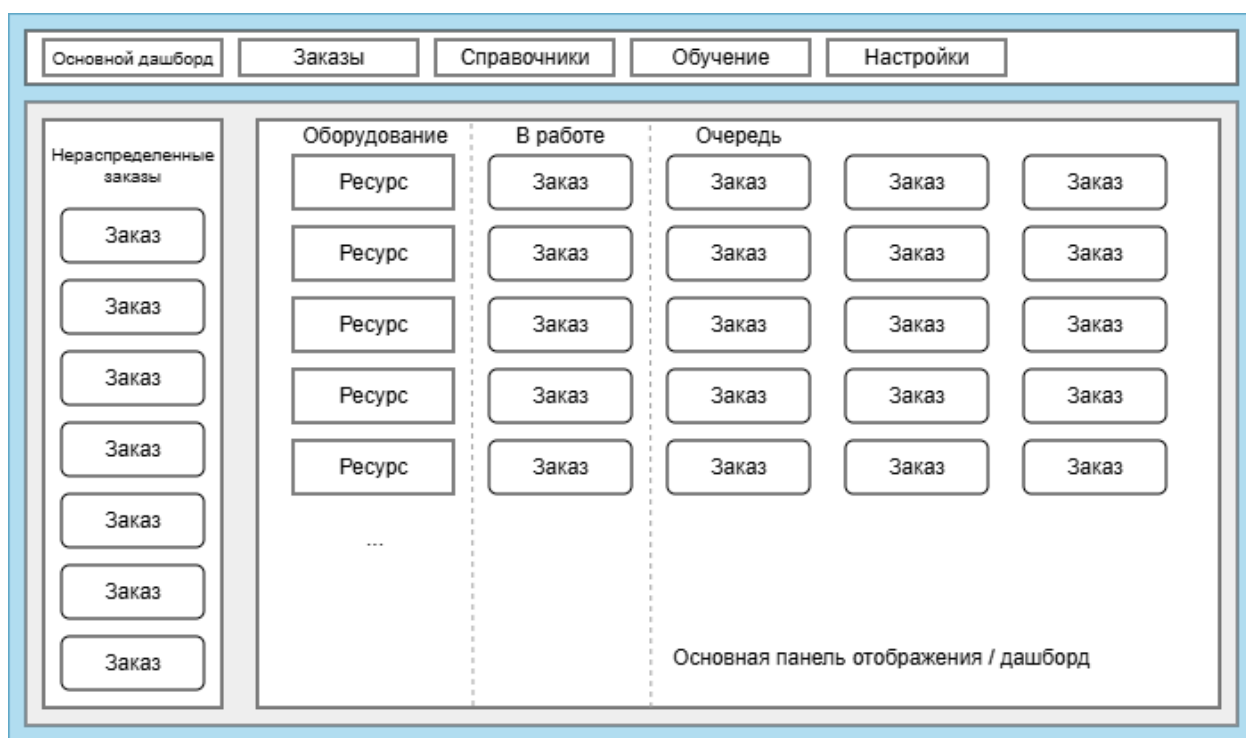


Рисунок 3.3 – Макет пользовательского интерфейса

3.1.3.5 Нефункциональные требования к программной системе

Нефункциональные требования к интеллектуальной системе производственного планирования определяют характеристики, обеспечивающие её надёжность, производительность и удобство эксплуатации.

Система должна обеспечивать высокую производительность при построении расписаний. Время формирования плана для типового объёма заказов (до 100 операций на 5–6 ресурсах) не должно превышать нескольких секунд как для эвристического алгоритма, так и для нейросетевого диспетчера.

Система должна быть отказоустойчивой и стабильной. Все изменения состояния ресурсов и заказов должны немедленно фиксироваться в базе данных, исключая потерю данных при нештатном завершении приложения. При возникновении ошибок во время обучения модели должно быть предусмотрено корректное завершение процесса с сохранением лога ошибок.

Обеспечение целостности данных является критически важным. База данных SQLite должна поддерживать ссылочную целостность между заказа-

ми, операциями, ресурсами и очередями. Все операции с данными выполняются в рамках одного приложения, поэтому дополнительная аутентификация не требуется, однако должна быть исключена возможность одновременной порчи данных из-за параллельных процессов.

Система должна быть масштабируемой в части увеличения числа заказов, ресурсов и типов операций без модификации исходного кода. Добавление новых ресурсов или типов операций должно выполняться через штатный интерфейс. Модели машинного обучения должны допускать повторное обучение на обновлённых выборках.

3.1.3.6 Требования к аппаратным средствам

Для корректного функционирования интеллектуальной системы производственного планирования достаточно одного локального компьютера, выполняющего одновременно функции сервера (хранение данных, обучение моделей) и клиентского рабочего места (пользовательский интерфейс).

Компьютер должен быть оснащён многоядерным процессором с тактовой частотой не менее 1.5 ГГц, иметь объём оперативной памяти не менее 4 ГБ и экран с разрешением не менее 1280×800 пикселей. Для хранения базы данных, исполняемого кода, обученных моделей и метрик требуется не менее 500 МБ свободного дискового пространства.

Графический процессор (GPU) не является обязательным, однако его наличие может существенно ускорить обучение нейросетевых моделей при большом объёме данных.

3.1.3.7 Требования к программным средствам

Программное обеспечение должно функционировать под управлением операционных систем семейства Windows (версия 10 или новее), Linux (основные дистрибутивы).

На компьютере должен быть установлен интерпретатор Python версии 3.9 или выше со следующими библиотеками:

- PySide6 (графический интерфейс);

- PyTorch (обучение и применение нейросетевых моделей);
- NumPy, Matplotlib, scikit-learn (обработка данных, визуализация метрик, расчёт метрик классификации);
- SQLite3 (встроен в стандартную библиотеку Python, используется для хранения данных).

Дополнительное серверное программное обеспечение не требуется: система использует локальную базу данных SQLite, доступ к которой осуществляется непосредственно из приложения.

3.1.3.8 Требования к архитектуре нейронной сети

Для решения задачи выбора оптимального ресурса при планировании используется искусственная нейронная сеть прямого распространения (многослойный перцептрон). Модель принимает на вход вектор признаков фиксированной размерности, описывающий одну технологическую операцию и текущее состояние всех доступных ресурсов[10].

Входной вектор состоит из 36 признаков: 6 признаков операции (one-hot кодировка типа операции, нормализованная длительность, приоритет заказа, slack до срока сдачи) и для каждого из 5 ресурсов по 6 признаков (признак готовности, текущая загрузка, совместимость с типом операции, средняя историческая загрузка, флаг ремонта, коэффициент надёжности).

Архитектура модели включает три полносвязных слоя:

1. входной слой размерностью 36 нейронов;
2. скрытый слой из 64 нейронов с функцией активации ReLU и пакетной нормализацией (BatchNorm);
3. скрытый слой из 32 нейронов с функцией активации ReLU и пакетной нормализацией;
4. выходной слой из N нейронов (по числу ресурсов), выдающий ненормированные логиты вероятностей выбора каждого ресурса.

Для принятия решения к выходным логитам применяется поправка, пропорциональная логарифму эффективной готовности ресурса, что позволяет явно учесть надёжность и ремонт при выборе.

Обучение модели выполняется по следующим парадигмам:

1. Поведенческое клонирование (BC, PointerNet). Минимизация кросс-энтропии между предсказанным и эталонным распределением, полученным эвристическим правилом минимальной эффективной загрузки.
2. Самоконтролируемое обучение с саморазметкой (SLIM). Аналогичная модель с периодическим добавлением уверенных предсказаний на неразмеченных данных.
3. Обучение с подкреплением (PPO). Модель обучается максимизировать отрицательное опоздание заказов, взаимодействуя с имитационной средой производственного планирования.

3.1.4 Требования к оформлению документации

Оформление документации должно осуществляться с учётом стандартов СТUY 02.030-2023, единой системы программной документации, единой системы конструкторской документации.

3.2 Технический проект

3.2.1 Общие сведения об интеллектуальной системе оптимизации

Интеллектуальная система производственного планирования (APS-система) предназначена для автоматизации процессов составления выполнимых расписаний и оперативной диспетчеризации на предприятиях с дискретным характером производства. Система предоставляет оператору возможность гибкого управления заказами, ресурсами и очередями, а также позволяет обучать и применять нейросетевые диспетчеры для повышения эффективности использования оборудования.

Программный продукт реализован в виде настольного приложения на языке Python с графическим интерфейсом PySide6 и локальной реляционной базой данных SQLite. Такая архитектура обеспечивает простоту развёртывания, автономность работы и низкие требования к инфраструктуре. Вся информация о заказах, операциях, ресурсах, технологических связях и истории

исполнения хранится централизованно в единой базе данных, что гарантирует целостность и непротиворечивость данных[36].

Система спроектирована с учётом возможности расширения числа производственных ресурсов и типов операций без модификации исходного кода. Добавление нового станка или технологической операции выполняется через штатный пользовательский интерфейс, после чего они сразу становятся доступными для планирования. Модульная архитектура позволяет также подключать новые нейросетевые модели и алгоритмы планирования, что делает систему гибкой платформой для исследований и практического применения.

3.2.2 Проект данных интеллектуальной системы

При проектировании системы производственного планирования была использована локальная реляционная база данных SQLite, обеспечивающая хранение всех сущностей предметной области и их взаимосвязей. Структура базы данных разработана с учётом требований нормализации и поддержки ссылочной целостности.

Таблица «Заказы» содержит информацию о производственных заказах. Для каждого заказа хранится уникальный идентификатор, срок сдачи, приоритет, наименование и статус (активен или завершён). В таблице 3.1 представлены атрибуты таблицы «Заказы».

Таблица 3.1 – Атрибуты таблицы «Заказы»

Атрибут	Тип	Описание
1	2	3
id	string	Уникальный идентификатор заказа (может быть с символами)
due_date	datetime	Плановая дата и время сдачи заказа
priority_weight	real	Весовой коэффициент приоритета (от 0.5 до 2.0)
name	string	Краткое наименование заказа
status	string	Текущее состояние заказа

Таблица «Операции» содержит детализацию технологических операций, входящих в заказы. Каждая операция связана с родительским заказом и характеризуется типом, нормативной длительностью, количеством изготавливаемых деталей и другими параметрами. В таблице 3.2 приведены атрибуты таблицы «Операции».

Таблица 3.2 – Атрибуты таблицы «Операции»

Атрибут	Тип	Описание
1	2	3
id	string	Уникальный идентификатор операции
order_id	string	Идентификатор заказа, к которому относится операция (внешний ключ)
item	string	Наименование изготавливаемого изделия
op_number	integer	Порядковый номер операции в заказе
norm_duration	real	Нормативная длительность операции в минутах
type_id	string	Идентификатор типа операции
predecessors	string	Список предшествующих операций (для реализации технологической последовательности)
quantity	integer	Количество деталей
urgency	real	Коэффициент срочности
slack_hours	real	Запас времени до дедлайна

Таблица «Ресурсы» описывает производственное оборудование. Для каждого ресурса задаются его наименование, текущий статус, нагрузочные характеристики и параметры готовности. В таблице 3.3 представлены атрибуты таблицы «Ресурсы».

Таблица 3.3 – Атрибуты таблицы «Ресурсы»

Атрибут	Тип	Описание
1	2	3
id	string	Уникальный идентификатор ресурса
name	string	Наименование ресурса
section	string	Производственный участок
status	string	Текущий статус
operator_name	string	ФИО оператора, закреплённого за ресурсом
load_minutes	real	Накопленная загрузка в минутах
downtime_minutes	real	Время простоя в минутах
work_hours	integer	Доступный фонд рабочего времени в часах
repair	integer	Признак нахождения в ремонте
reliability	real	Коэффициент готовности

Таблица «Типы операций» содержит справочник технологических типов (фрезеровка, сборка, сварка и др.). В таблице 3.4 приведены её атрибуты.

Таблица 3.4 – Атрибуты таблицы «Типы операций»

Атрибут	Тип	Описание
1	2	3
id	string	Уникальный идентификатор типа
name	string	Название типа операции

Таблица «Связи ресурсов с типами» определяет, какие типы операций может выполнять каждый ресурс. Используется для назначения операций на подходящее оборудование. В таблице 3.5 перечислены атрибуты.

Таблица 3.5 – Атрибуты таблицы «Связи ресурсов с типами»

Атрибут	Тип	Описание
1	2	3
resource_id	string	Идентификатор ресурса
operation_type_id	string	Идентификатор типа операции

Таблица «Очередь задач» хранит текущее состояние очередей на ресурсах. Каждая запись связывает операцию с конкретным ресурсом, определяет её позицию в очереди и статус выполнения. В таблице 3.6 даны атрибуты.

Таблица 3.6 – Атрибуты таблицы «Очередь задач»

Атрибут	Тип	Описание
1	2	3
resource_id	string	Идентификатор ресурса, в очередь которого помещена операция
operation_id	string	Идентификатор операции
position	integer	Позиция в очереди
status	string	Статус выполнения

Таблица «Журнал состояний операций» фиксирует историю изменения статусов операций. Это необходимо для отслеживания хода производства и формирования отчётов. В таблице 3.7 представлены её атрибуты.

Таблица 3.7 – Атрибуты таблицы «Журнал состояний операций»

Атрибут	Тип	Описание
1	2	3
id	integer	Автоинкрементный первичный ключ
operation_id	string	Идентификатор операции
status	string	Статус операции на момент записи
resource_id	string	Идентификатор ресурса, на котором выполнялась операция
state_id	string	Идентификатор типа состояния
timestamp	datetime	Метка времени события
source	string	Источник изменения

Таблица «Настройки приложения» хранит конфигурационные параметры в виде пар «ключ-значение». Используется для сохранения путей к файлам моделей, гиперпараметров обучения и других пользовательских предпочтений. В таблице 3.8 указаны атрибуты.

Таблица 3.8 – Атрибуты таблицы «Настройки приложения»

Атрибут	Тип	Описание
1	2	3
key	string	Уникальное имя параметра
value	string	Значение параметра
updated_at	datetime	Дата и время последнего обновления

На рисунке 3.4 представлена схема базы данных, отражающая основные таблицы и связи между ними.

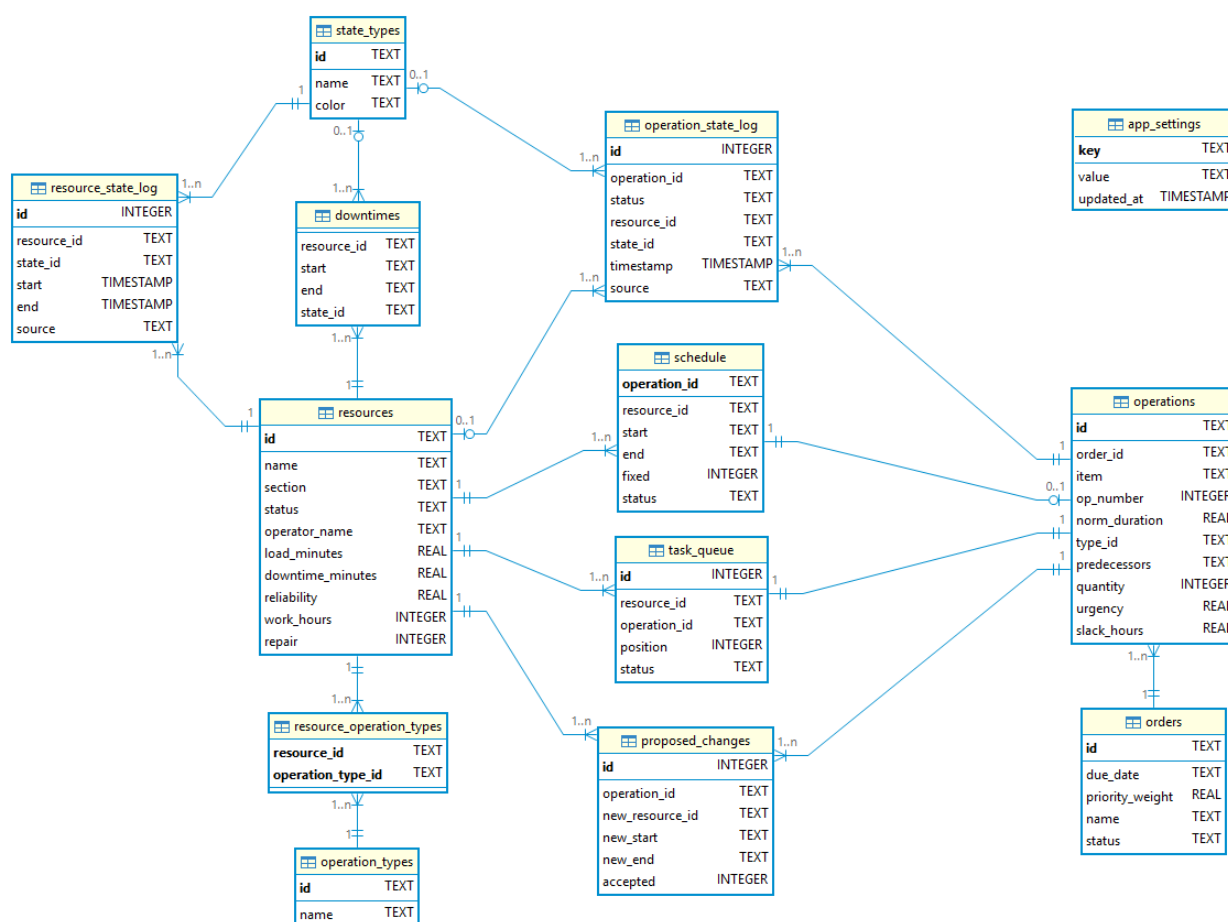


Рисунок 3.4 – Структурная схема локальной базы данных

3.2.3 Проектирование архитектуры программы

3.2.3.1 Компоненты программы

Разрабатываемая интеллектуальная система производственного планирования состоит из трёх логических групп компонентов, функционирующих в рамках одного настольного приложения.

Хранение информации обеспечивается реляционной базой данных SQLite, в которой размещены все основные сущности: заказы, операции, ресурсы, очереди задач, журналы состояний и настройки. Дополнительно используется файловое хранилище для размещения генерируемых обучающих выборок, обученных весов нейросетевых моделей и результатов метрик.

Для реализации системы используется язык программирования Python версии 3.9 и выше. Графический интерфейс построен с применением библиотеки PySide6, архитектура интерфейса основана на императивном подходе с выделением классов для каждой вкладки и диалогового окна. Бизнес-логика сосредоточена в классе APSCore, который координирует работу всех подсистем.

В процессе разработки используются следующие библиотеки:

1. PySide6 – библиотека для создания кроссплатформенных графических интерфейсов на языке Python, предоставляющая полный набор виджетов, поддержку тем оформления, работу с диаграммами и графическими сценами.

2. PyTorch – библиотека машинного обучения, используемая для построения, обучения и применения нейросетевых моделей диспетчеризации. Обеспечивает автоматическое дифференцирование, оптимизаторы и функции потерь.

3. NumPy – библиотека для работы с многомерными массивами, применяется при формировании векторов признаков операций и ресурсов, а также при расчёте метрик.

4. Matplotlib – библиотека для визуализации данных, используется для построения графиков обучения (кривые потерь и точности) и результатов сравнения методов.

5. scikit-learn – библиотека машинного обучения, применяется для расчёта метрик классификации (матрица ошибок, precision, recall, f1-score) при оценке качества нейросетевых диспетчеров.

6. SQLite3 – встроенная в Python библиотека для работы с реляционной базой данных SQLite, используется для хранения всех производственных данных и настроек.

Компоненты графического интерфейса включают:

1. Главное окно – компонент, реализующий основное окно приложения с тематическими вкладками: Дашборд, Заказы, Планирование, Оборудование, Состояния, Обучение, Настройки, Завершённые.

2. Дашборд – компонент, отображающий текущее состояние ресурсов, очередей и нераспределённых задач. Поддерживает механизм drag-and-drop для перемещения задач между очередями и ресурсами.

3. Формы заказов и ресурсов – компоненты, обеспечивающие создание, редактирование и удаление заказов и ресурсов с сохранением изменений в базу данных.

4. Диаграмма Ганта – компонент на основе QGraphicsScene, визуализирующий построенное расписание с цветовой кодировкой по типам операций.

5. Панель обучения – компонент, предоставляющий элементы управления для запуска обучения моделей, настройки гиперпараметров и отображения прогресса в реальном времени.

6. Панель метрик – компонент, отображающий логи обучения и результаты сравнения методов в текстовом и графическом виде.

Компоненты ядра системы включают:

1. APSCore – центральный компонент, реализующий бизнес-логику приложения: управление заказами и ресурсами, вызов методов планирования, синхронизацию расписаний с очередями, обработку завершения операций.

2. Модуль планирования – компонент, содержащий эвристический алгоритм EDD+LWKR с учётом готовности ресурсов, а также интерфейсы для вызова нейросетевых диспетчеров PointerNet, SLIM и PPO.

3. Модуль обучения – компонент, отвечающий за генерацию обучающих выборок, запуск тренировки моделей и сохранение весов и метрик.

4. Модуль сравнения – компонент, выполняющий тестирование обученных моделей на едином сценарии и вычисляющий ключевые производственные показатели.

5. Модуль управления данными – компонент, обеспечивающий взаимодействие с базой данных SQLite и файловым хранилищем.

Компоненты хранения данных включают:

1. База данных SQLite – реляционная база, содержащая таблицы заказов, операций, ресурсов, типов операций, связей, очередей задач, журналов состояний и настроек.

2. Файловое хранилище – папки с генерируемыми датасетами, файлами весов обученных моделей и результатами метрик.

На рисунке 3.5 представлена структурная схема программной системы, выполненная в виде блок-схемы UML.

3.2.3.2 Архитектура интеллектуальной системы

Интеллектуальная система производственного планирования представляет собой монолитное настольное приложение, построенное по трёхуровневой архитектуре в рамках одного процесса.

Уровни включают: слой представления (пользовательский интерфейс), слой бизнес-логики (ядро системы) и слой данных (база данных и файловое хранилище). Все компоненты функционируют на одном рабочем месте, взаимодействуя напрямую без сетевых протоколов.

Слой представления реализован на базе библиотеки PySide6 и содержит главное окно с тематическими вкладками, формы ввода заказов и параметров ресурсов, диаграмму Ганта, а также диалоговые окна для редактиро-

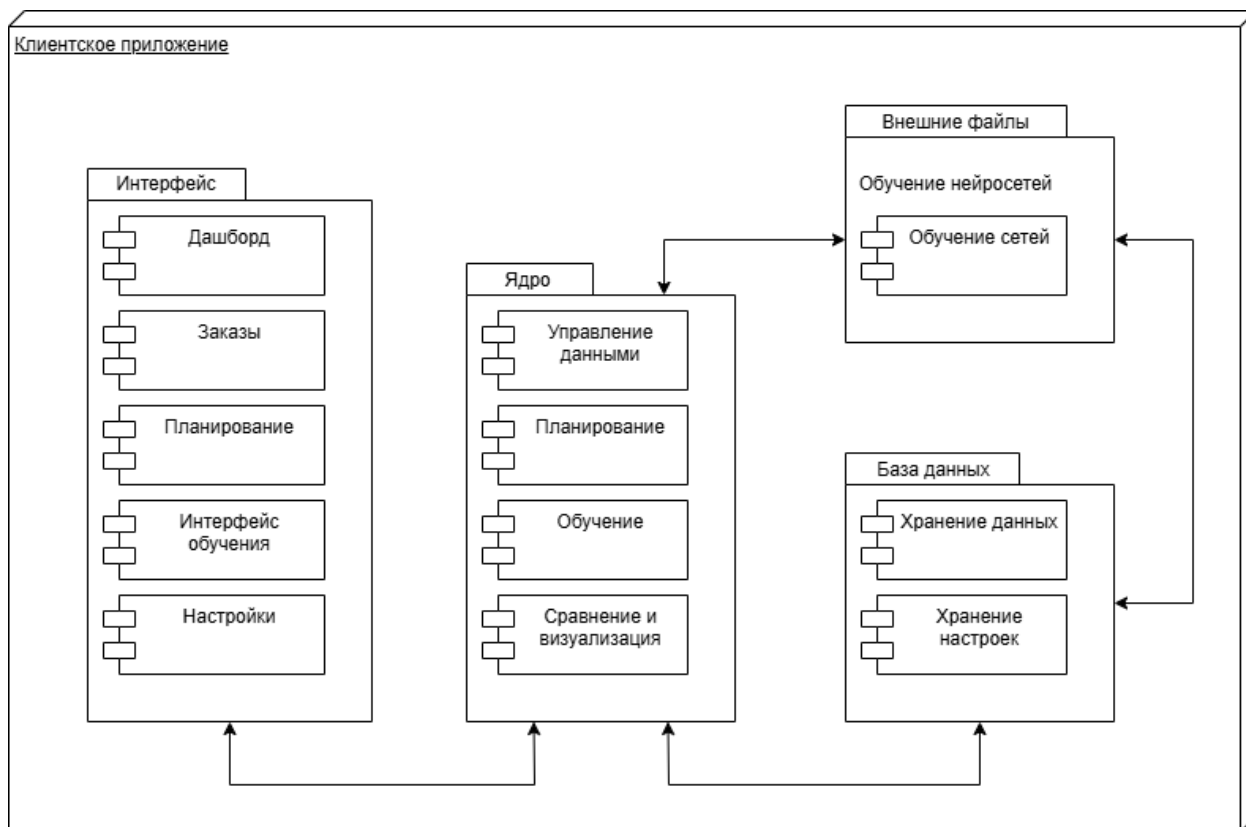


Рисунок 3.5 – Структурная схема компонентов программы

вания сущностей. Пользовательский интерфейс взаимодействует исключительно с ядром системы, отправляя запросы на выполнение операций и получая обновлённые данные для отображения.

Слой бизнес-логики представлен классом `APSCore`, который инкапсулирует всю прикладную логику: управление заказами и ресурсами, эвристическое и нейросетевое планирование, обучение моделей, сравнение методов и управление очередями. Ядро системы координирует работу вложенных модулей (модуль планирования, модуль обучения, модуль сравнения), которые вызываются по мере необходимости. Доступ к данным осуществляется через модуль управления данными, абстрагирующий работу с базой данных SQLite и файловой системой.

Слой данных включает реляционную базу данных SQLite, хранящую все основные сущности (заказы, операции, ресурсы, очереди, журналы состояний, настройки), и файловое хранилище, в котором размещаются генерируемые датасеты, обученные веса нейросетевых моделей, метрики обучения и результаты сравнения методов. База данных находится в локальном файле

aps.db, что исключает необходимость развёртывания серверного программного обеспечения и упрощает перенос системы.

Взаимодействие между уровнями организовано следующим образом: графический интерфейс формирует запрос, ядро системы обрабатывает его, при необходимости обращаясь к модулям планирования или обучения, и через модуль управления данными сохраняет или извлекает информацию. Результаты возвращаются в интерфейс для визуализации.

На рисунке 3.6 представлена многоуровневая диаграмма архитектуры системы, отображающая перечисленные слои и их основные компоненты.

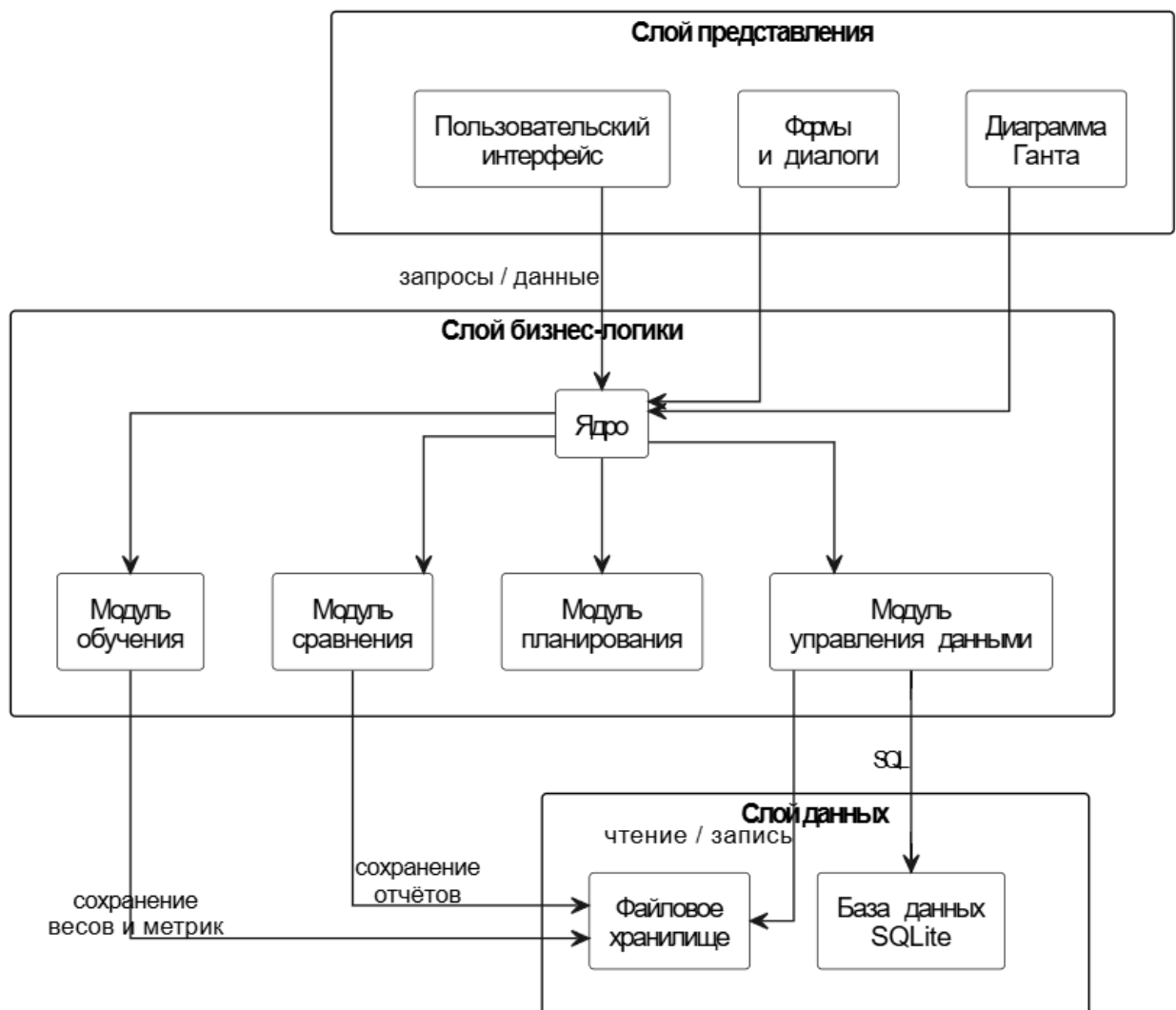


Рисунок 3.6 – Диаграмма архитектуры системы

3.2.3.3 Проектирование пользовательского интерфейса

На основе требований к пользовательскому интерфейсу, представленных в пункте технического задания, с помощью библиотеки PySide6 был разработан пользовательский интерфейс. Интерфейс построен по принципу тематических вкладок, каждая из которых предоставляет доступ к определённой функциональной группе. Главное окно приложения содержит следующие вкладки: Дашборд, Заказы, Планирование, Оборудование, Состояния, Обучение, Настройки, Завершённые.

На рисунке 3.7 представлен макет интерфейса вкладки «Дашборд».

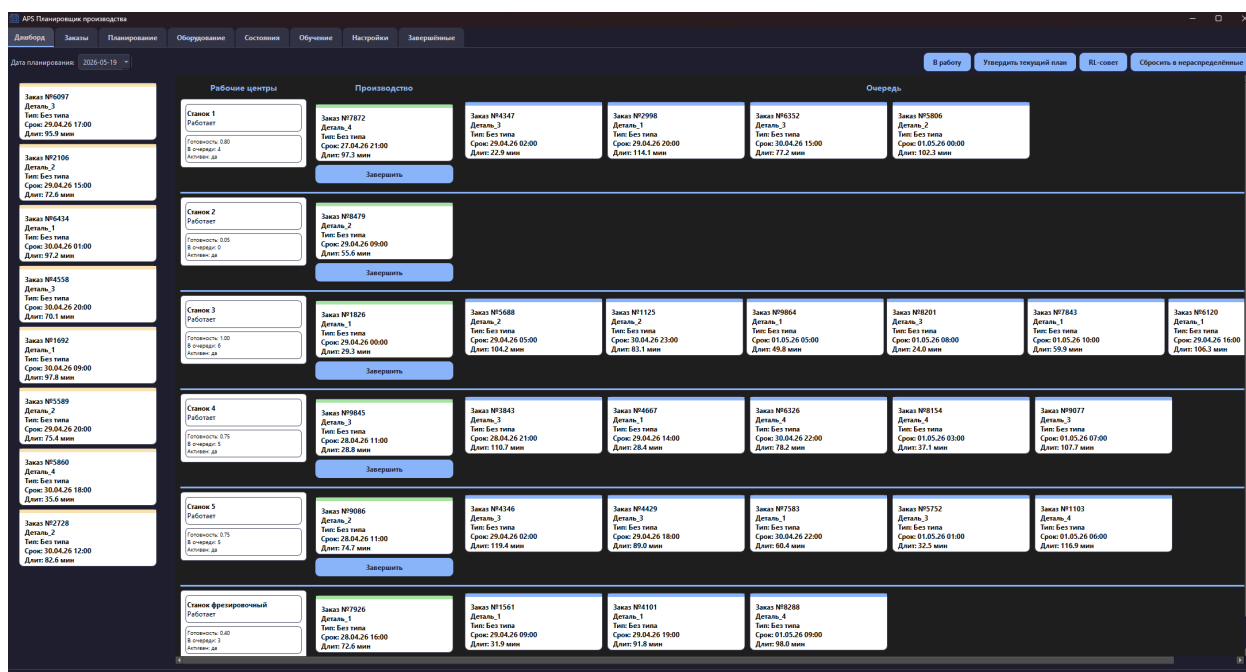


Рисунок 3.7 – Макет интерфейса вкладки «Дашборд»

На макете вкладки «Дашборд» представлено основное рабочее пространство оператора. В верхней части расположены выпадающий список для выбора даты планирования и кнопки управления: «Утвердить текущий план», «RL-совет», «В работу» и «Сбросить в нераспределённые». Левая часть вкладки отведена под нераспределённые задачи, которые отображаются в виде цветных карточек с информацией о заказе, типе операции и сроках. Правая часть представляет сетку, в которой для каждого ресурса показаны три колонки: название и статус ресурса, текущая выполняемая опера-

ция и очередь ожидающих задач. Карточки задач можно перетаскивать между нераспределёнными, очередью ресурса и обратно. Двойной клик по карточке ресурса открывает окно редактирования параметров ресурса.

На рисунке 3.8 представлен макет интерфейса вкладки «Заказы».



Рисунок 3.8 – Макет интерфейса вкладки «Заказы»

На макете вкладки «Заказы» представлена форма управления производственными заказами. В верхней части расположена форма создания нового заказа с полями: название заказа, номер, дата и время сдачи, приоритет, тип операции, длительность, изделие и количество деталей. Ниже размещена кнопка «Сохранить заказ», а также кнопки для генерации десяти случайных заказов и загрузки заказов из ERP-файла. Под формой отображается таблица существующих заказов с колонками: идентификатор заказа, название, срок сдачи, количество операций, приоритет и кнопка удаления. Двойной клик по карточке заказа в любой части интерфейса открывает окно редактирования параметров заказа.

На рисунке 3.9 представлен макет интерфейса вкладки «Планирование».

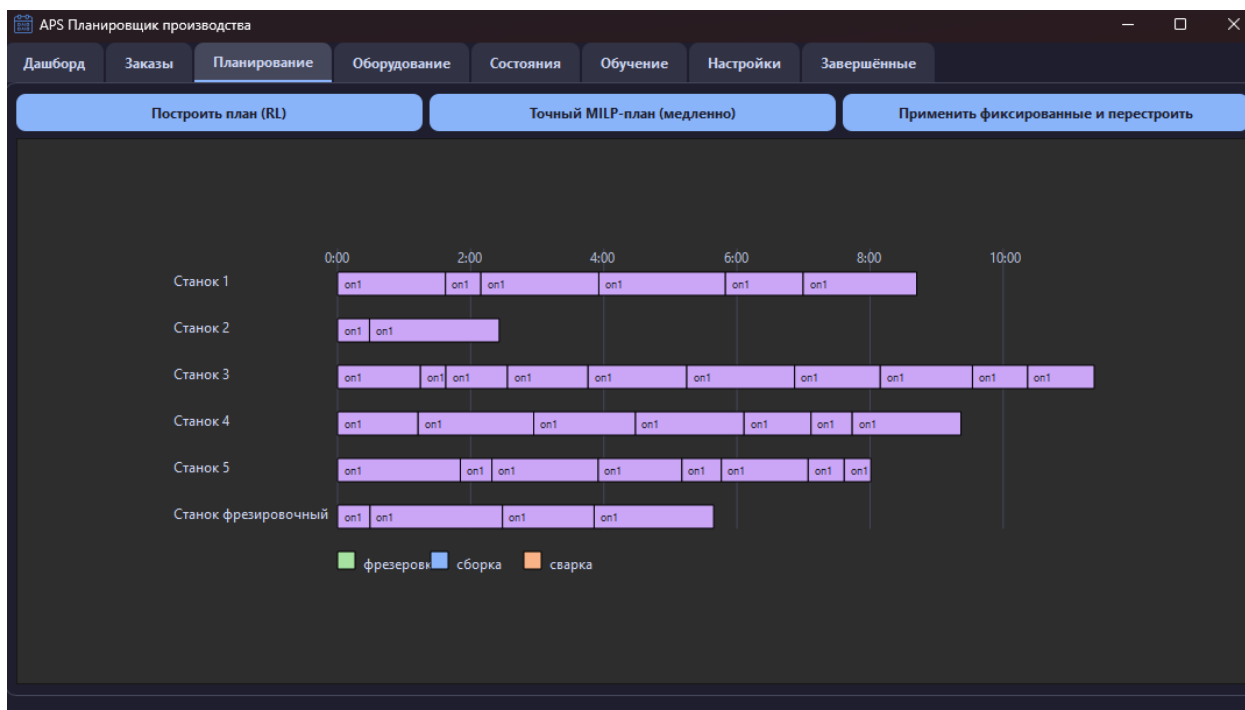


Рисунок 3.9 – Макет интерфейса вкладки «Планирование»

На макете вкладки «Планирование» расположены кнопки управления построением расписания: «Построить план (RL)», «Точный MILP-план (медленно)» и «Применить фиксированные и перестроить». Нижнюю часть вкладки занимает диаграмма Ганта, визуализирующая построенное расписание. На диаграмме операции отображаются в виде цветных блоков, привязанных к временной шкале и дорожкам ресурсов. Цвет блока соответствует типу операции.

На рисунке 3.10 представлен макет интерфейса вкладки «Оборудование».

На макете вкладки «Оборудование» представлены три подвкладки: Ресурсы, Типы и Связи. На подвкладке «Ресурсы» отображается список текущих ресурсов и форма для добавления нового ресурса с полями идентификатора и названия. На подвкладке «Типы» представлен список типов операций и форма для добавления нового типа. На подвкладке «Связи» расположены выпадающие списки для выбора ресурса и типа операции, кнопка «Добавить связь» и таблица существующих связей с возможностью удаления.

На рисунке 3.11 представлен макет интерфейса вкладки «Обучение».

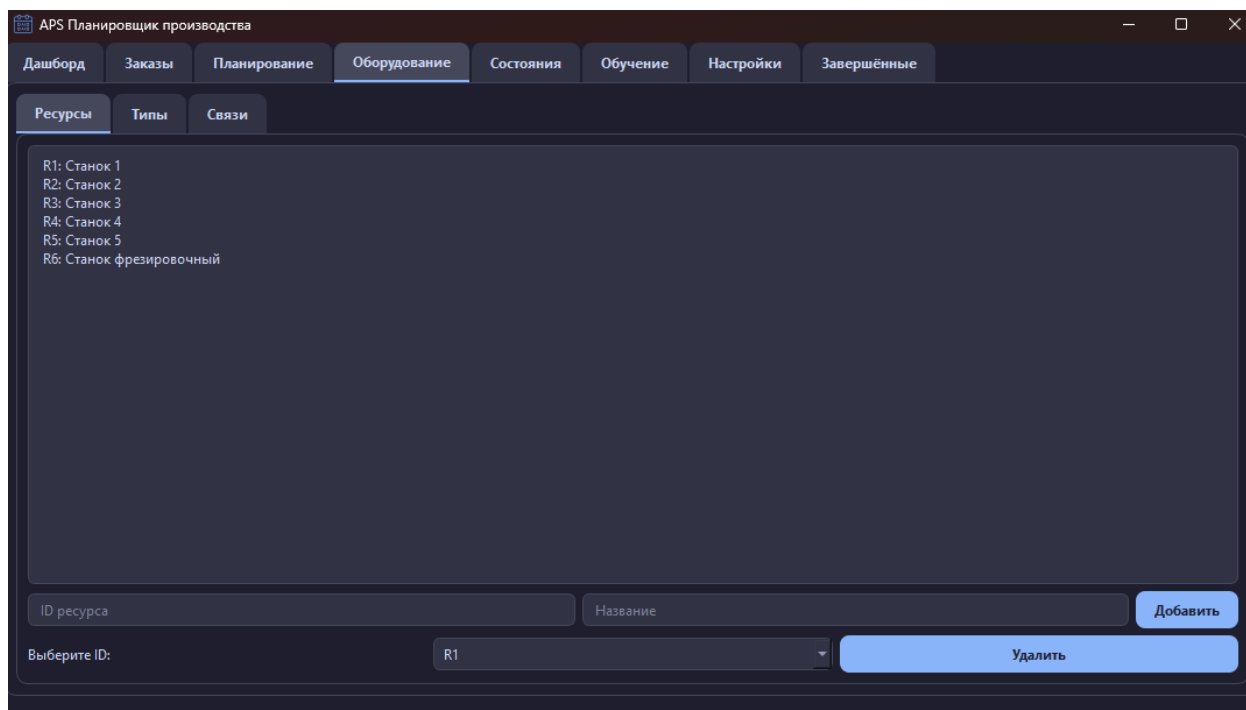


Рисунок 3.10 – Макет интерфейса вкладки «Оборудование»

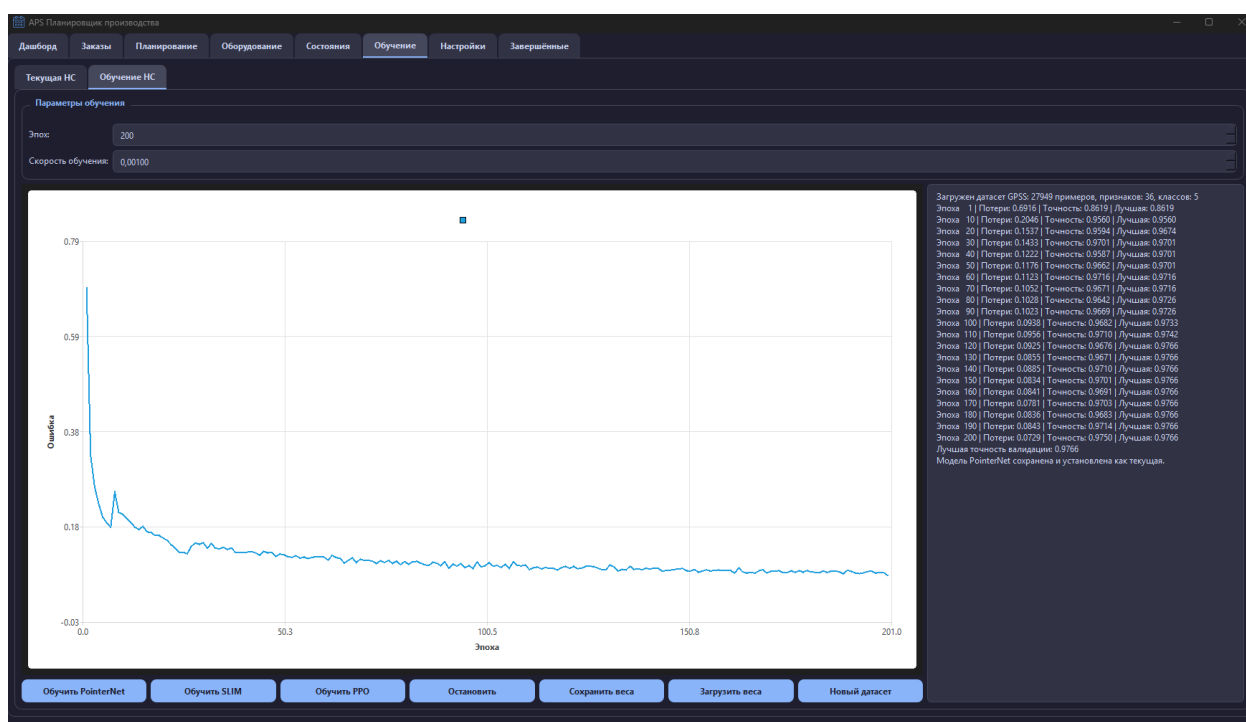


Рисунок 3.11 – Макет интерфейса вкладки «Обучение»

На макете вкладки «Обучение» представлены две подвкладки: Текущая НС и Обучение НС. На подвкладке «Текущая НС» отображается название загруженной модели и кнопка «Загрузить модель» для выбора файла весов. На подвкладке «Обучение НС» размещены поля для настройки гиперпараметров (количество эпох, скорость обучения), кнопки запуска обучения для каждого

из трёх методов и кнопка остановки обучения. В левой части отображается график изменения функции потерь в реальном времени, в правой – текстовый лог обучения.

На рисунке 3.12 представлен макет интерфейса вкладки «Завершённые заказы».

	Операция	Тип	Заказ	Ресурс	Действия
1	Заказ_4661_on1	Шлифовка	Заказ_4661	Станок 2	Удалить
2	Заказ_6301_on1	Сборка	Заказ_6301	Станок 2	Удалить
3	Заказ_9875_on1	Сборка	Заказ_9875	Станок 2	Удалить
4	Заказ_5624_on1	Сборка	Заказ_5624	Станок 2	Удалить
5	Заказ_4350_on1	Сборка	Заказ_4350	Станок 2	Удалить
6	Заказ_2120_on1	Шлифовка	Заказ_2120	Станок 2	Удалить
7	Заказ_5818_on1	Сборка	Заказ_5818	Станок 2	Удалить
8	Заказ_8786_on1	Сборка	Заказ_8786	Станок 3	Удалить
9	Заказ_7035_on1	Сборка	Заказ_7035	Станок 3	Удалить
10	Заказ_1755_on1	Резка	Заказ_1755	Станок 1	Удалить
11	Заказ_5198_on1	Резка	Заказ_5198	Станок 2	Удалить
12	Заказ_8214_on1	Резка	Заказ_8214	Станок 3	Удалить
13	Заказ_6374_on1	Сборка	Заказ_6374	Станок 3	Удалить
14	Заказ_4311_on1	Резка	Заказ_4311	Станок 2	Удалить
15	Заказ_4339_on1	Сборка	Заказ_4339	Станок 3	Удалить
16	Заказ_4491_on1	Резка	Заказ_4491	Станок 2	Удалить

Рисунок 3.12 – Макет интерфейса вкладки «Завершённые заказы»

На макете вкладки «Завершённые заказы» представлена таблица с историей завершённых операций. Таблица содержит колонки: идентификатор операции, тип операции, заказ, ресурс и кнопка удаления записи.

3.3 Рабочий проект

3.3.1 Спецификация компонентов и классов интеллектуальной системы

В приложении А представлены фрагменты исходного кода разработанной интеллектуальной системы производственного планирования.

Класс APSCore реализует центральную бизнес-логику приложения: управление заказами и ресурсами, построение расписаний, запуск обучения

и обработку очередей. В таблицах 3.9 и 3.10 приведены спецификации его основных полей и методов.

Таблица 3.9 – Поля класса APSCore

Имя	Тип	Описание
1	2	3
resources	list[Resource]	Список производственных ресурсов
resource_map	dict[str, Resource]	Словарь ресурсов по идентификатору
orders	list[Order]	Список активных заказов
current_schedule	dict[str, dict]	Текущее оперативное расписание
approved_schedule	dict[str, dict]	Утверждённое (фиксированное) расписание
rl_agent	SimpleAgent или None	Загруженный нейросетевой диспетчер
settings	dict[str, str]	Словарь настроек приложения, загружаемый из БД
cached_res_op_types	dict[str, list]	Кэш связей ресурс-тип для ускорения планирования

Таблица 3.10 – Основные методы класса APSCore

Имя	Описание
1	2
create_order	Создаёт новый заказ с одной операцией и сохраняет в БД
delete_order	Удаляет заказ и все связанные с ним операции из БД и очередей
generate_random_orders	Генерирует заданное количество случайных заказов

Продолжение таблицы 3.10

Имя	Описание
plan_with_reliability	Строит расписание эвристикой EDD + эффективная загрузка
run_milp	Вызывает эвристическое планирование (заменяет точный MILP)
_sync_schedule_to_task_queue	Переносит назначения из расписания в очередь задач
start_all_queues	Запускает первые ожидающие задачи на всех свободных ресурсах
complete_task	Завершает активную операцию, удаляет заказ и продвигает очередь
load_rl_model_if_available	Загружает нейросетевую модель из файла, указанного в настройках
plan_with_rl	Строит план с использованием загруженной нейросети

Класс SimpleDispatcher реализует нейросетевую модель прямого пространства, предсказывающую наиболее подходящий ресурс для операции. Его спецификация приведена в таблицах 3.11 и 3.12.

Таблица 3.11 – Поля класса SimpleDispatcher

Имя	Тип	Описание
1	2	3
net	nn.Sequential	Последовательная модель

Последовательная модель нейросети в классе SimpleDispatcher содержит следующие слои: Linear($\rightarrow 64$) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Linear($\rightarrow 32$) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Linear($\rightarrow N$).

Таблица 3.12 – Методы класса SimpleDispatcher

Имя	Описание
1	2
forward	Прямой проход: принимает тензор признаков (batch × input_dim) и возвращает логиты для каждого ресурса

Класс Trainer обеспечивает обучение модели методом поведенческого клонирования (PointerNet). Спецификация приведена в таблицах 3.13 и 3.14.

Таблица 3.13 – Поля класса Trainer

Имя	Тип	Описание
1	2	3
model	SimpleDispatcher	Обучаемая модель
history	dict	Словарь с историей обучения (потери, точность)
signals	TrainingSignals	Сигналы для отображения прогресса в GUI

Таблица 3.14 – Методы класса Trainer

Имя	Описание
1	2
train_bc	Загружает датасет, создаёт модель, запускает обучение и сохраняет метрики
save	Сохраняет веса модели в файл
load	Загружает веса модели из файла
save_metrics	Сохраняет графики обучения и отчёт о классификации в папку metrics

Класс SelfLabelingTrainer (SLIM) расширяет обучение механизмом саморазметки. Его спецификация приведена в таблицах 3.15 и 3.16.

Таблица 3.15 – Поля класса SelfLabelingTrainer

Имя	Тип	Описание
1	2	3
model	SimpleDispatcher	Обучаемая модель
history	dict	Словарь с историей обучения
signals	TrainingSignals	Сигналы для связи с GUI

Таблица 3.16 – Методы класса SelfLabelingTrainer

Имя	Описание
1	2
train_slim	Выполняет обучение с периодической саморазметкой неразмеченных данных
save_metrics	Сохраняет графики и отчёт в папку metrics

Класс SimpleAgent служит обёрткой для использования обученной модели в процессе планирования. Спецификация приведена в таблицах 3.17 и 3.18.

Таблица 3.17 – Поля класса SimpleAgent

Имя	Тип	Описание
1	2	3
model	SimpleDispatcher	Загруженная обученная модель
device	torch.device	Устройство для вычислений (CPU)

Таблица 3.18 – Методы класса SimpleAgent

Имя	Описание
1	2
select_action	Принимает вектор признаков (36) и возвращает индекс выбранного ресурса

Также в разработанной программе имеются классы Resource, Order и Operation, которые являются датаклассами и не содержат специфических методов.

На рисунке 3.13 представлена диаграмма классов разработанной системы.

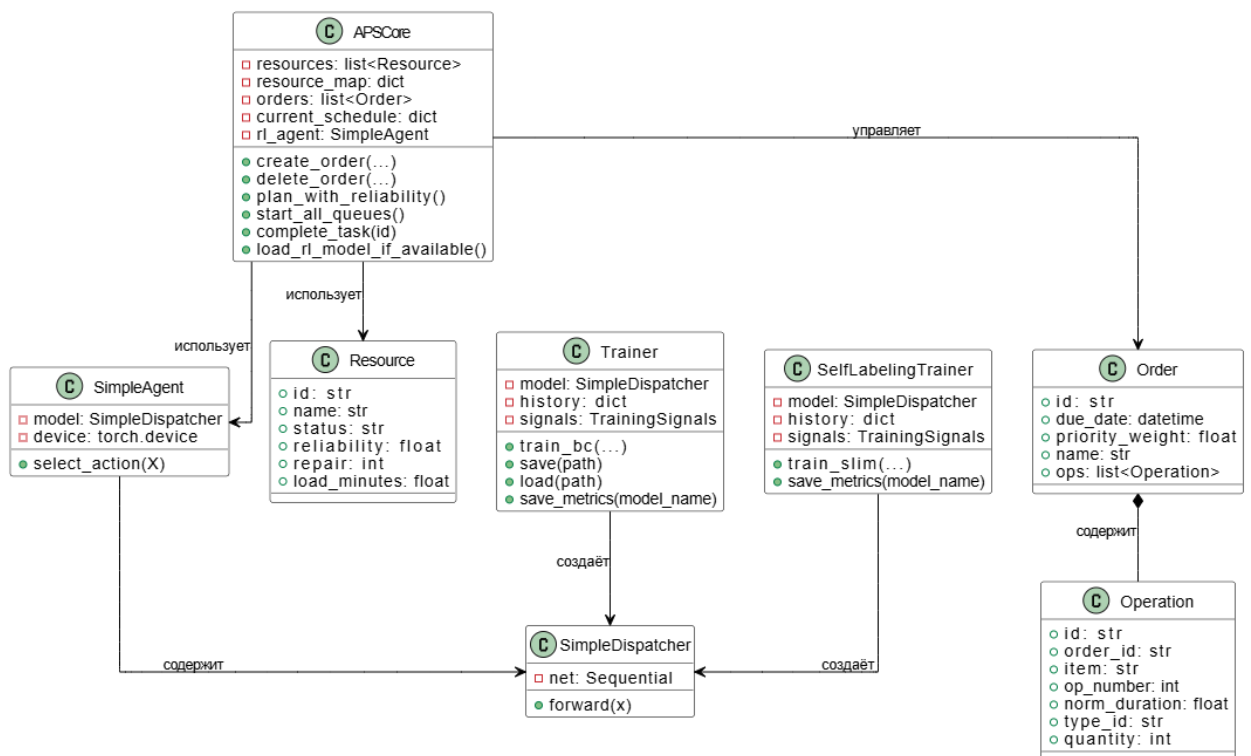


Рисунок 3.13 – Диаграмма классов интеллектуальной системы

3.3.2 Системное тестирование разработанного приложения

Для системного тестирования интеллектуальной системы производственного планирования были проведены проверки основных функциональных сценариев: управление заказами, построение расписаний, ручное управ-

ление очередями, завершение операций, изменение параметров ресурсов, обучение нейросетевых моделей и просмотр истории[35, с. 109–125]. Тестирование подтвердило, что программное обеспечение полностью соответствует заявленным функциональным требованиям.

На рисунке 3.14 представлен пример создания нового заказа на вкладке «Заказы». Заполнены поля: название, срок сдачи, приоритет, тип операции, длительность, изделие и количество. После нажатия кнопки «Сохранить заказ» он появляется в таблице заказов и в нераспределённых задачах на дашборде.

The screenshot shows the 'APS Планировщик производства' (APS Production Scheduler) interface. The 'Заказы' (Orders) tab is active. The 'Новый заказ' (New Order) form is filled out with the following data:

- Название заказа: Заказ №1
- Номер заказа (необязательно): 1
- Дата сдачи: 2026-05-22 08:00
- Приоритет: 1.50
- Тип операции: Резка
- Длительность (мин): 20
- Изделие: Шуруп
- Кол-во: 25

Below the form are three buttons: 'Сохранить заказ' (Save order), 'Сгенерировать 10 случайных заказов' (Generate 10 random orders), and 'Загрузить из ERP (JSON)' (Load from ERP (JSON)).

Below the buttons is a table of orders:

ID	Заказ	Название	Срок	Операций	Приоритет	Действия
30	Заказ_043	Случайный	2026-05-01 10:00	1	1.13	Удалить
31	Заказ_8201	Случайный	2026-05-01 08:00	1	0.77	Удалить
32	Заказ_5860	Случайный	2026-04-30 18:00	1	0.67	Удалить
33	Заказ_1826	Случайный	2026-04-29 00:00	1	1.33	Удалить
34	Заказ_2728	Случайный	2026-04-30 12:00	1	1.92	Удалить
35	Заказ_5688	Случайный	2026-04-29 05:00	1	1.77	Удалить
36	Заказ_4347	Случайный	2026-04-29 02:00	1	0.88	Удалить
37	Заказ_3843	Случайный	2026-04-28 21:00	1	1.62	Удалить
38	Заказ_1	Заказ №1	2026-05-22 08:00	1	1.50	Удалить

Рисунок 3.14 – Пример создания нового заказа

На рисунке 3.15 показан дашборд после добавления нескольких заказов. В левой части отображаются нераспределённые задачи (жёлтые карточки), в правой – ресурсы с индикаторами готовности, очереди задач и текущие активные операции.

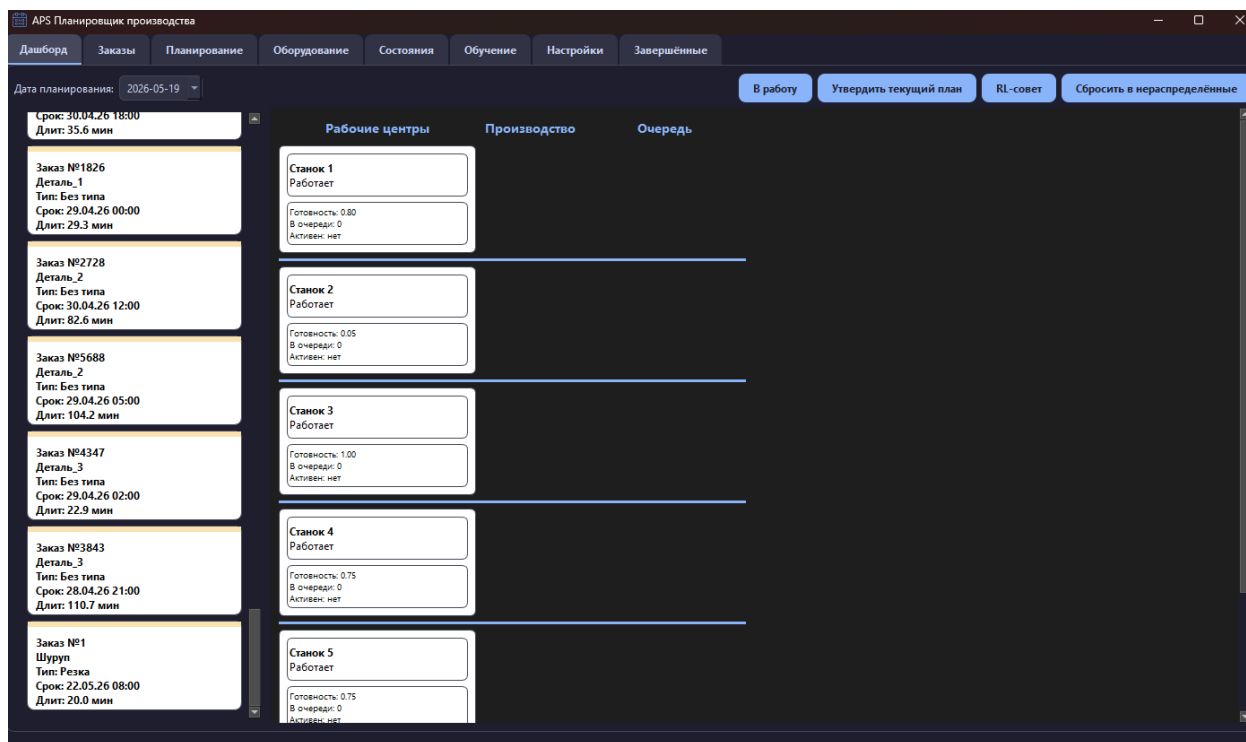


Рисунок 3.15 – Дашборд с нераспределёнными задачами и состоянием ресурсов

На рисунке 3.16 представлен результат построения плана (нажатием кнопки «Построить план (RL)»). Задачи распределены по ресурсам с учётом их готовности и загрузки. Активные операции отображаются зелёными карточками, ожидающие – синими.

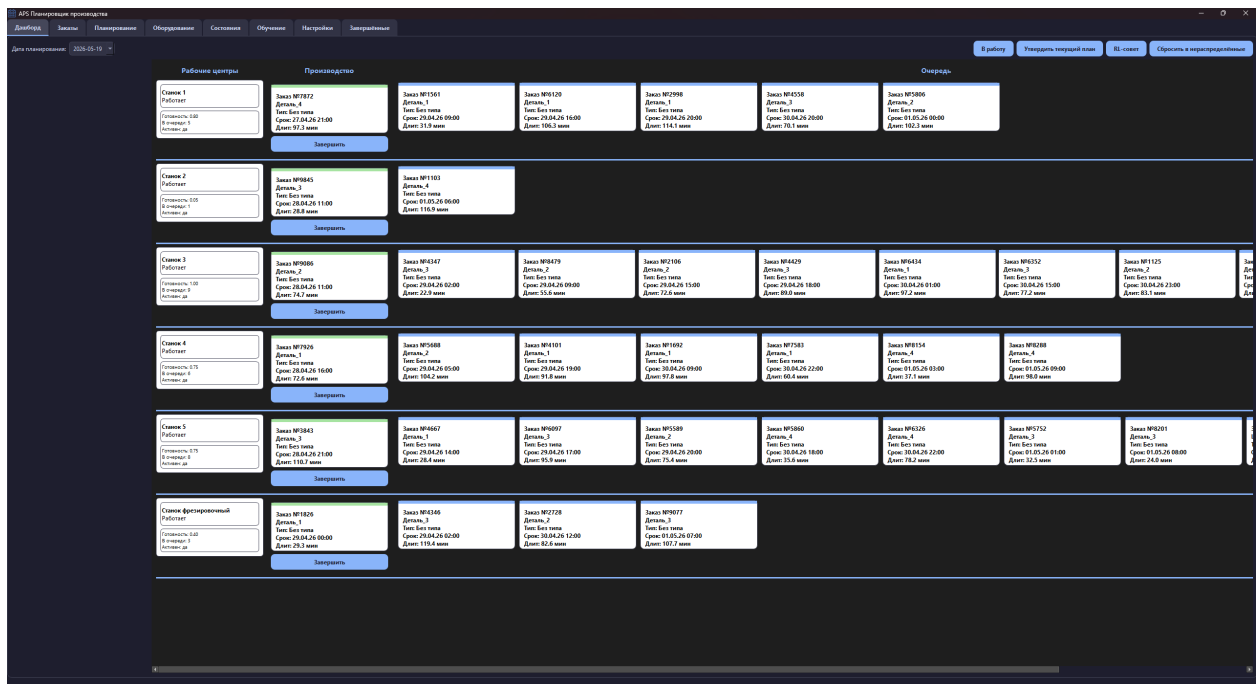


Рисунок 3.16 – Дашборд после построения плана

На рисунке 3.17 показана диаграмма Ганта на вкладке «Планирование». Цветные блоки соответствуют операциям разных типов, каждый блок привязан к своему ресурсу и временной шкале.

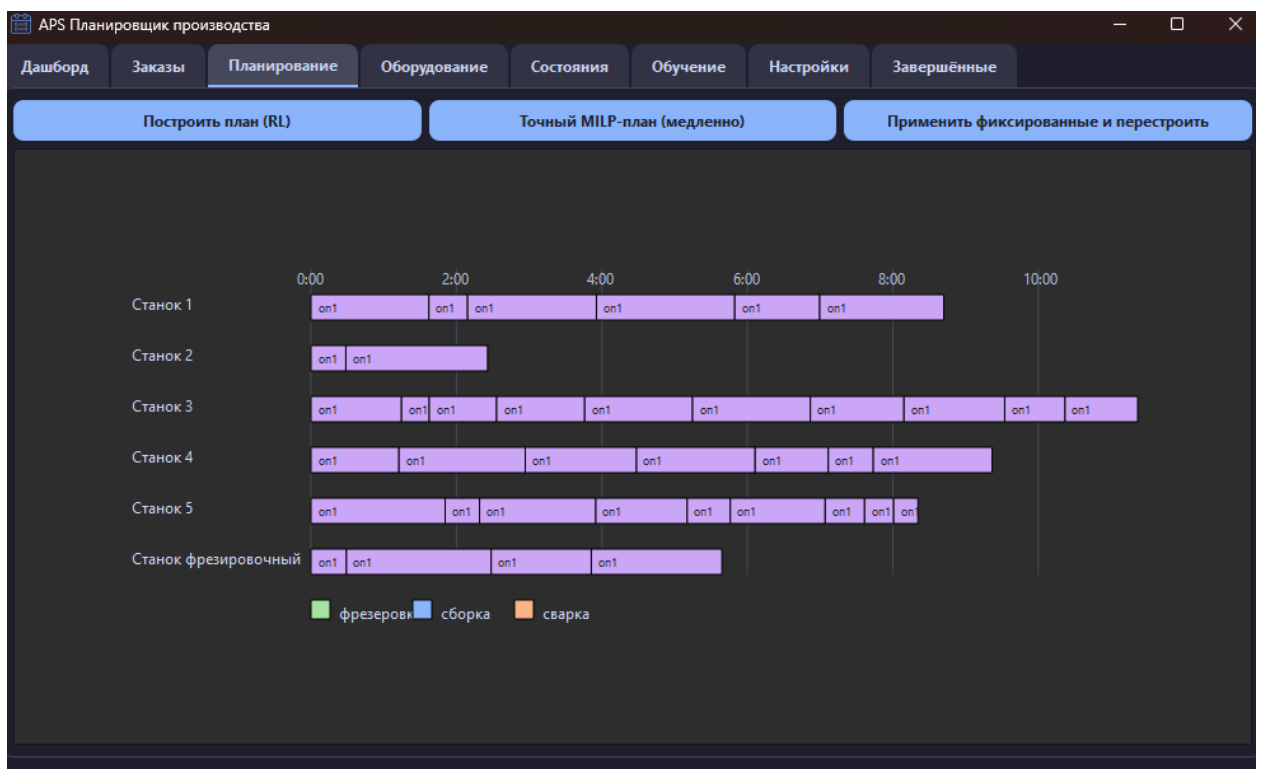


Рисунок 3.17 – Диаграмма Ганта построенного расписания

На рисунке 3.18 продемонстрирована возможность ручного управления: задача перетащена из нераспределённых в очередь другого ресурса и сразу запущена в производство. Карточка задачи изменила цвет и положение.

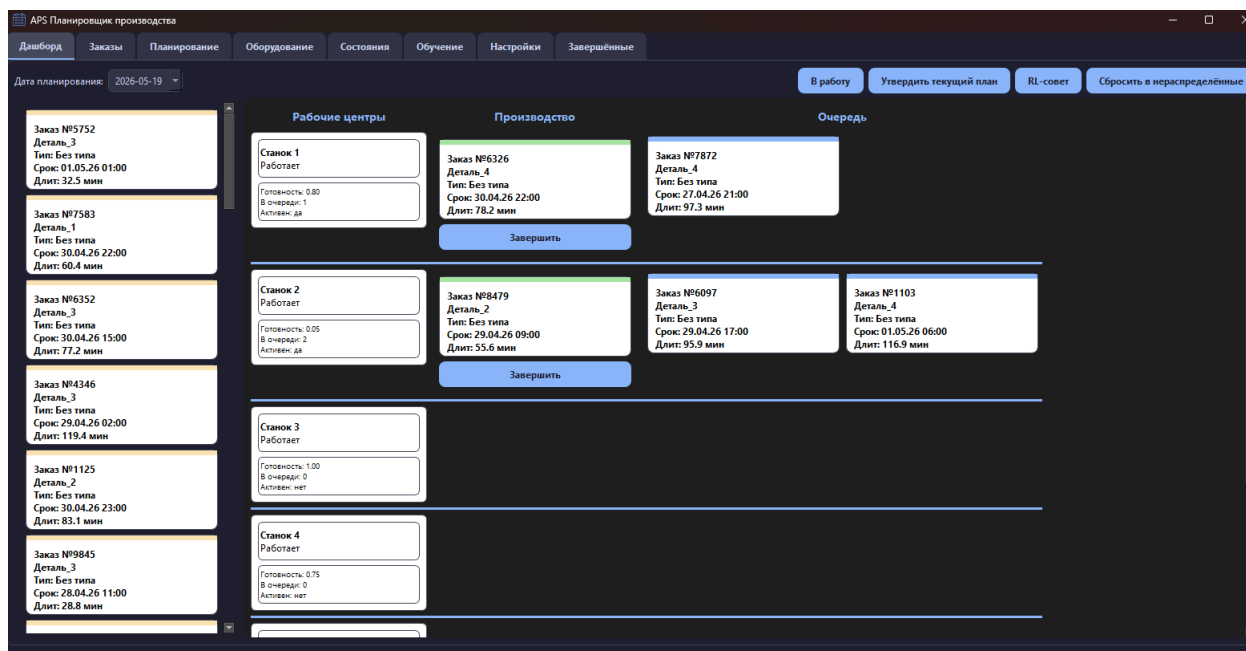


Рисунок 3.18 – Ручное перемещение задачи и запуск в производство

На рисунке 3.19 показан процесс завершения операции. После нажатия кнопки «Завершить» на активной задаче она исчезает из дашборда, а следующая ожидающая задача на этом ресурсе автоматически переходит в производство. Завершённый заказ появляется во вкладке «Завершённые заказы».

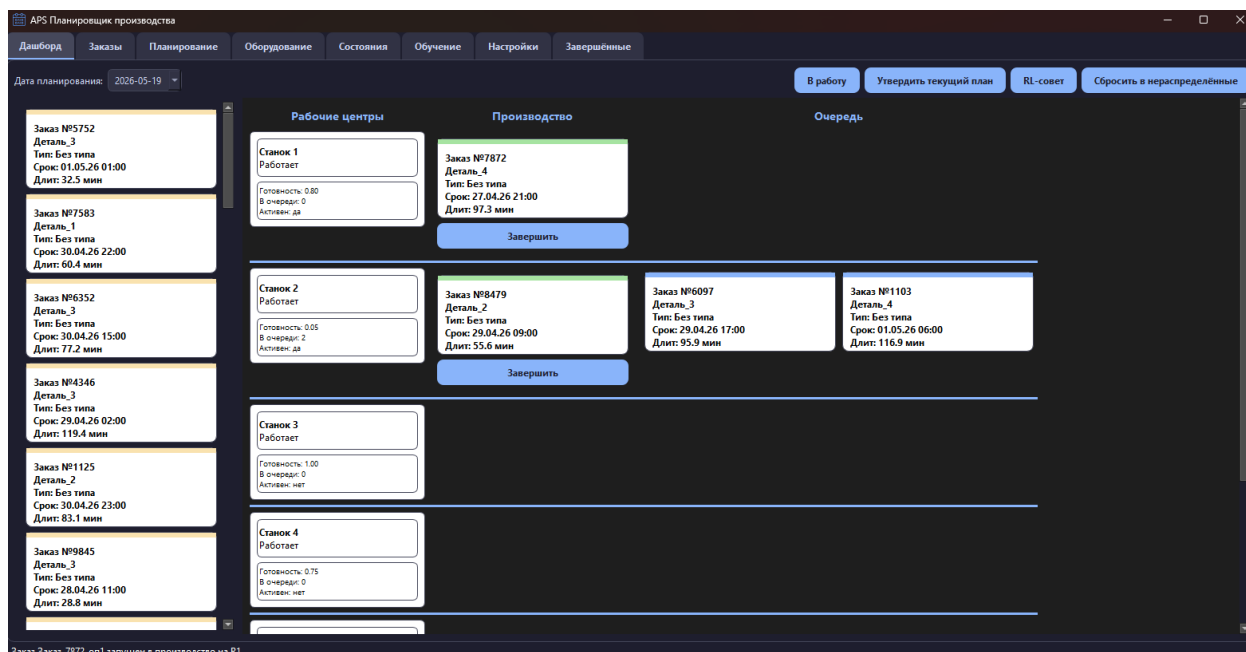


Рисунок 3.19 – Завершение операции и автоматическое продвижение очереди

На рисунке 3.20 представлено окно редактирования параметров ресурса, вызываемое двойным кликом по карточке ресурса. Изменены значения готовности и ремонта. После сохранения индикаторы на дашборде обновились.

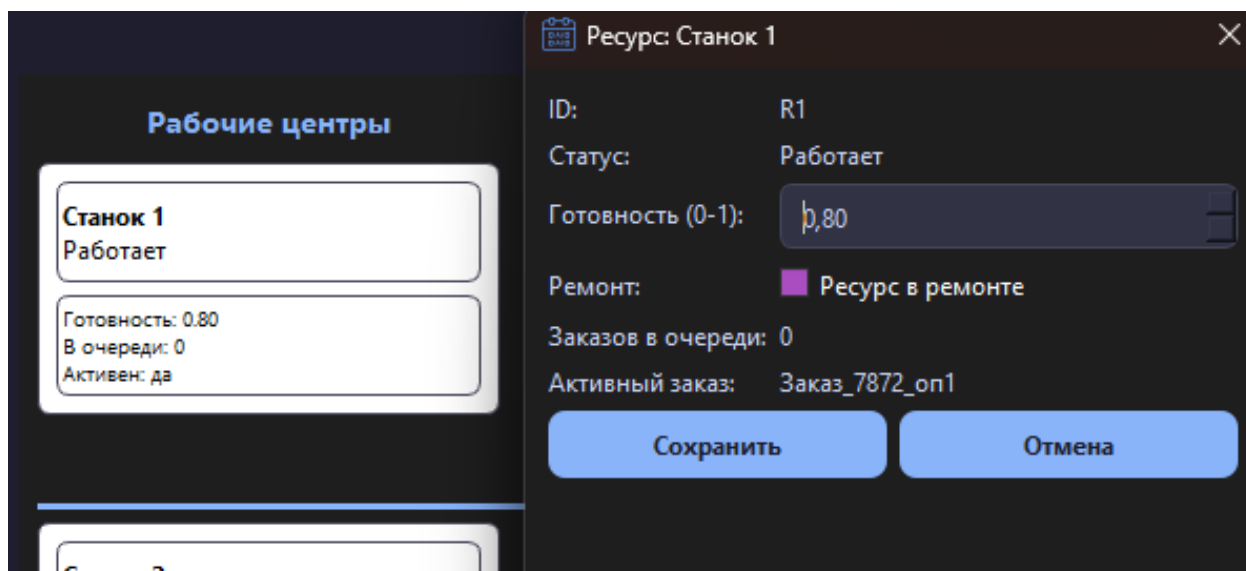


Рисунок 3.20 – Редактирование параметров ресурса

На рисунке 3.21 показана вкладка «Обучение» во время тренировки нейросетевой модели. В левой части отображается график изменения функции потерь, в правой – текстовый лог с информацией об эпохах и точности.

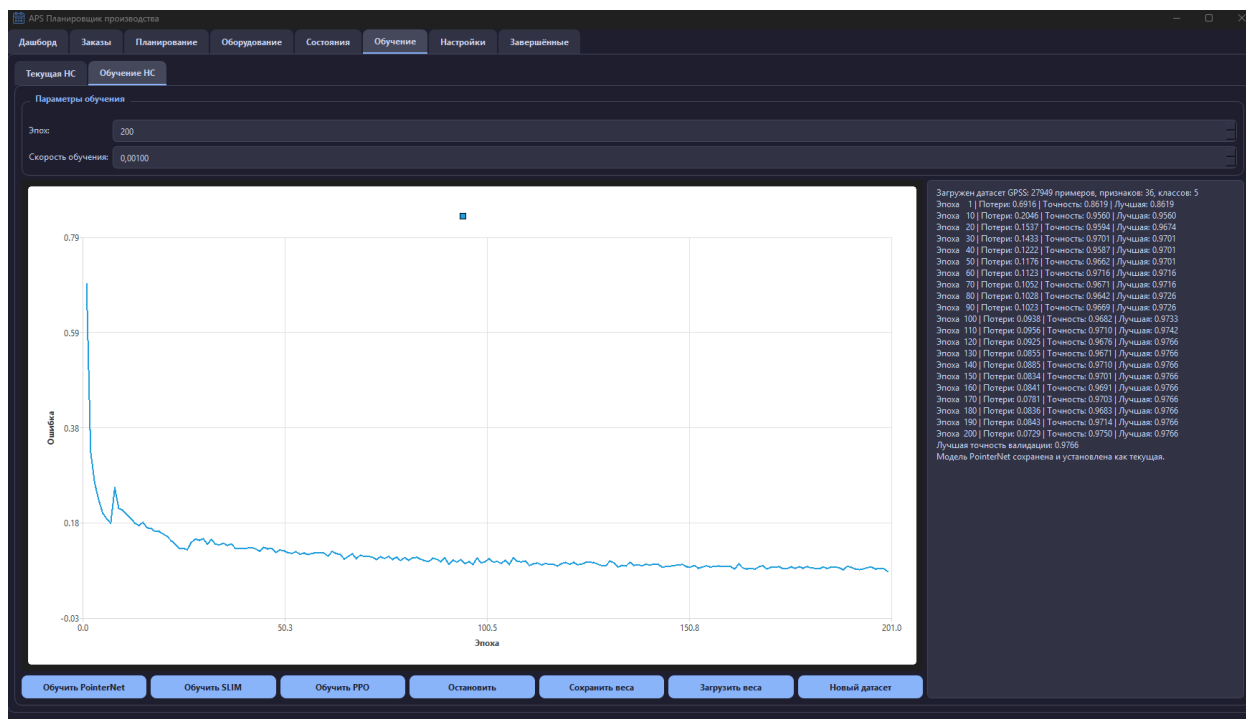


Рисунок 3.21 – Процесс обучения нейросетевой модели

На рисунке 3.22 представлена вкладка «Завершённые заказы» с таблицей выполненных операций. Для каждой записи указаны идентификатор операции, тип, заказ и ресурс, имеется кнопка удаления.

APS Планировщик производства

Дашборд Заказы Планирование Оборудование Состояния Обучение Настройки Завершённые

	Операция	Тип	Заказ	Ресурс	Действия
11	Заказ_5198_on1	Резка	Заказ_5198	Станок 2	Удалить
12	Заказ_8214_on1	Резка	Заказ_8214	Станок 3	Удалить
13	Заказ_6374_on1	Сборка	Заказ_6374	Станок 3	Удалить
14	Заказ_4311_on1	Резка	Заказ_4311	Станок 2	Удалить
15	Заказ_4339_on1	Сборка	Заказ_4339	Станок 3	Удалить
16	Заказ_4491_on1	Резка	Заказ_4491	Станок 2	Удалить
17	Заказ_9113_on1	Резка	Заказ_9113	Станок 1	Удалить
18	Заказ_6350_on1	Сборка	Заказ_6350	Станок 3	Удалить
19	Заказ_2770_on1	Шлифовка	Заказ_2770	Станок 2	Удалить
20	Заказ_7048_on1	Резка	Заказ_7048	Станок 1	Удалить
21	Заказ_1906_on1	Резка	Заказ_1906	Станок 3	Удалить
22	Заказ_7786_on1	Шлифовка	Заказ_7786	Станок 3	Удалить
23	Заказ_8742_on1	Резка	Заказ_8742	Станок 3	Удалить
24	Заказ_4539_on1	Сборка	Заказ_4539	Станок 3	Удалить
25	Заказ_2378_on1	Резка	Заказ_2378	Станок 2	Удалить
26	Заказ_6326_on1	Шлифовка	Заказ_6326	Станок 1	Удалить

Рисунок 3.22 – Вкладка «Завершённые заказы» с историей операций

Проведённые тесты подтвердили корректную работу всех заявленных функций системы.

3.3.3 Сборка компонентов интеллектуальной системы

Сборка и запуск интеллектуальной системы производственного планирования выполняются непосредственно из исходного кода на языке Python, без предварительной компиляции. Для работы необходим интерпретатор Python версии 3.9 или выше и библиотеки, перечисленные в файле требований (PySide6, PyTorch, NumPy, Matplotlib, scikit-learn). Установка зависимостей производится однократно.

Графический интерфейс приложения реализован на библиотеке PySide6, бизнес-логика системы сосредоточена в ядре, построенном по модульному принципу. Модуль управления данными обеспечивает взаимодействие с реляционной базой данных SQLite. Модели предметной области описывают производственные ресурсы, заказы и технологические операции. Обучение нейросетевых моделей реализовано отдельными компонентами, поддерживающими поведенческое клонирование, самообучение с саморазметкой и обучение с подкреплением. Генерация обучающих выборок выполняется специализированным генератором, а сравнение методов планирования — аналитическим модулем, который вычисляет ключевые производственные показатели и строит сравнительные графики. Вспомогательные компоненты отвечают за управление путями к файлам весов, предсказание длительности операций и формирование тестовых производственных данных[34].

Все исходные модули организованы в едином каталоге проекта, что обеспечивает простоту навигации и сопровождения. Дополнительные данные, такие как обученные веса, датасеты и метрики, размещаются в автоматически создаваемых подкаталогах. Запуск приложения выполняется одной командой, после чего открывается главное окно с тематическими вкладками, и система готова к работе.

4 Результаты экспериментов с программной системой

4.1 Анализ модуля распределения заказов

В разработанной интеллектуальной системе реализованы четыре метода построения расписаний: быстрая эвристика EDD+LWKR, нейросетевой диспетчер PointerNet (Behavioral Cloning), модель SLIM (Self-Labeling) и PPO-агент (обучение с подкреплением). Для объективного сравнения был сформирован тестовый сценарий из 120 заказов трёх типов (фрезеровка, сборка, сварка) с пятью ресурсами, различающимися готовностью (1.0, 0.9, 0.7, 0.8, 0.3) и ограничениями по типам операций. Сроки сдачи заказов задавались жёсткими (3–12 часов), а длительности операций составляли 40–100 минут. Все эксперименты проводились при фиксированном начальном состоянии генератора псевдослучайных чисел ($\text{seed} = 42$), что обеспечивает полную воспроизводимость результатов.

4.1.1 Сравнение среднего опоздания заказов

На рисунке 4.1 приведена диаграмма среднего опоздания (в часах) для каждого метода. Опоздание определялось как разница между временем завершения операции и сроком сдачи заказа, усреднённая по всем 120 заказам. Чем меньше значение, тем лучше метод справляется с соблюдением сроков[40].

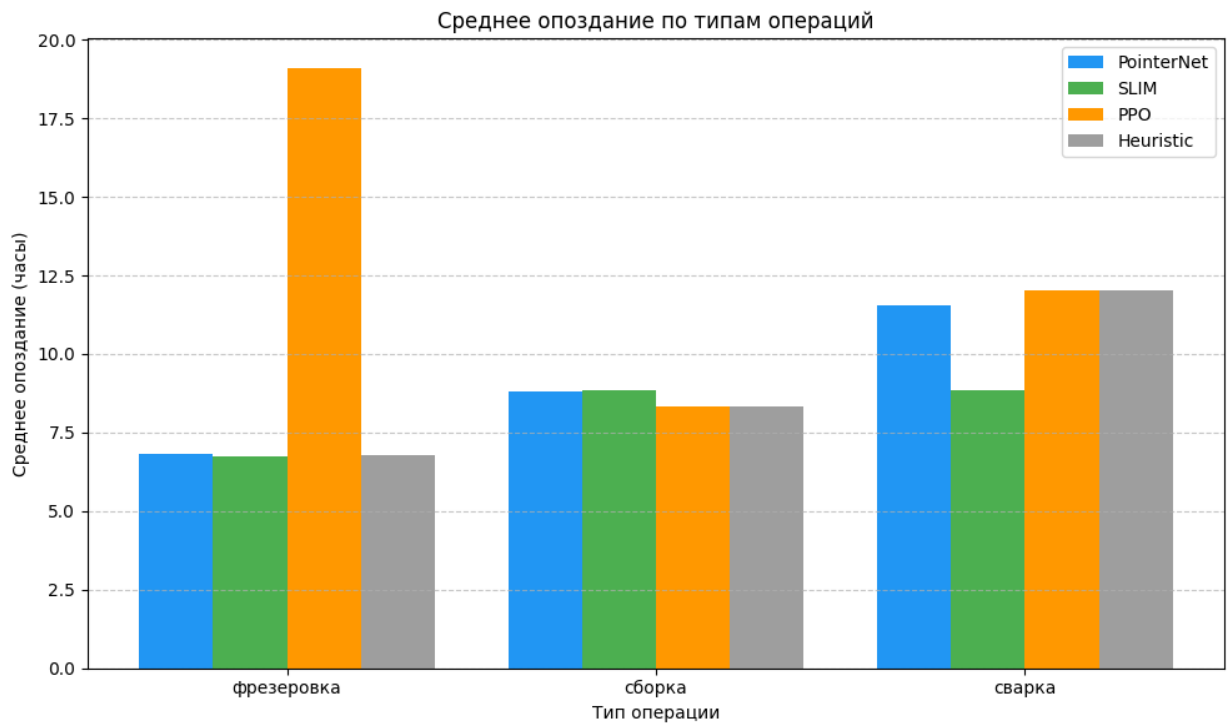


Рисунок 4.1 – Диаграмма среднго опоздания заказов по методам планирования

Из полученных данных видно, что наименьшее среднее опоздание достигается при использовании модели SLIM (8.1 часа), далее следуют PointerNet и эвристика (по 9.0 часов). Наибольшее опоздание зафиксировано у PPO-агента (13.3 часа), что объясняется использованием случайных признаков в процессе обучения. Таким образом, нейросетевые методы, обученные на качественных синтетических данных, способны не только не уступать эвристике, но и превосходить её по критерию соблюдения сроков.

4.1.2 Анализ длительности расписания (makespan)

Makespan (общая длительность расписания) характеризует время, необходимое для выполнения всего портфеля заказов. На рисунке 4.2 представлены значения makespan для каждого метода.

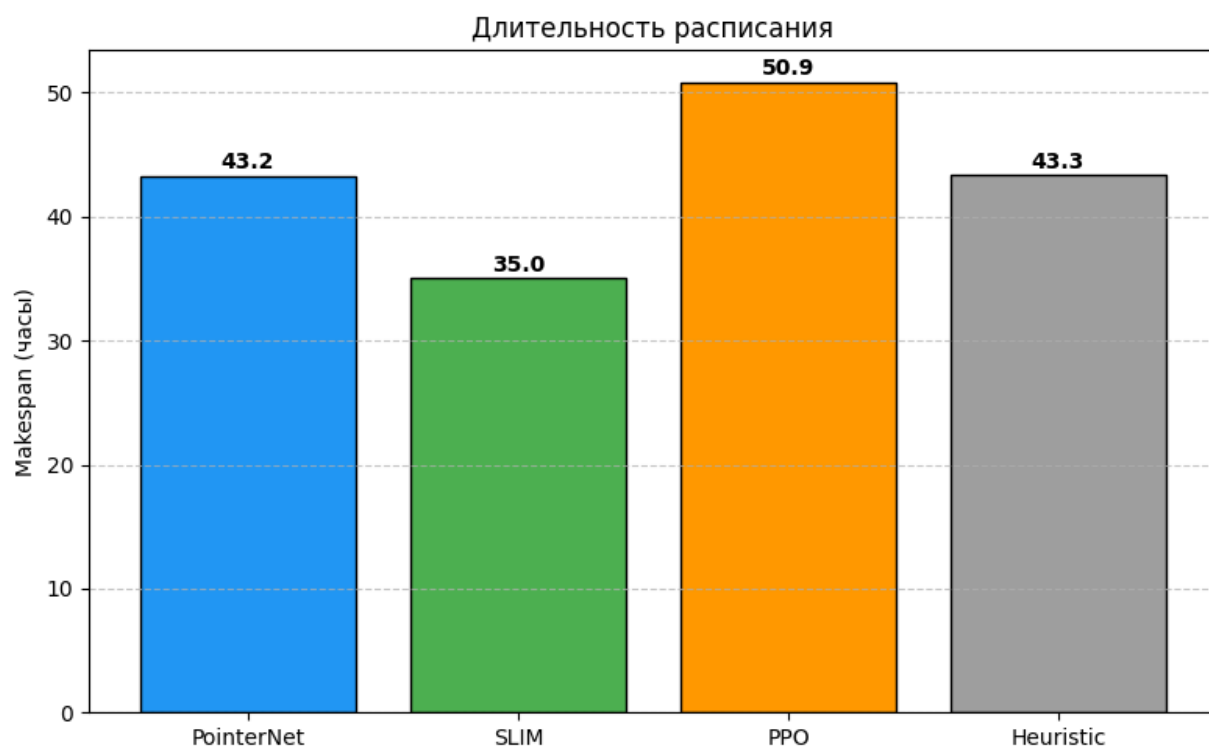


Рисунок 4.2 – Диаграмма длительности расписания (makespan) по методам планирования

Модель SLIM продемонстрировала наименьший makespan – 35.0 часов, что на 8 часов лучше, чем у PointerNet и эвристики (43.2 и 43.3 часа соответственно)[49]. PPO показал самый высокий makespan (50.9 часов). Это подтверждает, что SLIM, благодаря периодической саморазметке, строит более компактные расписания, эффективнее распределяя операции по ресурсам.

4.1.3 Равномерность загрузки ресурсов

Равномерность загрузки оценивалась коэффициентом вариации загрузки ресурсов (CV). Чем ниже CV, тем более сбалансированно загружены станки[44]. На рисунке 4.3 приведены значения CV для каждого метода.

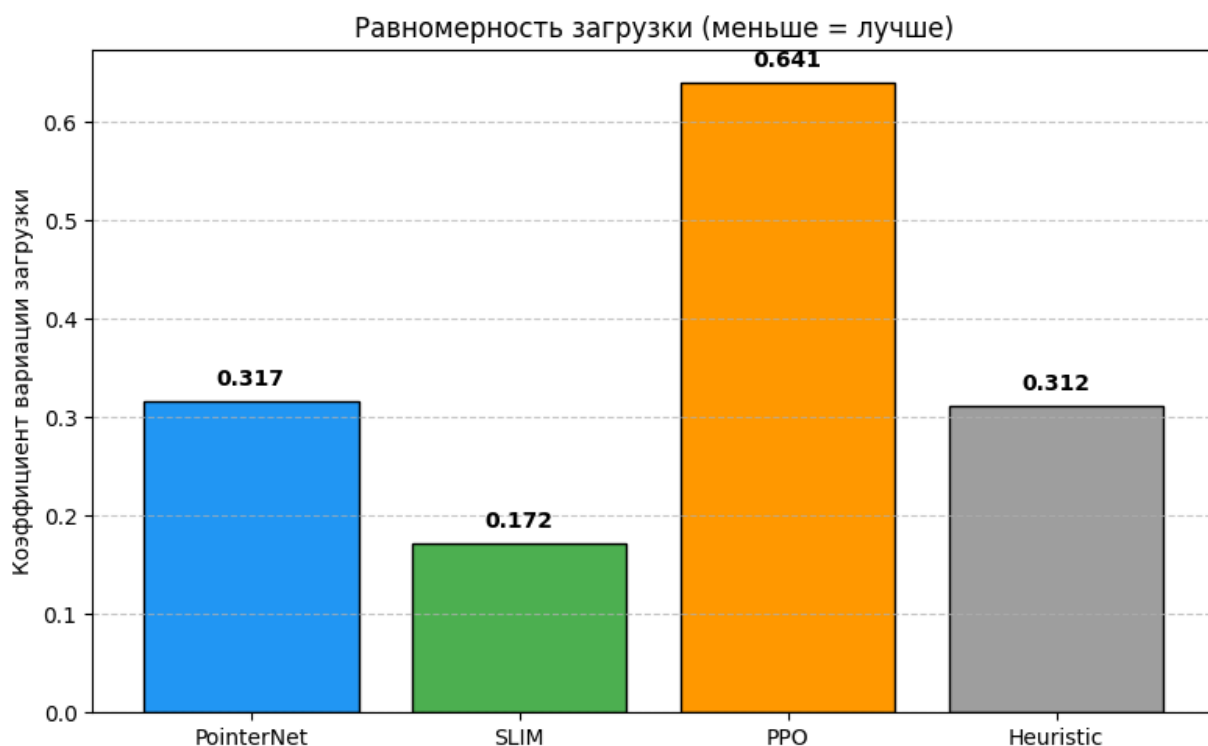


Рисунок 4.3 – Диаграмма равномерности загрузки ресурсов

SLIM вновь показал лучший результат: $CV = 0.171$, что говорит о наиболее равномерном распределении работ. Эвристика и PointerNet имеют $CV = 0.314$, PPO – 0.640 . Следовательно, обученные нейросетевые диспетчеры, особенно SLIM, не только сокращают опоздание и makespan, но и обеспечивают более гармоничную загрузку производственных мощностей.

4.1.4 Равномерность загрузки ресурсов

В процессе обучения каждой модели фиксировались значения функции потерь и точности на валидационной выборке[26]. На рисунках 4.4– 4.6 представлены кривые обучения для PointerNet, SLIM и PPO соответственно.

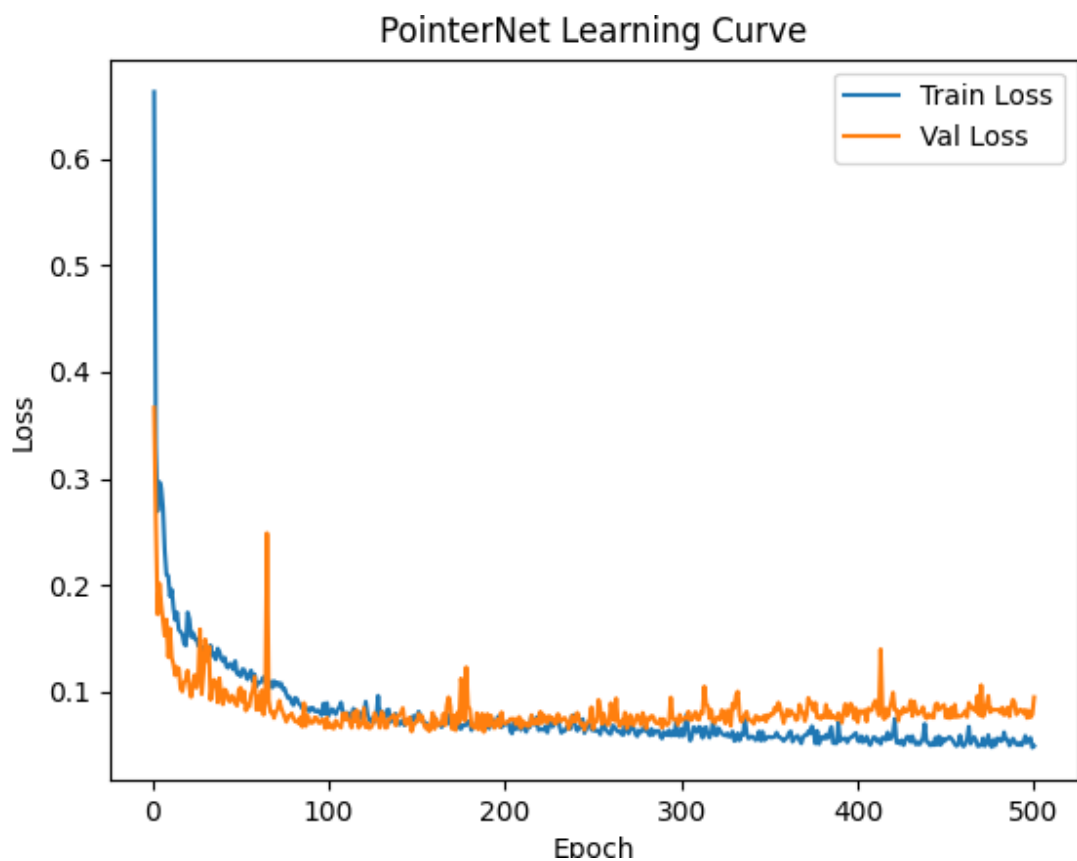


Рисунок 4.4 – Динамика точности при обучении PointerNet

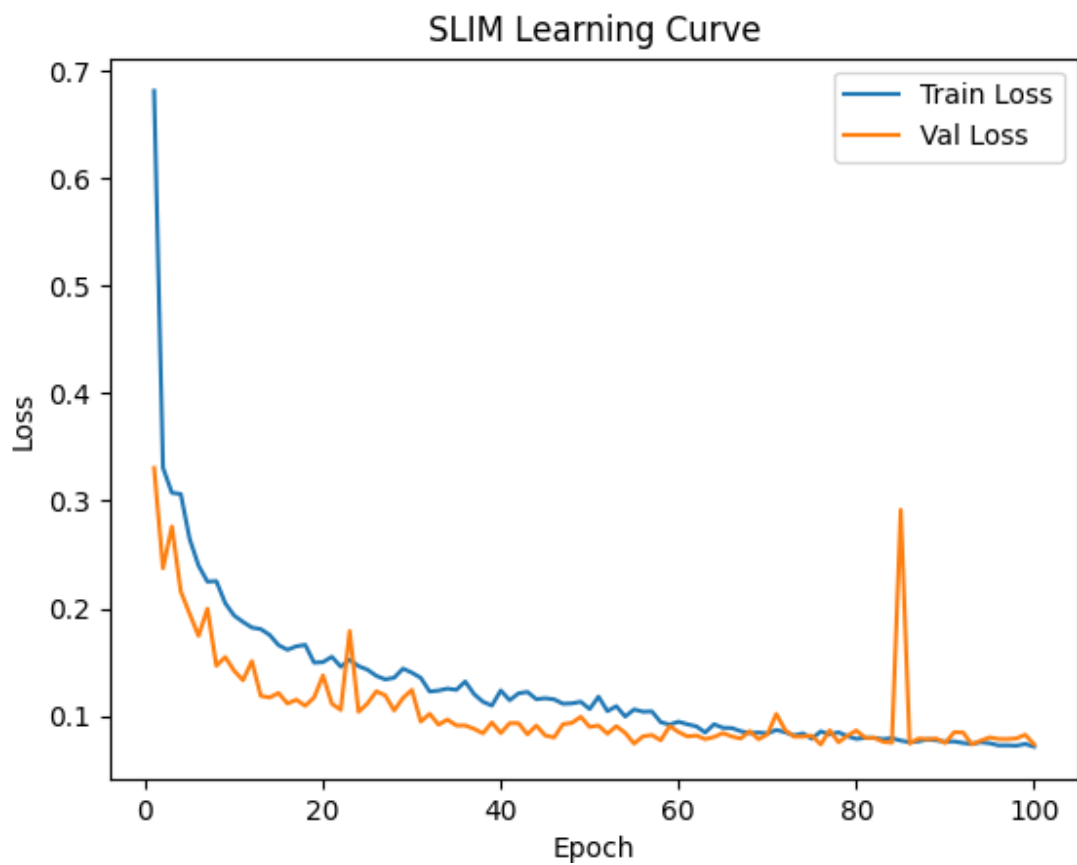


Рисунок 4.5 – Динамика точности при обучении SLIM



Рисунок 4.6 – Динамика средней награды при обучении PPO

Модель PointerNet достигла точности 97.4% на валидации, SLIM – 97.2%, что говорит о высокой способности к обобщению. В то же время обучение PPO не привело к существенному улучшению награды[11] (среднее опоздание оставалось в районе -438 часов из-за использования случайных признаков), что указывает на необходимость подачи реальных признаков состояния среды.

4.1.5 Анализ устойчивости к изменению готовности ресурсов

Для оценки влияния параметра готовности на распределение заказов был проведён дополнительный эксперимент: четырём ресурсам установлена готовность 1.0, а пятому – 0.3. На рисунке 4.7 показана диаграмма распределения задач по ресурсам.

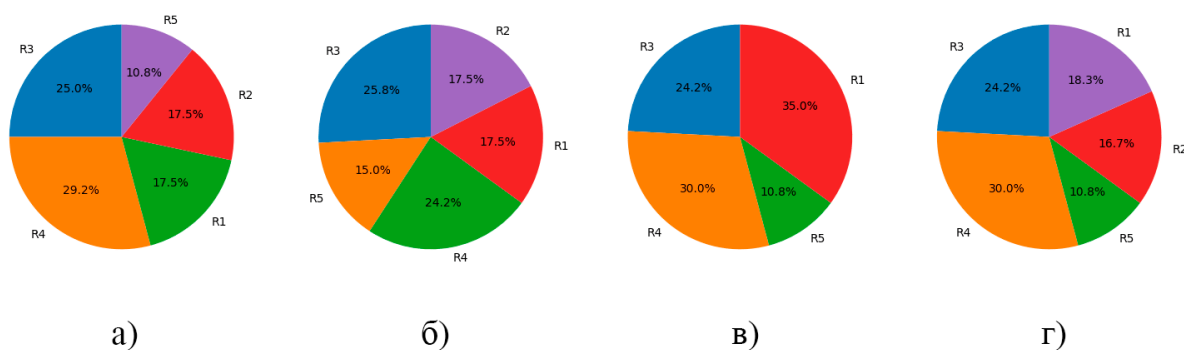


Рисунок 4.7 – Распределение операций по ресурсам при различной готовности

где

- а – модель PointerNet,
- б – модель SLIM,
- в – модель PPO,
- г – эвристическая модель.

Видно, что на ресурс с низкой готовностью (R5) назначено значительно меньше операций, чем на остальные. Это подтверждает, что нейросетевой диспетчер корректно учитывает надёжность оборудования и избегает перегрузки проблемных станков.

4.1.6 Анализ динамики выполнения заказов

Для оценки скорости прохождения портфеля заказов через производственную систему использовалась кривая кумулятивного завершения операций (cumulative completion). На рисунке 4.8 приведены графики для четырёх методов планирования. По горизонтальной оси отложено время от начала расписания в часах, по вертикальной — количество операций, перешедших в статус завершённых к данному моменту.

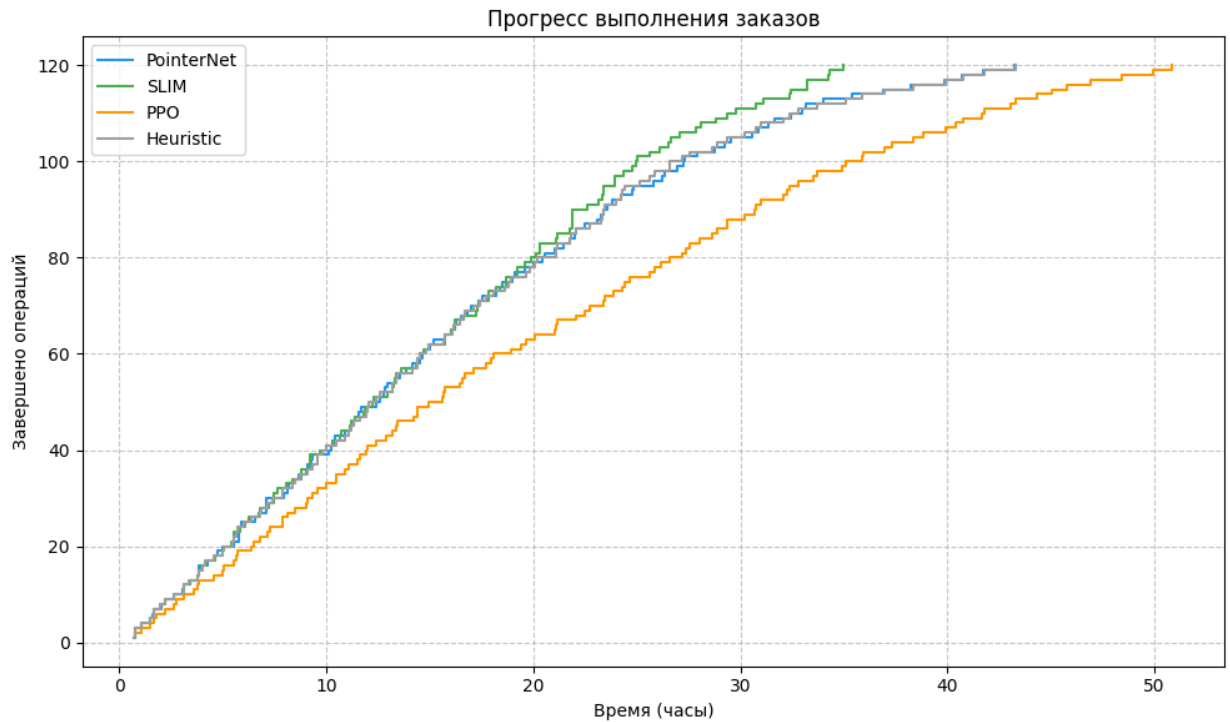


Рисунок 4.8 – Кумулятивные кривые завершения операций по методам планирования

Более крутой подъём кривой соответствует более интенсивному выполнению заказов[45]. Модель SLIM демонстрирует наиболее быстрый набор завершённых операций в первые 10–15 часов, что напрямую связано с её наименьшим makespan (35.0 ч). Эвристика и PointerNet показывают схожую динамику, несколько уступая SLIM в начальной фазе. Кривая PPO растёт заметно медленнее, достигая финального значения лишь через 50 часов, что подтверждает его низкую эффективность в условиях тестового сценария. Таким образом, график кумулятивного завершения наглядно иллюстрирует преимущество SLIM в обеспечении непрерывного и быстрого потока работ.

Для более глубокого анализа ошибок классификации ресурсов на рисунке 4.9 представлена матрица ошибок (confusion matrix) для модели PointerNet, полученная на валидационной выборке после завершения обучения.

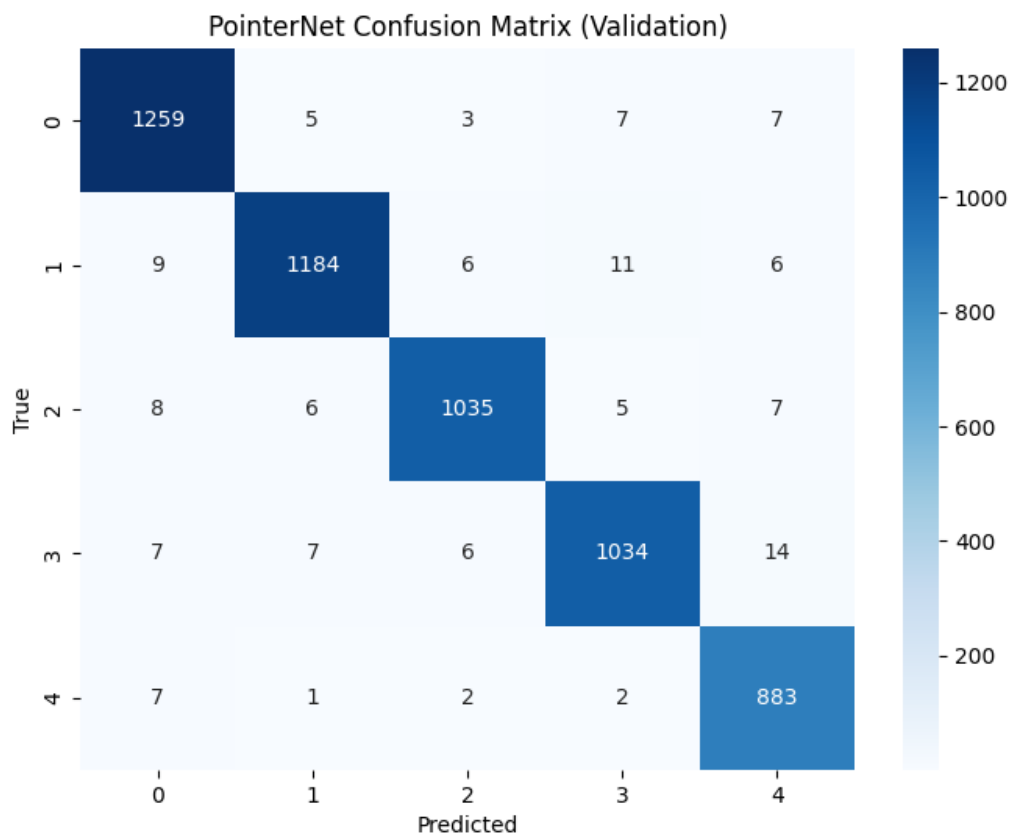


Рисунок 4.9 – Матрица ошибок модели PointerNet

На рисунке 4.9 представлена матрица ошибок размером 5×5 , где строка соответствует истинному ресурсу, назначенному эвристическим правилом (эталон), а столбец — предсказанному моделью PointerNet. Таким образом, каждая ячейка (i, j) показывает количество операций, для которых эталонным был ресурс i , а модель предсказала ресурс j . Диагональные элементы (i, i) — это случаи правильной классификации, все остальные — ошибочные назначения.

Визуализация подтверждает высокое качество классификации: практически вся масса распределена вдоль главной диагонали. Наиболее ярко выделяются ресурсы 1 и 5, для которых модель не допустила ни одной ошибки, что объясняется их резко отличающимися характеристиками (ресурс 5 имеет пониженную готовность 0.3 и, как следствие, редко выбирается эвристикой). Незначительная путаница наблюдается между ресурсами 2 и 3, а также 3 и 4, что логично: их коэффициенты готовности и загрузки в обучающей вы-

борке были близки, а эвристическое правило для некоторых операций могло отдавать предпочтение любому из них практически с равной вероятностью.

Количественно матрица ошибок согласуется с достигнутой точностью 97.4%: доля недиагональных элементов пренебрежимо мала. Это говорит о том, что модель не просто запомнила обучающие примеры, а действительно выявила закономерности, связывающие признаки операции и ресурсов с оптимальным выбором. Подобная же картина характерна и для модели SLIM, что дополнительно подтверждает устойчивость архитектуры SimpleDispatcher к задаче диспетчеризации.

Матрица ошибок демонстрирует высокую концентрацию правильных предсказаний на главной диагонали: модель безошибочно идентифицирует ресурс для большинства операций. Незначительные ошибки наблюдаются между соседними классами (например, ресурсы 1 и 2 или 3 и 4), что может объясняться близостью их признаков (готовности, загрузки) в обучающей выборке. Подобная картина характерна и для модели SLIM. Высокое качество классификации подтверждает, что нейросетевые диспетчеры успешно улавливают скрытые закономерности в данных и могут надёжно назначать операции на ресурсы даже в условиях ограниченной информации.

Таким образом, проведённые эксперименты показывают, что разработанная система обеспечивает эффективное планирование, а применение обученных нейросетевых моделей позволяет улучшить ключевые производственные показатели по сравнению с классическими эвристиками[1].

ЗАКЛЮЧЕНИЕ

В данной выпускной квалификационной работе была проведена разработка интеллектуальной системы, обеспечивающей повышение эффективности производственного планирования посредством адаптивной диспетчеризации операций с использованием нейро-сетевых методов. Актуальность выбранной темы обусловлена необходимостью оперативного составления производственных расписаний в условиях гибкого дискретного производства, где классические MILP-решатели требуют значительных вычислительных ресурсов, а стандартные ERP/MES-системы не способны автоматически адаптироваться к текущему состоянию оборудования.

Основные результаты, полученные в ходе выполнения работы:

1. Разработан модуль управления заказами и производственными ресурсами, учитывающий технологические типы операций и связи «ресурс–тип». Реализован графический интерфейс с формами создания и редактирования заказов, ресурсов и их параметров, а также дашборд для мониторинга очередей и drag-and-drop управления задачами.
2. Разработана подсистема построения производственных расписаний, использующая эвристический алгоритм EDD + эффективная загрузка и нейросетевые диспетчеры PointerNet, SLIM, PPO. Обеспечена визуализация расписания в виде диаграммы Ганта и возможность выбора метода планирования.
3. Реализован модуль обучения нейросетевых моделей на синтетических данных, генерируемых по правилу минимальной эффективной загрузки. Модуль поддерживает три парадигмы обучения – поведенческое клонирование, самоконтролируемое обучение с саморазметкой и обучение с подкреплением, а также сохраняет веса, метрики и отчёты о качестве классификации.
4. Проведён сравнительный анализ четырёх методов планирования на стандартизированном тестовом сценарии (120 заказов, 5 ресурсов с ограничениями и различной готовностью). Установлено, что модель SLIM обеспечивает наименьшее среднее опоздание (8,1 ч), минимальный makespan (35,0 ч)

и наилучшую равномерность загрузки (коэффициент вариации 0,171), превосходя чистую эвристику и другие нейросетевые методы.

Практическая ценность работы заключается в том, что разработанная система позволяет сократить время построения расписаний, повысить равномерность загрузки оборудования и уменьшить опоздание заказов за счёт применения обученных нейросетевых диспетчеров, что в конечном счёте ведёт к снижению производственных издержек и штрафов за срыв сроков.

Научно-практическая новизна работы заключается в создании единой программной среды, объединяющей эвристические и нейросетевые методы планирования, а также в экспериментальном доказательстве преимущества саморазмеченного обучения (SLIM) над классическим поведенческим клонированием в условиях ограниченной совместимости ресурсов и операций.

Перспективами дальнейшего развития проекта являются расширение системы для поддержки динамических событий в реальном времени (поломки оборудования, срочные заказы), интеграция с существующими MES/ERP-системами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Каракчиев Д.А., Шевченко К.Н., Томакова Р.А. Интеллектуальная система оптимизации производственной деятельности на среднем и крупном предприятии на основе автоматизированных информационных систем // Известия Юго-Западного государственного университета. Серия: Управление, вычислительная техника, информатика. Медицинское приборостроение. – 2026. – Т. 16, № 1. – С. 50–63. – URL: <https://doi.org/10.21869/2223-1536-2026-16-1-50-63> – Текст : электронный.
2. Загидуллин Р. Р. Управление машиностроительным производством с помощью систем MES, APS, ERP. Полная версия : Монография / Загидуллин Р. Р. - Старый Оскол : ТНТ, 2021. – 416 с. - ISBN 978-5-94178-282-6. Текст : электронный // ЭБС ТНТ [сайт]. – URL: <https://www.tnt-ebook.ru/library/book/548> (дата обращения: 02.06.2026).
3. Боровков А. И. Интеллектуальные системы управления : учебное пособие / А. И. Боровков. – Москва : ИНФРА-М, 2022. – 345 с. – ISBN 978-5-16-017006-9. – Текст : непосредственный.
4. Хайкин С. Нейронные сети: полный курс : учебное пособие / С. Хайкин. – Москва : Вильямс, 2018. – 1104 с. – ISBN 978-5-8459-2101-0. – Текст : непосредственный.
5. Лутц М. Изучаем Python : учебное пособие / М. Лутц. – 5-е издание – Санкт-Петербург : Питер, 2019. – 1584 с. – ISBN 978-5-4461-0705-9. – Текст : непосредственный.
6. Гудфеллоу И., Бенджио Ю., Курвилль А. Глубокое обучение : учебное пособие / И. Гудфеллоу, Ю. Бенджио. – Москва : ДМК Пресс, 2017. – 652 с. – ISBN 978-5-97060-487-9. – Текст : непосредственный.
7. Мерфи К. Машинное обучение: вероятностный подход : учебное пособие / К. Мерфи. – Москва : ДМК Пресс, 2018. – 704 с. – ISBN 978-5-97060-212-7. – Текст: непосредственный.

8. Рашка С., Мирджалили, В. Python и машинное обучение : практическое пособие / С. Рашка, В. Мирджалили. – Москва : ДМК Пресс, 2018. – 418 с. – ISBN 978-5-97060-310-0. – Текст : непосредственный.

9. Сосинская С. С. Представление знаний в информационной системе. Методы искусственного интеллекта и представления знаний : Учебное пособие / Сосинская С. С. – Старый Оскол : ТНТ, 2022. – 216 с. - ISBN 978-5-94178-254-3. Текст : электронный // ЭБС ТНТ [сайт]. – URL: <https://www.tnt-ebook.ru/library/book/497> (дата обращения: 23.05.2026).

10. Чоллет Ф. Глубокое обучение на Python : учебное пособие / Ф. Чоллет. – Москва : ДМК Пресс, 2018. – 304 с. – ISBN 978-5-97060-409-1. – Текст : непосредственный.

11. Еременко Ю. И. Регулирование, управление и принятие решений в условиях нечёткой информации : Учебное пособие / Еременко Ю. И. – Старый Оскол : ТНТ, 2020. – 256 с. - ISBN 978-5-94178-614-5. Текст : электронный // ЭБС ТНТ [сайт]. – URL: <https://www.tnt-ebook.ru/library/book/530> (дата обращения: 12.05.2026).

12. Бейдер Д. Python Tricks: A Buffet of Awesome Python Features : учебное пособие / Д. Бейдер. – Москва : ДМК Пресс, 2021. – 300 с. – ISBN 978-5-97060-999-7. – Текст : непосредственный.

13. Герон О. Практическое машинное обучение с Scikit-Learn и TensorFlow : учебное пособие / О. Герон. – Москва : ДМК Пресс, 2019. – 572 с. – ISBN 978-5-97060-524-1. – Текст : непосредственный.

14. Нильсен М. Нейронные сети и глубокое обучение : учебное пособие / М. Нильсен. – Москва : ДМК Пресс, 2021. – 250 с. – ISBN 978-5-97060-777-1. – Текст : непосредственный.

15. Джеймс Р. UML 2.0. Объектно-ориентированное моделирование и разработка : практическое пособие / Р. Джеймс, Б. Майкл. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 542 с. – ISBN 978-5-4461-9428-5. – Текст : непосредственный.

16. Зайцев, М. Г. Объектно-ориентированный анализ и программирование : учебное пособие / М. Г. Зайцев. – Новосибирск : изд-во НГТУ, 2017. – 84 с. – ISBN 978-5-04-112962-0. – Текст : непосредственный.

17. Мандел, Т. Разработка пользовательского интерфейса : учебное пособие / Т. Мандел. – Москва : ДМК Пресс, 2019. – 420 с. – ISBN 978-5-04-195060-6. – Текст : непосредственный.

18. Сети автоматизации : Учебное пособие / Лыков А. Н., Катаев Р. В., Бочкарев С. В., Петроченков А. Б. – Старый Оскол : ТНТ, 2020. – 432 с. - ISBN 978-5-94178-674-9. Текст : непосредственный.

19. Свиридова С. В. Разработка теоретического и научно-методического обеспечения стратегического инновационного развития промышленных предприятий : дис. ... д-ра экон. наук : 08.00.05 / С. В. Свиридова ; науч. конс. д-р экон. наук, проф. Ю. П. Анисимова ; Воронеж. гос. техн. ун-т. – Воронеж, 2016. – 420 с. – Библиогр.: с. 355–383. – Текст : непосредственный.

20. Петровский Э. А. Управление качеством производственных и технологических систем : Учебник / Петровский Э. А. – 2-е издание, переработанное и дополненное – Старый Оскол : ТНТ, 2026. – 351 с. - ISBN 978-5-94178-547-6. Текст : электронный // ЭБС ТНТ [сайт]. – URL: <https://www.tnt-ebook.ru/library/book/146> (дата обращения: 01.03.2026).

21. Томакова Р.А. Основы теории нейрокомпьютерных систем : учебное пособие / Р. А. Томакова ; Юго-Зап. гос. ун-т. - Курск : [б. и.], 2021. - 135 с. - ISBN 978-5-907555-36-5 : Б. ц. - Текст : непосредственный.

22. Томакова Р.А. Методологические основы научных исследований : учебное пособие / Р. А. Томакова, В. И. Томаков ; Юго-Зап. гос. ун-т. - Курск : ЮЗГУ, 2017. - 204 с. - Библиогр.: с. 199-203. - ISBN 978-5-7681-1210-3. - Текст : непосредственный.

23. Программные системы статистического анализа: обнаружение закономерностей в данных с использованием системы R и языка Python : учебное пособие / В. М. Волкова, М. А. Семенова, Е. С. Четвертакова, С. С. Вожов. - Новосибирск : Новосибирский государственный технический университет, 2017. - 74 с. : ил., табл. - URL:

<https://biblioclub.ru/index.php?page=book&id=576496> (дата обращения: 02.03.2022) . - Режим доступа: по подписке. - Библиогр.: с. 48. - ISBN 978-5-7782-3183-2 : Б. ц. - Текст : электронный.

24. Шелудько, В. М. Язык программирования высокого уровня Python: функции, структуры данных, дополнительные модули : учебное пособие / В. М. Шелудько ; Министерство науки и высшего образования РФ ; Федеральное государственное автономное образовательное учреждение высшего образования «Южный федеральный университет» ; Институт компьютерных технологий и информационной безопасности. - Ростов-на-Дону|Таганрог : Издательство Южного федерального университета, 2017. - 108 с. : ил. - URL: <http://biblioclub.ru/index.php?page=book&id=500060> (дата обращения: 24.04.2026) . - Режим доступа: по подписке. - Библиогр. в кн. - ISBN 978-5-9275-2648-2 : Б. ц. - Текст : электронный.

25. Демидова Л.А. Принятие решений в условиях неопределенности : монография / Л. А. Демидова. - 2-е изд., перераб. - Москва : Телеком, 2016. - 289 с. - ISBN 978-5-9912-0513-9 - Текст : непосредственный.

26. Яхьяева Г. Э. Основы теории нейронных сетей : учебное пособие / Г. Э. Яхьяева. – 2-е изд., испр. – Москва : Национальный Открытый Университет «ИНТУИТ», 2016. – 200 с. – (Основы информационных технологий). – URL: <http://biblioclub.ru/index.php?page=book&id=429110> (дата обращения: 01.06.2026). – ISBN 978-5-94774-818-5 : Б. ц. – Текст : электронный.

27. Келлехер, Д. Наука о данных: базовый курс : учебное пособие / Д. Келлехер, Б. Тирни ; науч. ред. З. Мамедьяров ; пер. с англ. М. Белоголовский. - Москва : Альпина Паблишер, 2020. - 224 с. - URL: <http://biblioclub.ru/index.php?page=book&id=598235> (дата обращения: 09.02.2026) . - Режим доступа: по подписке. - ISBN 978-5-9614-3170-4 : Б. ц. - Текст : электронный.

28. Кассим К.Д.А. Моделирование систем искусственного интеллекта в среде Matlab и Fuzzytech : учебное пособие / К. Д. А. Кассим, С. А. Филист, О. В. Шаталова. - Курск : Деловая полиграфия, 2016. - 186 с. - Библиогр.: с. 185. - ISBN 978-5-9908582-8-2. - Текст : непосредственный.

29. Д. С. Пономарев Нейросетевые методы прогнозирования в производственном секторе: MLP или RNN? // Информатика. Экономика. Управление / Informatics. Economics. Management. 2025. №2. URL: <https://cyberleninka.ru/article/n/neyrosetevye-metody-prognozirovaniya-v-proizvodstvennom-sektore-mlp-ili-rnn> (дата обращения: 08.06.2026).

30. Программирование, тестирование, проектирование, нейросети, технологии аппаратно-программных средств (практические задания и способы их решения) : учебник / С. В. Веретехина, К. С. Кармицкий, Д. Д. Лукашин [и др.]. - Москва : Директ-Медиа, 2022. - 144 с. - URL: <https://biblioclub.ru/index.php?page=book&id=694782> (дата обращения: 10.01.2023) . - Режим доступа: по подписке. - Библиогр. в кн. - ISBN 978-5-4499-3321-8 : Б. ц. - Текст : электронный.

31. Интеллектуальные системы и технологии : учебное пособие / С. П. Ющенко [и др.] ; Юго-Зап. гос. ун-т. - Курск : Университетская книга, 2018. - 226 с. : ил. - Библиогр.: с. 238-240 (32 назв.). - ISBN 978-5-907138-22-3 - Текст : непосредственный.

32. Сидоркина И.Г. Системы искусственного интеллекта : учебное пособие / И. Г. Сидоркина. - Москва : КНОРУС, 2016. - 246 с. : рис. - Библиогр.: с. 244-245. - ISBN 978-5-406-04876-4 . - Текст : непосредственный.

33. Пугачева Дарья Борисовна, Юдина Майя Викторовна Исследование программных решений для определения оптимального решения по заданным параметрам // Computational nanotechnology. 2024. №5. URL: <https://cyberleninka.ru/article/n/issledovanie-programmnyh-resheniy-dlya-opredeleniya-optimalnogo-resheniya-po-zadannym-parametram> (дата обращения: 08.06.2026).

34. Лютов А. Г. Управление качеством. Интегрированные информационно-аналитические системы : Учебник / Лютов А. Г., Огородов В. А., Чугунова О. И. 1– Старый Оскол : ТНТ, 2025. – 284 с. - ISBN 978-5-94178-868-2. Текст : непосредственный.

35. Лазарсон Э. В. Теория и методы решения многовариантных неформализованных задач выбора : Монография / Лазарсон Э. В. 1– Старый Оскол

: ТНТ, 2022. – 240 с. - ISBN 978-5-94178-042-6. - Текст : электронный // ЭБС ТНТ сайт. – URL: <https://www.tnt-ebook.ru/library/book/476> (дата обращения: 02.06.2026).

36. Исакова, А. И. Основы информационных технологий : учебное пособие / А. И. Исакова. - Томск : ТУСУР, 2016. - 206 с. : ил. - URL: <http://biblioclub.ru/index.php?page=book&id=480808> (дата обращения: 18.12.2025) . - Режим доступа: по подписке. - Библиогр.: с. 197-198. - Б. ц. - Текст : электронный.

37. Гунько, А. В. Программирование (в среде Windows) : учебное пособие / А. В. Гунько ; Новосибирский государственный технический университет. - Новосибирск : Новосибирский государственный технический университет, 2019. - 155 с. - URL: <https://biblioclub.ru/index.php?page=book&id=575417> (дата обращения: 03.05.2024) . - Режим доступа: по подписке. - Библиогр. в кн. - ISBN 978-5-7782-3890-9 : Б. ц. - Текст : электронный.

38. Черных Владимир Сергеевич, Жихарев Александр Геннадиевич, Федосеев Артемий Дмитриевич, Мартон Никита Андреевич СРАВНЕНИЕ ЭФФЕКТИВНОСТИ РАЗЛИЧНЫХ МЕТОДОВ ОБУЧЕНИЯ НЕЙРОННЫХ СЕТЕЙ // Научный результат. Информационные технологии. 2023. №1. URL: <https://cyberleninka.ru/article/n/sravnenie-effektivnosti-razlichnyh-metodov-obucheniya-neyronnyh-setey> (дата обращения: 08.06.2026).

39. Аверченков, В. И. Основы математического моделирования технических систем : учебное пособие / В. И. Аверченков, В. П. Федоров, М. Л. Хейфец. - 4-е изд., стер. - Москва : Флинта, 2021. - 271 с. - URL: <http://biblioclub.ru/index.php?page=book&id=93344> (дата обращения: 16.02.2026) . - Режим доступа: по подписке. - ISBN 978-5-9765-1278-8 : Б. ц. - Текст : электронный.

40. Чистякова Т.Б., Шашихина О.Е. Интеллектуальный программный комплекс моделирования процесса планирования многоассортиментных промышленных производств // Прикладная информатика. 2022. Т. 17. № 5. С. 41–50. DOI: 10.37791/2687-0649-2022-17-5-41-50 – Текст : электронный.

41. Речкалов А.В., Артюхов А.В., Куликов Г.Г. Логико-семантическое определение цифрового двойника производственного процесса. RTJ. 2023;11(1):70–80. - URL: <https://doi.org/10.32362/2500-316X-2023-11-1-70-80> (дата обращения: 16.02.2026) - Текст : электронный.

42. Виталий Романович Александров, Алексей Алексеевич Щеткин, Александр Сергеевич Бевз ГЕНЕРАТИВНЫЙ ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ В ПЛАНИРОВАНИИ ПРОИЗВОДСТВА // Измерение. Мониторинг. Управление. Контроль. 2025. №1 (51). URL: <https://cyberleninka.ru/article/n/generativnyy-iskusstvennyy-intellekt-v-planirovanii-proizvodstva> (дата обращения: 02.06.2026).

43. Гвоздинский Анатолий Николаевич, Серик Екатерина Эдуардовна Исследование и разработка интеллектуальных методов управления современным предприятием // Радиоэлектроника и информатика. 2017. №3. URL: <https://cyberleninka.ru/article/n/issledovanie-i-razrabotka-intellektualnyh-metodov-upravleniya-sovremennym-predpriyatiem> (дата обращения: 02.06.2026).

44. Рассомагин А. С. Оптимизация стратегии технического обслуживания и ремонтов с применением интеллектуальных методов // Инновации и инвестиции. 2019. №7. URL: <https://cyberleninka.ru/article/n/optimizatsiya-strategii-tehnicheskogo-obsluzhivaniya-i-remontov-s-primeneniem-intellektualnyh-metodov> (дата обращения: 04.06.2026).

45. Яковлева Дарья Дмитриевна Особенности системы планирования на основе интеллектуальных решений // ЭПП. 2024. №4. URL: <https://cyberleninka.ru/article/n/osobennosti-sistemy-planirovaniya-na-osnove-intellektualnyh-resheniy> (дата обращения: 04.06.2026).

46. ПЛАНИРОВАНИЕ ПРОИЗВОДСТВЕННЫХ РЕСУРСОВ НА ОСНОВЕ МЕТОДОВ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА / В.Р. Александров [и др.] // Автоматизация в промышленности .— 2024 .— №2 .— С. 36-39 .— Текст : непосредственный.

47. Иващенко А. В., Никифорова Т. В. ЦИФРОВИЗАЦИЯ ОРГАНИЗАЦИОННОЙ СТРУКТУРЫ УПРАВЛЕНИЯ ПРОИЗВОДСТВЕННЫМ ПРЕД-

ПРИЯТИЕМ // Известия Самарского научного центра РАН. 2021. №2. URL: <https://cyberleninka.ru/article/n/tsifrovizatsiya-organizatsionnoy-struktury-upravleniya-proizvodstvennym-predpriyatiem> (дата обращения: 07.06.2026).

48. Преображенский А. П., Линкина А. В. РЕШЕНИЕ ЗАДАЧИ ОПТИМИЗАЦИИ РЕСУРСОЭФФЕКТИВНОСТИ ПРЕДПРИЯТИЯ С ПРИМЕНЕНИЕМ ТЕХНОЛОГИЙ БЕРЕЖЛИВОГО ПРОИЗВОДСТВА // Вестник ВГУИТ. 2021. №4 (90). URL: <https://cyberleninka.ru/article/n/reshenie-zadachi-optimizatsii-resursoeffektivnosti-predpriyatiya-s-primeneniem-tehnologiy-berezhlivogo-proizvodstva> (дата обращения: 05.06.2026).

49. Шевцов Е.С., Шамин Р.В. Логическая интеграция информационных систем на основе экспертных систем. Russian Technological Journal. 2025;13(2):27-35. <https://doi.org/10.32362/2500-316X-2025-13-2-27-35>. EDN: SRKXBR – Текст : электронный.

50. Бабич, Татьяна Николаевна. Оперативно-производственное планирование : учебное пособие / Т. Н. Бабич, Ю. В. Вертакова. - Москва : РИОР : ИНФРА-М, 2017. - 260 с. - (Высшее образование). - ISBN 978-5-369-01616-9 (РИОР). - ISBN 978-5-16-012455-1 (ИНФРА-М) - Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.11.

Сведения о ВКРМ

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ МАГИСТРАТУРЫ

«Интеллектуальная система оптимизации производственной
деятельности на основе АИС»

Руководитель ВКРМ
д.т.н, профессор
Томакова Римма Александровна

Автор ВКРМ
студент группы ПО-41м
Каракчиев Даниил Александрович

ВКРМ 2068443.09.04.04.26.010			
		Ф.И.О.	Подпись
Автор работы	Каракчиев Д.А.		
Руководитель	Томакова Р.А.		
Нормировщик	Какурина А.В.		
		Дата	Лист
		1	11
		Выпускная квалификационная работа магистра	
		ЮЗГУ ПО-41м	

Рисунок А.1 – Сведения о ВКРМ

Актуальность темы

Современные производственные предприятия с дискретным характером выпуска сталкиваются с необходимостью оперативного составления расписаний, учитывающих загрузку и текущее состояние оборудования. Классические методы целочисленного линейного программирования требуют значительных вычислительных ресурсов и времени, что ограничивает их применение в реальном масштабе времени. Развитие методов глубокого обучения открывает возможность построения быстрых нейросетевых диспетчеров, способных адаптироваться к изменяющейся производственной обстановке без полного пересчёта плана.

Актуальной является задача создания интеллектуальной системы планирования, объединяющей эвристические алгоритмы и нейросетевые модели для повышения эффективности использования оборудования и сокращения времени опоздания заказов.

ВКРМ 2068443.09.04.04.26.010			
Автор работы	Фамилия И. Ф.	Подпись	Дата
Руководитель	Томашова Д.А.		
Нормировщик	Томашова Р.А.		
	Какурин А.В.		
Актуальность темы			Апр. 11
Выполнен и квалификационная работа магистра			ЮЗГУ ПО-41м

Рисунок А.2 – Актуальность темы

Цели и задачи разработки

Цель работы - разработка интеллектуальной системы оптимизации производственной деятельности на основе АИС

Для достижения поставленной цели необходимо решить следующие задачи:

- Разработать модуль управления заказами и производственными ресурсами с учётом технологических типов операций и связей «ресурс–тип».
- Разработать подсистему построения производственных расписаний, использующую эвристический алгоритм и нейросетевые диспетчеры.
- Реализовать модуль обучения нейросетевых моделей на синтетических данных с сохранением весов и метрик обучения.
- Провести сравнительный анализ методов планирования по критериям опоздания, периоду обработки и равномерности загрузки ресурсов.

ВКРМ 2068443.09.04.04.26.010			
Автор работы	Фамилия И. Ф.	Подпись	Дата
Руководитель	Томашова Д.А.		
Нормировщик	Томашова Р.А.		
	Капустина А.В.		
Цели и задачи разработки			Дата
Выполнена и одобрена работа магистра			ЮЗГУ ПО-41м

Рисунок А.3 – Цель и задачи разработки



Рисунок А.4 – Концептуальная модель данных

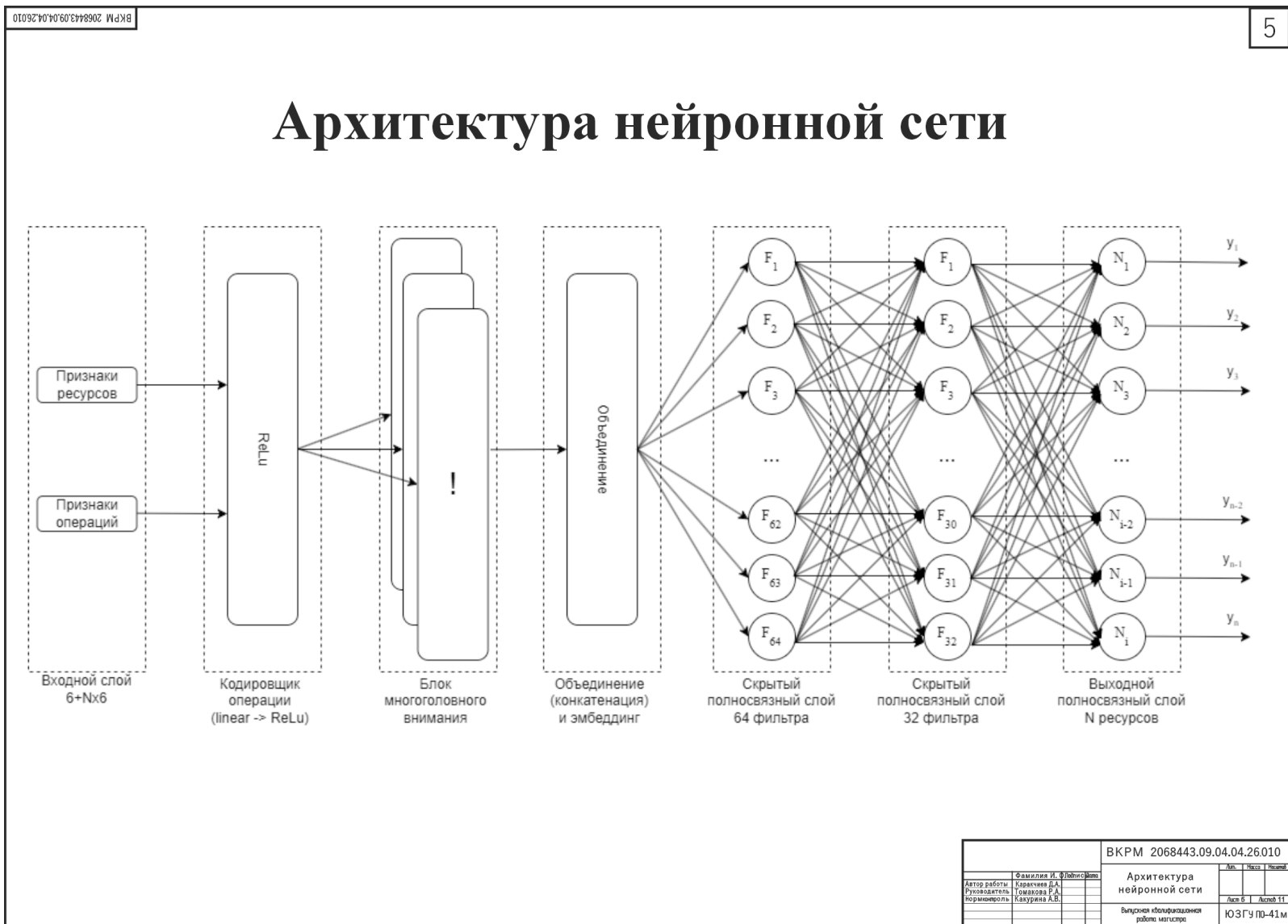


Рисунок А.5 – Архитектура нейронной сети

Функция активации ReLU

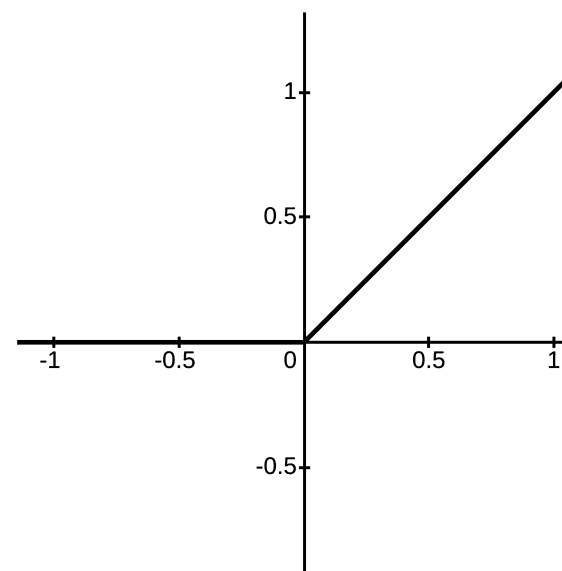
ReLU (rectified linear unit)

$$f(s) = \max(0, s)$$

$$f'(s) = \begin{cases} 1, & s > 0 \\ \text{rand}(0.01, 0.05), & s \leq 0 \end{cases}$$

Достоинства

- лишена ресурсоемких операций
- отсекает ненужные детали
- отсутствует разрастание/затухание градиента
- быстрое обучение



ВКРМ 2068443.09.04.04.26.010			
Функция активации ReLU			
Автор работы	Федина И. Ф.	Дата	Итого
Руководитель	Томашова Р. А.	Дата	Итого
Нормировщик	Какурина А. В.	Дата	Итого
Выполнена квалификационная работа магистра			
ЮЗГУ ПО-41м			

Рисунок А.6 – Функция активации ReLU



Рисунок А.7 – Диаграмма компонентов клиентского приложения

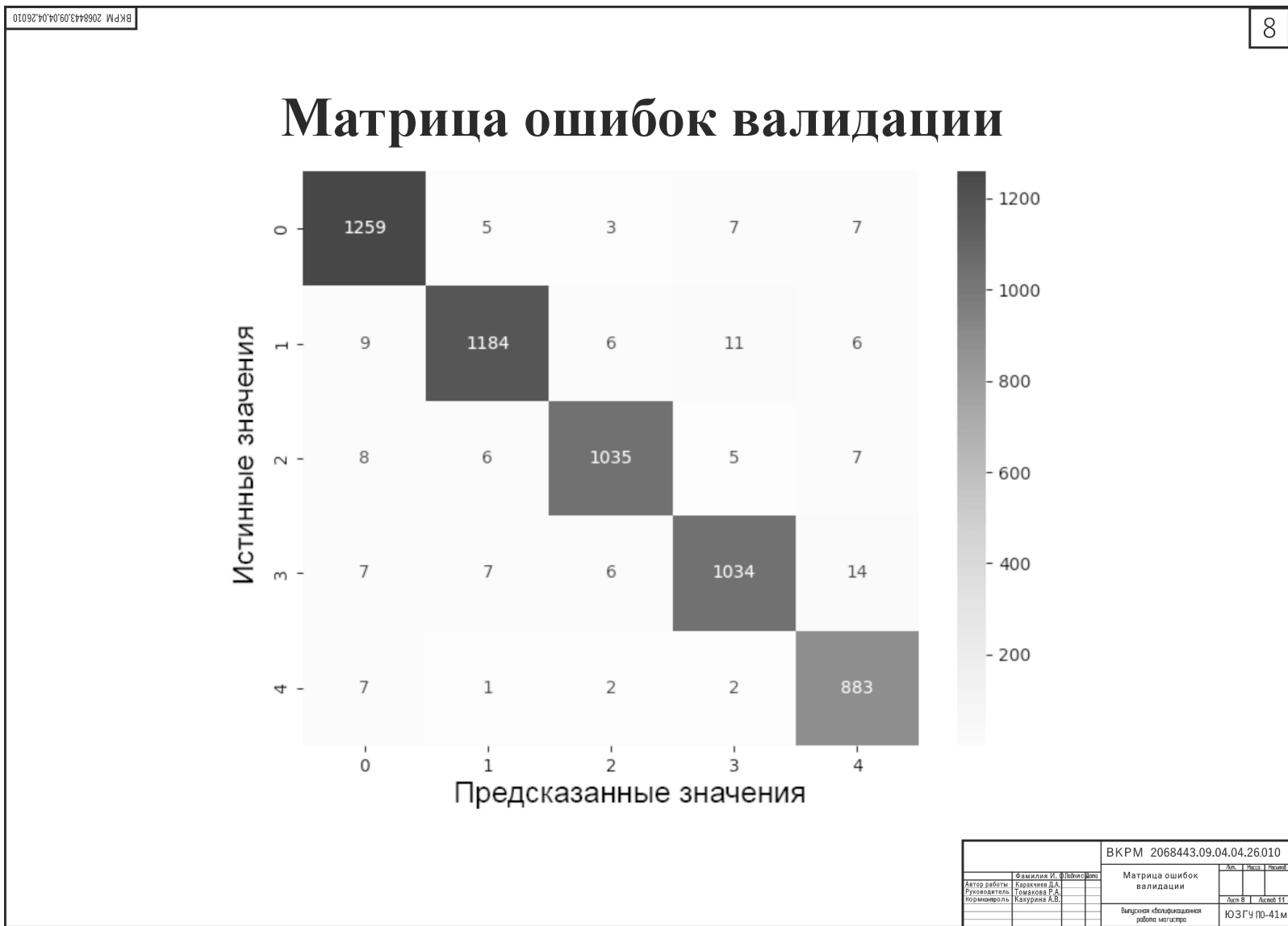
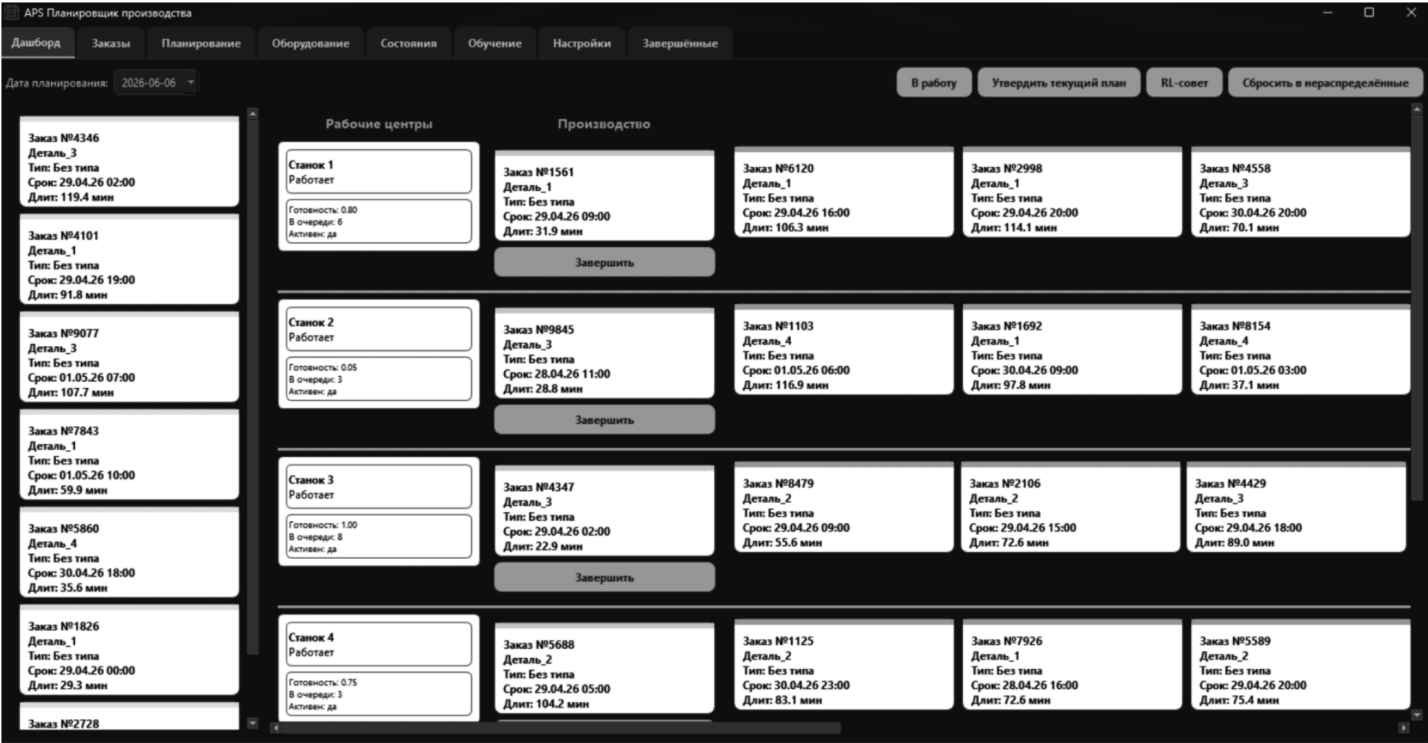


Рисунок А.8 – Матрица ошибок валидации

Интерфейс интеллектуальной системы



		ВКРМ 2068443.09.04.04.26.010	
		Интерфейс интеллектуальной системы	
Автор работы	Фамилия И. О. Инициалы	Акт 9	Акт 11
Руководитель	Томасова Г.А.		
Нормировщик	Какурина А.В.		
		Выпуск квалификационной работы магистров	
		ЮЗГУ ПО-41м	

Рисунок А.9 – Интерфейс интеллектуальной системы

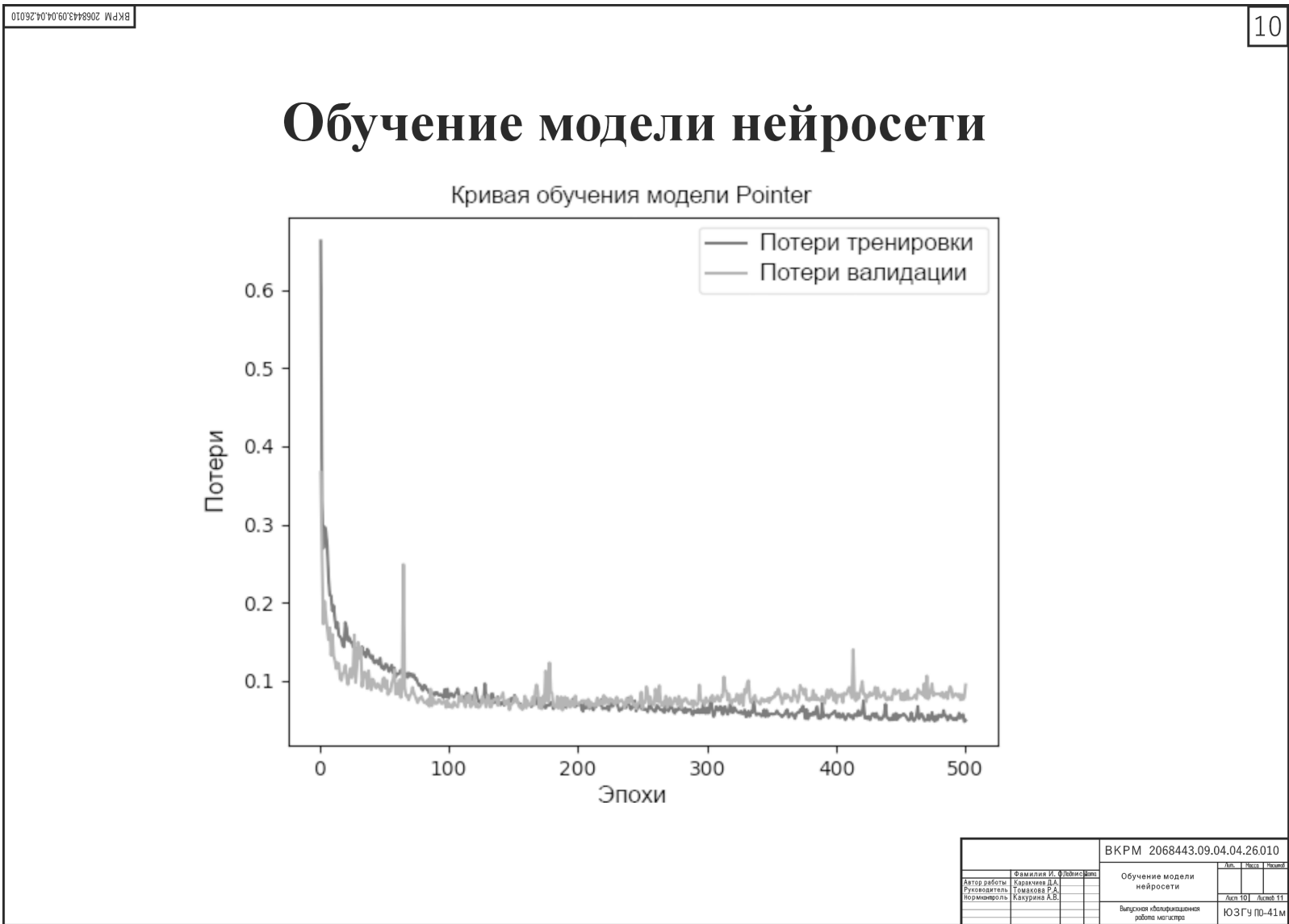


Рисунок А.10 – Обучение модели нейросети

Заключение

Основные результаты, полученные в ходе работы:

- Разработан модуль управления заказами и производственными ресурсами с учётом технологических типов операций и связей «ресурс–тип».
- Разработана подсистема построения производственных расписаний, использующая эвристический алгоритм и нейросетевые диспетчеры.
- Реализован модуль обучения нейросетевых моделей на синтетических данных с сохранением весов и метрик обучения.
- Проведен сравнительный анализ методов планирования по критериям опоздания, периоду обработки и равномерности загрузки ресурсов.

Разработанная интеллектуальная система может использоваться для автоматизации оперативного планирования и диспетчеризации производственными предприятиями с гибкой структурой выпуска, нуждающимися в доступном и настраиваемом решении для построения выполнимых расписаний с учётом текущего состояния оборудования.

Перспективами дальнейшего развития проекта являются расширение системы для поддержки динамических событий в реальном времени (поломки оборудования, срочные заказы), а также интеграция с существующими MES/ERP-системами.

ВКРМ 2068443.09.04.04.26.010			
Автор работы	Фамилия И. Ф.	Имя	Отчество
Рисователь	Томашов Р.Д.		
Полномочный	Калугина А.В.		
Заключение		Лист 11	Лист 11
Выпускная квалификационная работа магистра		ЮЗГУ ПО-41М	

Рисунок А.11 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

core.py

```
1 # core.py
2 import os, json, threading, pickle
3 import numpy as np
4 from datetime import datetime, timedelta
5 from datamodels import Order, Operation, Resource
6 from predictor import DurationPredictor
7 from optimizer_milp import build_milp_schedule
8 from aps_core import HybridAPS
9 from database import (
10     init_db, save_orders_to_db, load_orders_from_db,
11     update_schedule, get_fixed_operations, save_resources, load_resources,
12     save_downtime, load_downtimes, delete_resource, delete_downtime,
13     get_state_types, add_state_type, delete_state_type,
14     get_planned_operations, apply_accepted_changes,
15     get_operation_types, add_operation_type, delete_operation_type,
16     set_resource_operation_types, get_resource_operation_types,
17     add_task_to_queue, update_task_status, get_tasks_for_resource,
18     delete_task_from_queue,
19     delete_order_from_db, delete_operation_from_db,
20     get_setting, set_setting, get_all_settings,
21     log_resource_state, log_operation_state, get_execution_data,
22     set_order_status
23 )
24 from PySide6.QtCore import QObject, Signal
25 from training import ResourceOpPointerNet, SimpleDispatcher
26 import torch
27
28 class BCAgent:
29     def __init__(self, model_path):
30         with open(model_path, 'rb') as f:
31             self.model = pickle.load(f)
32     def select_action(self, state, op_feat=None, mask=None):
33         probs = self.model.predict_proba([state])[0]
34         probs[~np.array(mask)] = -1
35         return np.argmax(probs)
36
37 class PointerNetAgent:
38     def __init__(self, model_path):
39         self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
40         # Размеры, использованные при обучении
41         self.d_res = 7
42         self.d_op = 6
43         self.model = ResourceOpPointerNet(d_res=self.d_res, d_op=self.d_op).to(self.device)
44         self.model.load_state_dict(torch.load(model_path, map_location=self.device))
45         self.model.eval()
```

```

45 def select_action(self, res_feat, op_feat, mask):
46     """
47     res_feat: (R, d_res)
48     op_feat: (O, d_op)
49     mask: (O, R) bool
50     Возвращает индекс выбранной операции (0..O-1).
51     """
52     with torch.no_grad():
53         r = torch.tensor(res_feat, dtype=torch.float32).unsqueeze(0).to(
54             self.device)
55         o = torch.tensor(op_feat, dtype=torch.float32).unsqueeze(0).to(
56             self.device)
57         m = torch.tensor(mask, dtype=torch.bool).unsqueeze(0).to(self.
58             device)
59         probs = self.model(r, o, m) # (1, O, R)
60         # Выбираем операцию с наибольшей вероятностью хотя бы на одном
61         # ресурсе
62         values, _ = probs.max(dim=-1) # (1, O)
63         action = values.argmax(dim=-1).item()
64         return action, None
65
66 class SimpleAgent:
67     def __init__(self, model_path):
68         self.device = torch.device("cpu")
69         state = torch.load(model_path, map_location=self.device)
70
71         # Автоматически находим первый и последний Linear слои среди ключей
72         # state_dict
73         linear_weights = [k for k in state.keys() if k.endswith('.weight')
74             and 'net.' in k]
75         if not linear_weights:
76             raise RuntimeError("В загруженной модели не найдено полносвязных
77             слоёв (net.*.weight)")
78
79         first_weight_key = sorted(linear_weights)[0] # например, 'net.0.
80             weight'
81         last_weight_key = sorted(linear_weights)[-1] # например, 'net.6.
82             weight'
83
84         input_dim = state[first_weight_key].shape[1] # количество входных
85             признаков
86         output_dim = state[last_weight_key].shape[0] # количество классов (
87             ресурсов)
88
89         # Создаём модель с теми же размерами
90         self.model = SimpleDispatcher(input_dim, output_dim).to(self.device)
91         self.model.load_state_dict(state)
92         self.model.eval()
93
94     def select_action(self, X):
95         """X – плоский вектор признаков (36,) □ возвращает индекс ресурса."""
96         ""
97
98         with torch.no_grad():

```

```

86         x = torch.tensor(X, dtype=torch.float32).unsqueeze(0).to(self.
            device)
87         logits = self.model(x) # (1, output_dim)
88         return logits.argmax(dim=1).item()
89
90 class APSCore(QObject):
91     dashboard_changed = Signal()
92     orders_changed = Signal()
93     message_signal = Signal(str, str)
94
95     def __init__(self):
96         super().__init__()
97         init_db()
98         self.resources = load_resources()
99         if not self.resources:
100             self.resources = [Resource('R1', 'Станок 1'), Resource('R2', '
                Станок 2'), Resource('R3', 'Станок 3')]
101             save_resources(self.resources)
102         self.resource_map = {r.id: r for r in self.resources}
103         self.predictor = DurationPredictor()
104         self.aps_engine = HybridAPS(self.resources, predictor=self.predictor)
105         self.rl_agent = None
106         self.use_rl = False
107         self.orders = []
108         self.current_schedule = {}
109         self.approved_schedule = {}
110         self.downtimes = load_downtimes()
111         self.state_types = get_state_types()
112         self.selected_date = datetime.today().replace(hour=0, minute=0,
            second=0, microsecond=0)
113         self.loaded_model_name = "" # <-- новое поле
114
115         self.settings = get_all_settings()
116         if not self.settings:
117             self.settings = {
118                 'bc_model_path': 'behavior_cloning.pkl',
119                 'dqn_episodes': '500', 'dqn_lr': '0.0003',
120                 'erp_file': 'erp_orders.json', 'mes_file': 'mes_events.json',
121                 'gantt_slot_minutes': '120'
122             }
123             for k, v in self.settings.items():
124                 set_setting(k, v)
125
126         # Попытка загрузить RL-модель, если путь сохранён в настройках
127         self.load_rl_model_if_available()
128
129         saved_orders = load_orders_from_db(self.resources)
130         if saved_orders:
131             self.orders = saved_orders
132             # Приведение к "1 заказ – 1 операция" для уже существующих в БД
            записей
133             for order in self.orders:
134                 if len(order.ops) > 1:
135                     # Удаляем лишние операции из БД

```

```

136         for op in order.ops[1:]:
137             delete_operation_from_db(op.id)
138         order.ops = order.ops[:1]
139         # Пересохраняем заказ с одной операцией
140         save_orders_to_db([order])
141         # Удаляем из очереди все операции, которых нет среди активных
            заказов
142         valid_op_ids = set()
143         for order in self.orders:
144             for op in order.ops:
145                 valid_op_ids.add(op.id)
146         import sqlite3
147         conn = sqlite3.connect("aps.db")
148         cur = conn.cursor()
149         cur.execute("SELECT DISTINCT operation_id FROM task_queue")
150         for row in cur.fetchall():
151             if row[0] not in valid_op_ids:
152                 cur.execute("DELETE FROM task_queue WHERE operation_id=?",
                    , (row[0],))
153         conn.commit()
154         conn.close()
155         self.aps_engine.load_orders(self.orders)
156         self.load_schedules_from_db()
157         if not self.current_schedule and not self.approved_schedule and self.
            orders:
158             self.run_milp()
159
160     def get_loaded_model_info(self):
161         return self.loaded_model_name if self.loaded_model_name else "Нет"
162
163     def load_settings(self):
164         self.settings = get_all_settings()
165
166     def save_settings(self, settings_dict):
167         for k, v in settings_dict.items():
168             set_setting(k, str(v))
169         self.settings = get_all_settings()
170
171     def _build_pointer_input(self, env, ready_ops):
172         type_map = get_operation_types() # id □ имя
173         type_list = ['фрезеровка', 'сборка', 'сварка'] # порядок,
            использованный в датасете
174         type_to_idx = {name: i for i, name in enumerate(type_list)}
175         res_types = get_resource_operation_types() # rid □ [type_id]
176
177         R = len(env.resource_ids)
178         d_res = 7
179         res_feat = np.zeros((R, d_res), dtype=np.float32)
180         for i, rid in enumerate(env.resource_ids):
181             res = self.resource_map[rid]
182             status_active = 1.0 if res.status == 'Работает' else 0.0
183             load_norm = res.load_minutes / 240.0
184             work_hours = float(res.work_hours)
185             type_onehot = np.zeros(3)

```

```

186         for tid in res_types.get(rid, []):
187             tname = type_map.get(tid, '')
188             if tname in type_to_idx:
189                 type_onehot[type_to_idx[tname]] = 1.0
190         repair = float(res.repair)
191         res_feat[i, 0] = status_active
192         res_feat[i, 1] = load_norm
193         res_feat[i, 2] = work_hours
194         res_feat[i, 3:6] = type_onehot
195         res_feat[i, 6] = repair
196
197     O = len(ready_ops)
198     d_op = 6
199     op_feat = np.zeros((O, d_op), dtype=np.float32)
200     mask = np.zeros((O, R), dtype=bool)
201     for idx, op_info in enumerate(ready_ops):
202         op_id = op_info['op_id']
203         op = self._get_operation_by_id(op_id)
204         type_onehot = np.zeros(3)
205         if op and op.type_id:
206             tname = type_map.get(op.type_id, '')
207             if tname in type_to_idx:
208                 type_onehot[type_to_idx[tname]] = 1.0
209         op_feat[idx, :3] = type_onehot
210         op_feat[idx, 3] = op.norm_duration / 120.0 if op else 0.5
211         op_feat[idx, 4] = op.urgency if op else 1.0
212         op_feat[idx, 5] = (op.slack_hours if op else 0) / 48.0
213
214         # Допустимый ресурс из ready_ops
215         allowed_rid = op_info['resource_id']
216         if allowed_rid in env.resource_ids:
217             mask[idx, env.resource_ids.index(allowed_rid)] = True
218         else:
219             mask[idx, :] = True
220
221     return res_feat, op_feat, mask
222
223 def load_schedules_from_db(self):
224     fixed = get_fixed_operations()
225     self.approved_schedule = {op_id: {'start': v['start'], 'end': v['end'],
226                                     'resource_id': v['resource_id']}
227                             for op_id, v in fixed.items()}
228     planned = get_planned_operations()
229     self.current_schedule = {op_id: {'start': v['start'], 'end': v['end'],
230                                     'resource_id': v['resource_id']}
231                             for op_id, v in planned.items() if v['start']
232                             is not None}
233
234 def create_order(self, due_date, priority, name, order_number, operations):
235     if order_number:
236         order_id = f"Заказ_{order_number}"
237     else:
238         order_id = f"Заказ_{np.random.randint(1, 99)}"

```

```

236 order = Order(order_id, due_date, priority, name=name)
237 for i, op_data in enumerate(operations, start=1):
238     op = Operation(
239         f"{order_id}_on{i}", order_id, op_data.get('item', ''),
240         i, op_data.get('duration', 60),
241         type_id=op_data.get('type_id')
242     )
243     if 'quantity' in op_data:
244         op.quantity = op_data['quantity']
245     if 'predecessors' in op_data:
246         op.predecessors = op_data['predecessors']
247     order.ops.append(op)
248 self.orders.append(order)
249 save_orders_to_db([order])
250 self.aps_engine.load_orders([order])
251 # self.run_incremental_milp() - обучение милп
252 self.orders_changed.emit()
253 self.dashboard_changed.emit()
254 return order
255
256 def delete_order(self, order_id):
257     order = next((o for o in self.orders if o.id == order_id), None)
258     if not order:
259         return
260     for op in order.ops:
261         self._remove_from_schedules(op.id)
262         delete_task_from_queue(op.id)
263     self.orders.remove(order)
264     delete_order_from_db(order_id)
265     self.orders_changed.emit()
266     self.dashboard_changed.emit()
267
268 def delete_completed_task(self, op_id):
269     self._remove_from_schedules(op_id)
270     delete_task_from_queue(op_id)
271     for order in self.orders:
272         for op in order.ops:
273             if op.id == op_id:
274                 order.ops.remove(op)
275                 delete_operation_from_db(op_id)
276                 self.dashboard_changed.emit()
277             return
278
279 def generate_random_orders(self, count=10):
280     # Защита от некорректного count
281     if not isinstance(count, int) or count <= 0:
282         count = 10
283
284     types = list(get_operation_types().keys())
285     if not types:
286         self.message_signal.emit("warning", "Сначала добавьте типы
287                                     операций на вкладке «Оборудование □ Типы».")
288     return

```

```

289 base = datetime(2026, 4, 27, 8, 0, 0)
290 generated = []
291 for _ in range(count):
292     due = base + timedelta(hours=np.random.randint(10, 100))
293     order = Order(
294         f"Заказ_{np.random.randint(1000, 9999)}",
295         due,
296         round(np.random.uniform(0.5, 2.0), 2),
297         name="Случайный"
298     )
299     # Единственная операция
300     op_id = f"{order.id}_on1"
301     type_id = np.random.choice(types)
302     op = Operation(
303         op_id, order.id, f"Деталь_{np.random.randint(1, 5)}", 1,
304         round(np.random.uniform(20, 120), 1)
305     )
306     op.type_id = type_id
307     op.quantity = int(np.random.randint(1, 10))
308     order.ops.append(op)
309     generated.append(order)
310
311 self.orders.extend(generated)
312 save_orders_to_db(generated)
313 for o in generated:
314     self.aps_engine.load_orders([o])
315
316 # self.run_incremental_milp() - если нужно закинуть обучение после
    генерации
317 self.orders_changed.emit()
318 self.dashboard_changed.emit()
319 self.message_signal.emit("info", f"Создано {count} случайных заказов.
    Перейдите на вкладку «Дашборд».")
320
321 def load_erp_orders(self, file_path):
322     with open(file_path, 'r', encoding='utf-8') as f:
323         erp_data = json.load(f)
324     loaded = []
325     for od in erp_data:
326         order = Order(...)
327         # Берём только первую операцию
328         op_d = od['operations'][0] if od.get('operations') else {}
329         if op_d:
330             op = Operation(op_d['id'], order.id, op_d['item'], 1, op_d['
                norm_duration'])
331             op.predecessors = [] # больше нет зависимостей
332             op.type_id = op_d.get('type_id', '')
333             op.quantity = int(op_d.get('quantity', 1))
334             order.ops.append(op)
335         loaded.append(order)
336     self.orders.extend(loaded)
337     save_orders_to_db(loaded)
338     for o in loaded:
339         self.aps_engine.load_orders([o])

```

```

340     self.run_incremental_milp()
341     self.orders_changed.emit()
342     self.dashboard_changed.emit()
343
344     def _build_heuristic_schedule(self, only_unscheduled=True):
345         """
346         Быстрый эвристический планировщик: EDD + Least Workload.
347         Возвращает словарь current_schedule.
348         """
349         from datetime import datetime
350         work_until = {r.id: datetime.now() for r in self.resources}
351         schedule = {}
352         ops = []
353         for order in self.orders:
354             for op in order.ops:
355                 if only_unscheduled and (op.id in self.current_schedule or op
356                                         .id in self.approved_schedule):
357                     continue
358                 ops.append((order, op))
359
360         # Сортируем по дате сдачи (Earliest Due Date)
361         ops.sort(key=lambda x: x[0].due_date)
362
363         # Если у ресурса не указаны типы, он универсален
364         if not hasattr(self, 'cached_res_op_types'):
365             self.cached_res_op_types = get_resource_operation_types()
366
367         for order, op in ops:
368             # Допустимые ресурсы: если у операции есть тип, берём те, что с
369             # ним связаны
370             if op.type_id:
371                 allowed_rids = self.cached_res_op_types.get(op.type_id, [])
372             else:
373                 allowed_rids = list(self.resource_map.keys())
374             if not allowed_rids:
375                 allowed_rids = list(self.resource_map.keys())
376
377             # Выбираем ресурс, который освободится раньше всех (Least
378             # Workload)
379             best_rid = min(allowed_rids, key=lambda rid: work_until.get(rid,
380                                                                     datetime.max))
381             start = max(work_until[best_rid], datetime.now())
382             dur = timedelta(minutes=op.norm_duration)
383             end = start + dur
384             schedule[op.id] = {'resource_id': best_rid, 'start': start, 'end':
385                               : end}
386             work_until[best_rid] = end
387
388         return schedule
389
390     def _try_start_next_on_resource(self, resource_id):
391         res = self.resource_map.get(resource_id)
392         if res and (getattr(res, 'repair', 0) or getattr(res, 'reliability',
393                                                         1.0) <= 0):

```



```

388         return
389     tasks = get_tasks_for_resource(resource_id)
390     if any(t['status'] == 'active' for t in tasks):
391         return
392     pending = [t for t in tasks if t['status'] == 'pending']
393     if pending:
394         self.move_task_to_production(pending[0]['operation_id'],
395                                     resource_id)
396
397     def start_all_queues(self):
398         """Запускает первую ожидающую задачу на каждом ресурсе, если он
399         свободен и исправен."""
400         for res in self.resources:
401             if getattr(res, 'repair', 0) or getattr(res, 'reliability', 1.0)
402                 <= 0:
403                 continue
404             self._try_start_next_on_resource(res.id)
405             self.dashboard_changed.emit()
406             self.message_signal.emit("info", "Очереди запущены.")
407
408     def run_milp(self):
409         """Быстрое эвристическое планирование (замена точного MILP)."""
410         if not self.orders:
411             return
412         self.current_schedule = self._build_heuristic_schedule(
413             only_unscheduled=True)
414         self._sync_schedule_to_task_queue()
415         self.dashboard_changed.emit()
416         self.message_signal.emit("info", "Эвристический план построен.")
417
418     def reset_all_queues(self):
419         """Перемещает все задачи из очередей и планов в нераспределённые."""
420         # Удаляем из очереди и расписаний все операции
421         for res in self.resources:
422             tasks = get_tasks_for_resource(res.id)
423             for t in tasks:
424                 self._remove_from_schedules(t['operation_id'])
425                 delete_task_from_queue(t['operation_id'])
426         # Очищаем текущий план
427         self.current_schedule.clear()
428         # Обновляем дашборд
429         self.dashboard_changed.emit()
430         self.message_signal.emit("info", "Все задачи возвращены в
431             нераспределённые.")
432
433     def _build_state_for_agent(self, env, ready_ops):
434         state = env._get_state()
435         busy = np.array([1.0 if env.resource_remaining[rid] > 0 else 0.0 for
436                         rid in env.resource_ids])
437         ext_state = np.concatenate([state, busy])
438         op_feat = np.zeros((env.max_actions, 5))
439         mask = np.zeros(env.max_actions, dtype=bool)
440         for i, op_info in enumerate(ready_ops):
441             if i >= env.max_actions:

```

```

436         break
437     op_id = op_info['op_id']
438     rid = op_info['resource_id']
439     due_min = 0;
440     rem_time = 0;
441     weight = 0;
442     progress = 0
443     for order in env.orders:
444         for op in order.ops:
445             if op.id == op_id:
446                 due_min = (order.due_date - env.start_datetime).
447                     total_seconds() / 60.0
448                 rem_time = env.op_remaining[op.id]
449                 weight = order.priority_weight
450                 completed = sum(1 for o in order.ops if env.op_status
451                     [o.id] == 'completed')
452                 progress = completed / len(order.ops)
453                 break
454     slack = (due_min - env.current_time) / 1440.0
455     rem_time_norm = rem_time / 1440.0
456     weight_norm = weight / 10.0
457     res_free = 0.0 if env.resource_remaining[rid] > 0 else 1.0
458     op_feat[i] = [slack, rem_time_norm, weight_norm, progress,
459         res_free]
460     mask[i] = True
461     return ext_state, op_feat, mask
462
463 def _sync_schedule_to_task_queue(self):
464     for op_id, info in self.current_schedule.items():
465         if info['resource_id'] not in self.resource_map:
466             continue
467         delete_task_from_queue(op_id)
468         add_task_to_queue(info['resource_id'], op_id)
469     # Больше не активируем задачи автоматически – они остаются в статусе
470     'pending'
471
472 def run_fixed_milp(self):
473     self.aps_engine.run_milp_with_fixed()
474     self.current_schedule = self.aps_engine.current_schedule
475     self._sync_schedule_to_task_queue()
476     self.dashboard_changed.emit()
477
478 def run_incremental_milp(self):
479     self.aps_engine.run_milp_incremental()
480     self.current_schedule = self.aps_engine.current_schedule
481     self._sync_schedule_to_task_queue()
482     self.dashboard_changed.emit()
483
484 def approve_plan(self):
485     if not self.current_schedule:
486         self.message_signal.emit("info", "Нет рабочего плана для
487             утверждения.")
488     return
489     for op_id, info in self.current_schedule.items():

```

```

485         update_schedule(op_id, info['resource_id'], info['start'], info['
            end'], fixed=1, status='fixed')
486         self.approved_schedule[op_id] = info
487         self.current_schedule.clear()
488         self.dashboard_changed.emit()
489         self.message_signal.emit("info", "План утверждён.")
490
491     def move_task_to_queue(self, op_id, resource_id):
492         # Удаляем операцию из всех планов и других очередей
493         self._remove_from_schedules(op_id)
494         delete_task_from_queue(op_id)
495         # Добавляем в очередь выбранного ресурса
496         add_task_to_queue(resource_id, op_id)
497         log_operation_state(op_id, 'pending', resource_id)
498         # Пытаемся сразу запустить, если ресурс свободен
499         self._try_start_next_on_resource(resource_id)
500         # Обновляем интерфейс
501         self.dashboard_changed.emit()
502         self.message_signal.emit("info", f"Заказ {op_id} помещён в очередь {
            resource_id}")
503
504     def move_task_to_production(self, op_id, resource_id):
505         # Делаем операцию активной на указанном ресурсе
506         update_task_status(op_id, 'active')
507         log_operation_state(op_id, 'active', resource_id)
508         # Оповещаем GUI
509         self.dashboard_changed.emit()
510         self.message_signal.emit("info", f"Заказ {op_id} запущен в
            производство на {resource_id}")
511
512     def complete_task(self, op_id):
513         # Ищем ресурс, на котором операция сейчас (active/pending)
514         found_resource = None
515         for res in self.resources:
516             tasks = get_tasks_for_resource(res.id)
517             if any(t['operation_id'] == op_id for t in tasks):
518                 found_resource = res.id
519                 break
520
521         if not found_resource:
522             self.dashboard_changed.emit()
523             return
524
525         # Завершаем операцию
526         update_task_status(op_id, 'completed')
527         log_operation_state(op_id, 'completed', resource_id=found_resource)
528         self._remove_from_schedules(op_id)
529         delete_task_from_queue(op_id)
530
531         # Находим заказ (теперь операция одна на заказ)
532         order = self._get_order_by_op(op_id)
533         if order:
534             # Помечаем заказ завершённым в БД
535             set_order_status(order.id, 'completed')

```

```

536         # Убираем из активного списка (чтобы не отображался в backlog и
           # на дашборде)
537         self.orders = [o for o in self.orders if o.id != order.id]
538         # Оповещаем GUI
539         self.orders_changed.emit()
540         self.dashboard_changed.emit()
541     else:
542         self.dashboard_changed.emit()
543     return
544
545     # Пробуем запустить следующую ожидающую задачу на освободившемся
       # ресурсе
546     self._try_start_next_on_resource(found_resource)
547
548 def plan_with_rl(self):
549     if not self.rl_agent:
550         self.message_signal.emit("error", "Модель не загружена.")
551     return
552
553     # Собираем неразмещённые операции, сортируем по EDD
554     unscheduled = []
555     for order in self.orders:
556         for op in order.ops:
557             if op.id not in self.current_schedule and op.id not in self.
               approved_schedule:
558                 unscheduled.append((order, op))
559     if not unscheduled:
560         return
561     unscheduled.sort(key=lambda x: x[0].due_date)
562
563     # Кэш типов
564     if not hasattr(self, 'cached_res_op_types'):
565         self.cached_res_op_types = get_resource_operation_types()
566     type_map = get_operation_types()
567     types_list = ['фрезеровка', 'сборка', 'сварка']
568
569     # Подготавливаем базовые признаки ресурсов
570     R = len(self.resources)
571     # Для модели мы можем подкорректировать repair и reliability, чтобы
       # не выходить за рамки обучения
572     model_repair = np.zeros(R, dtype=np.float32)
573     model_reliability = np.zeros(R, dtype=np.float32)
574     for i, res in enumerate(self.resources):
575         rel = res.reliability
576         rep = res.repair
577         # Приводим к допустимому диапазону (как в обучающей выборке)
578         if rel < 0.25:
579             rep = 1 # низкая готовность интерпретируется как ремонт
580             rel = max(rel, 0.2) # минимальная надёжность 0.2
581         model_repair[i] = rep
582         model_reliability[i] = rel
583
584     work_until = {r.id: self.selected_date for r in self.resources}
585     schedule = {}

```

```

586
587     for order, op in unscheduled:
588         # Признаки операции (6 чисел)
589         type_onehot = np.zeros(3)
590         if op.type_id and op.type_id in type_map:
591             tname = type_map[op.type_id]
592             if tname in types_list:
593                 type_onehot[types_list.index(tname)] = 1.0
594         op_feat = np.array([
595             *type_onehot,
596             op.norm_duration / 120.0,
597             order.priority_weight,
598             (op.slack_hours if hasattr(op, 'slack_hours') else 0) / 48.0
599         ])
600
601         # Допустимые ресурсы по типам
602         allowed = self.cached_res_op_types.get(op.type_id, []) if op.
603             type_id else list(self.resource_map.keys())
604         if not allowed:
605             allowed = list(self.resource_map.keys())
606
607         # Строим полный вектор признаков (6 + R*6)
608         full = op_feat.copy()
609         for i, res in enumerate(self.resources):
610             # Текущая загрузка (в минутах работы, нормированная)
611             cur_load = (work_until[res.id] - self.selected_date).
612                 total_seconds() / 60.0 / 240.0
613             compat = 1.0 if res.id in allowed else 0.0
614             res_vec = np.array([
615                 1.0 if res.status == 'Работает' else 0.0, # ready
616                 cur_load,
617                 compat,
618                 res.load_minutes / 240.0, # avg load
619                 model_repair[i], # repair (скорректированный)
620                 model_reliability[i] # reliability (скорректированная)
621             ])
622             full = np.concatenate([full, res_vec])
623
624         # --- Получаем логиты от модели ---
625         with torch.no_grad():
626             x = torch.tensor(full, dtype=torch.float32).unsqueeze(0)
627             logits = self.rl_agent.model(x).squeeze(0) # (R,)
628
629         # --- Штраф за низкую готовность (постобработка) ---
630         # Используем скорректированные значения, чтобы штраф был
631         # согласован с моделью
632         effective = model_reliability * (1.0 - 0.5 * model_repair)
633         adjusted = logits + torch.tensor(np.log(effective + 1e-6))
634         best_idx = adjusted.argmax().item()
635         best_rid = self.resources[best_idx].id
636
637         # Планируем
638         start = max(work_until[best_rid], self.selected_date)
639         dur = timedelta(minutes=op.norm_duration)

```

```

637         end = start + dur
638         schedule[op.id] = {'resource_id': best_rid, 'start': start, 'end'
639                             : end}
640         work_until[best_rid] = end
641
642     self.current_schedule = schedule
643     self._sync_schedule_to_task_queue()
644     self.dashboard_changed.emit()
645     self.message_signal.emit("info", "План построен нейросетью с учётом
        готовности.")
646
647     def plan_with_reliability(self):
648         """
649         Планирование с учётом готовности ресурсов.
650         Использует эвристику EDD + эффективная загрузка (workload /
651         reliability).
652         """
653         unscheduled = []
654         for order in self.orders:
655             for op in order.ops:
656                 if op.id not in self.current_schedule and op.id not in self.
657                     approved_schedule:
658                     unscheduled.append((order, op))
659         if not unscheduled:
660             return
661
662         unscheduled.sort(key=lambda x: x[0].due_date)
663
664         if not hasattr(self, 'cached_res_op_types'):
665             self.cached_res_op_types = get_resource_operation_types()
666
667         work_until = {r.id: self.selected_date for r in self.resources}
668         schedule = {}
669
670         for order, op in unscheduled:
671             if op.type_id:
672                 allowed_rids = self.cached_res_op_types.get(op.type_id, [])
673             else:
674                 allowed_rids = list(self.resource_map.keys())
675             if not allowed_rids:
676                 allowed_rids = list(self.resource_map.keys())
677
678             def effective_load(rid):
679                 workload = (work_until[rid] - self.selected_date).
680                     total_seconds() / 60.0
681                 rel = getattr(self.resource_map[rid], 'reliability', 1.0)
682                 if rel <= 0.0:
683                     return float('inf')
684                 if getattr(self.resource_map[rid], 'repair', 0):
685                     rel *= 0.5
686                 return workload / rel
687
688             best_rid = min(allowed_rids, key=effective_load)
689             start = work_until[best_rid]

```

```

686         dur = timedelta(minutes=op.norm_duration)
687         end = start + dur
688         schedule[op.id] = {'resource_id': best_rid, 'start': start, 'end'
689                             : end}
689         work_until[best_rid] = end
690
691     self.current_schedule = schedule
692     self._sync_schedule_to_task_queue()
693     self.dashboard_changed.emit()
694     self.message_signal.emit("info", "План построен с учётом готовности."
695                               )
696
697     def add_resource(self, rid, name):
698         if rid in self.resource_map:
699             self.message_signal.emit("error", "Ресурс уже существует.")
700             return
701         r = Resource(rid, name, reliability=1.0, repair=0)
702         self.resources.append(r)
703         self.resource_map[rid] = r
704         save_resources(self.resources)
705         self.aps_engine.resources[rid] = r
706         self.dashboard_changed.emit()
707
708     def delete_resource(self, rid):
709         for order in self.orders:
710             for op in order.ops:
711                 if op.resource_id == rid:
712                     self.message_signal.emit("error", "На ресурсе есть
713                                             операции.")
714                     return
715         self.resources = [r for r in self.resources if r.id != rid]
716         del self.resource_map[rid]
717         delete_resource(rid)
718         save_resources(self.resources)
719         self.dashboard_changed.emit()
720
721     def add_operation_type_core(self, name):
722         tid = add_operation_type(name)
723         return tid
724
725     def delete_operation_type_core(self, tid):
726         delete_operation_type(tid)
727
728     def set_resource_types_core(self, rid, type_ids):
729         set_resource_operation_types(rid, type_ids)
730
731     def add_state_type_core(self, name, color):
732         sid = add_state_type(name, color)
733         self.state_types[sid] = (name, color)
734         return sid
735
736     def delete_state_type_core(self, sid):
737         delete_state_type(sid)
738         if sid in self.state_types:

```

```

737         del self.state_types[sid]
738
739     def add_downtime_core(self, rid, start, end, state_id):
740         self.downtimes.setdefault(rid, []).append((start, end, state_id))
741         save_downtime(rid, start, end, state_id)
742
743     def clear_downtimes_core(self, rid):
744         if rid in self.downtimes:
745             for item in self.downtimes[rid]:
746                 delete_downtime(rid, item[0], item[1])
747             del self.downtimes[rid]
748
749     def train_predictor(self):
750         try:
751             self.aps_engine.train_predictor()
752             self.message_signal.emit("info", "Предсказатель обновлён.")
753         except Exception as e:
754             self.message_signal.emit("error", str(e))
755
756     def _get_op_duration(self, op_id):
757         for order in self.orders:
758             for op in order.ops:
759                 if op.id == op_id:
760                     return timedelta(minutes=op.norm_duration)
761         return None
762
763     def _remove_from_schedules(self, op_id):
764         self.current_schedule.pop(op_id, None)
765         self.approved_schedule.pop(op_id, None)
766
767     def _get_operation_by_id(self, op_id):
768         for order in self.orders:
769             for op in order.ops:
770                 if op.id == op_id:
771                     return op
772         return None
773
774     def _get_order_by_op(self, op_id):
775         for order in self.orders:
776             for op in order.ops:
777                 if op.id == op_id:
778                     return order
779         return None
780
781     def load_rl_model_if_available(self):
782         path = self.settings.get('model_path')
783         if not path or not os.path.exists(path):
784             return False
785         try:
786             if path.endswith('.pkl'):
787                 self.rl_agent = BCAgent(path)
788             elif path.endswith('.pth'):
789                 self.rl_agent = SimpleAgent(path) # <-- теперь используем простую модель

```



```

790         else:
791             return False
792         self.loaded_model_name = os.path.splitext(os.path.basename(path))
793             [0]
794             return True
795     except Exception as e:
796         self.message_signal.emit("error", f"Ошибка загрузки модели: {e}")
797         return False

```

training.py

```

1  # training.py
2  import numpy as np
3  import torch
4  import torch.nn as nn
5  import torch.optim as optim
6  from torch.utils.data import DataLoader, TensorDataset, random_split
7  from PySide6.QtCore import Signal, QObject
8  import threading
9  import random
10 from datetime import datetime, timedelta
11 from datamodels import Order, Operation, Resource
12 from optimizer_milp import build_milp_schedule
13 import os
14 import json
15
16 import matplotlib
17 matplotlib.use('Agg')
18 import matplotlib.pyplot as plt
19
20 from sklearn.metrics import confusion_matrix, classification_report
21 import seaborn as sns # для красивой матрицы (опционально), если нет, можно
    без
22
23 class ResourceOpPointerNet(nn.Module):
24     def __init__(self, d_res, d_op, embed_dim=128, hidden_dim=256, num_heads
        =8):
25         super().__init__()
26         self.d_res = d_res
27         self.d_op = d_op
28         self.embed_dim = embed_dim
29
30         self.res_encoder = nn.Sequential(
31             nn.Linear(d_res, embed_dim),
32             nn.ReLU(),
33             nn.Linear(embed_dim, embed_dim)
34         )
35         self.op_encoder = nn.Sequential(
36             nn.Linear(d_op, embed_dim),
37             nn.ReLU(),
38             nn.Linear(embed_dim, embed_dim)
39         )
40         self.attn = nn.MultiheadAttention(embed_dim=embed_dim, num_heads=
        num_heads, batch_first=True)
41         self.proj = nn.Sequential(

```

```

42         nn.Linear(embed_dim * 2, hidden_dim),
43         nn.ReLU(),
44         nn.Dropout(0.1),
45         nn.Linear(hidden_dim, 1)
46     )
47
48     # ----- Простая модель -----
49     class SimpleDispatcher(nn.Module):
50     def __init__(self, input_dim, output_dim):
51         super().__init__()
52         self.net = nn.Sequential(
53             nn.Linear(input_dim, 64),
54             nn.ReLU(),
55             nn.BatchNorm1d(64),
56             nn.Linear(64, 32),
57             nn.ReLU(),
58             nn.BatchNorm1d(32),
59             nn.Linear(32, output_dim)
60         )
61     def forward(self, x):
62         return self.net(x)
63
64     # ----- Сигналы и Тренер -----
65     class TrainingSignals(QObject):
66         progress = Signal(int, float)
67         log = Signal(str)
68
69     class Trainer:
70     def __init__(self, signals=None):
71         self.signals = signals
72         self.model = None
73         self.stop_event = threading.Event()
74         self.history = {'epoch': [], 'train_loss': [], 'val_loss': [], '
75                         val_acc': []}
76         self.y_true = None
77         self.y_pred = None
78         self.class_names = None
79
80     def save(self, path):
81         if self.model is None:
82             return False
83         try:
84             torch.save(self.model.state_dict(), path)
85             return True
86         except Exception as e:
87             if self.signals:
88                 self.signals.log.emit(f"Ошибка сохранения весов: {e}")
89             return False
90
91     def load(self, path):
92         if self.model is None:
93             return False
94         try:
95             self.model.load_state_dict(torch.load(path, map_location='cpu'))

```

```

95         if self.signals:
96             self.signals.log.emit(f"Веса загружены из {path}")
97         return True
98     except Exception as e:
99         if self.signals:
100             self.signals.log.emit(f"Не удалось загрузить веса: {e}")
101         return False
102
103 def train_bc(self, epochs=50, batch_size=32, lr=1e-3, load_path=None,
104 force_regen_data=False):
105     self.stop_event.clear()
106     if self.signals:
107         self.signals.log.emit("Загрузка/генерация данных...")
108
109     # Используем GPSS датасет
110     if os.path.exists("gpss_dataset.npz"):
111         data = np.load("gpss_dataset.npz")
112         X = data['X']
113         y = data['y']
114         if self.signals:
115             self.signals.log.emit(f"Загружен датасет GPSS: {X.shape[0]}
116                                 примеров, признаков: {X.shape[1]}, классов: {np.max(y)+1}"
117                                 )
118     else:
119         if self.signals:
120             self.signals.log.emit("gpss_dataset.npz не найден.
121                                 Сгенерируйте его.")
122         return None
123
124     input_dim = X.shape[1]
125     output_dim = len(np.unique(y))
126     self.class_names = [str(i) for i in range(output_dim)] # имена
127                                     классов (индексы ресурсов)
128
129     dataset = TensorDataset(torch.FloatTensor(X), torch.LongTensor(y))
130     train_size = int(0.8 * len(dataset))
131     val_size = len(dataset) - train_size
132     train_set, val_set = random_split(dataset, [train_size, val_size])
133     train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=
134                               True)
135     val_loader = DataLoader(val_set, batch_size=batch_size)
136
137     self.model = SimpleDispatcher(input_dim, output_dim)
138     if load_path:
139         self.load(load_path)
140
141     optimizer = optim.Adam(self.model.parameters(), lr=lr)
142     criterion = nn.CrossEntropyLoss()
143
144     best_val_acc = 0.0
145     best_model_state = None
146
147     self.history = {'epoch': [], 'train_loss': [], 'val_loss': [], '
148                     val_acc': []}

```

```

142
143     for epoch in range(1, epochs + 1):
144         if self.stop_event.is_set():
145             if self.signals:
146                 self.signals.log.emit("Обучение остановлено пользователем
147                                     .")
148                 break
149
150         self.model.train()
151         train_loss = 0
152         for bx, by in train_loader:
153             optimizer.zero_grad()
154             out = self.model(bx)
155             loss = criterion(out, by)
156             loss.backward()
157             optimizer.step()
158             train_loss += loss.item()
159         avg_train_loss = train_loss / len(train_loader)
160
161         self.model.eval()
162         val_loss = 0
163         correct = 0
164         total = 0
165         all_preds = []
166         all_labels = []
167         with torch.no_grad():
168             for bx, by in val_loader:
169                 out = self.model(bx)
170                 loss = criterion(out, by)
171                 val_loss += loss.item()
172                 pred = out.argmax(dim=1)
173                 correct += (pred == by).sum().item()
174                 total += by.size(0)
175                 all_preds.extend(pred.cpu().numpy())
176                 all_labels.extend(by.cpu().numpy())
177         avg_val_loss = val_loss / len(val_loader)
178         val_acc = correct / total if total > 0 else 0
179
180         self.history['epoch'].append(epoch)
181         self.history['train_loss'].append(avg_train_loss)
182         self.history['val_loss'].append(avg_val_loss)
183         self.history['val_acc'].append(val_acc)
184
185         if val_acc > best_val_acc:
186             best_val_acc = val_acc
187             best_model_state = self.model.state_dict()
188             # сохраняем предсказания лучшей эпохи для матрицы ошибок
189             self.y_true = all_labels
190             self.y_pred = all_preds
191
192         if self.signals:
193             self.signals.progress.emit(epoch, avg_train_loss)
194             if epoch % 10 == 0 or epoch == 1:
195                 self.signals.log.emit(

```

```

195         f"Эпоха {epoch:4d} | Потери: {avg_train_loss:.4f} |
          Точность: {val_acc:.4f} | Лучшая: {best_val_acc:.4f}"
196     )
197
198     if best_model_state is not None:
199         self.model.load_state_dict(best_model_state)
200         if self.signals:
201             self.signals.log.emit(f"Лучшая точность валидации: {best_val_acc:.4f}")
202
203     # Сохраняем метрики
204     self.save_metrics("PointerNet")
205     return self.model
206
207 def save_metrics(self, model_name):
208     if not self.history:
209         return
210     import os, json
211     from datetime import datetime
212     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
213     folder = f"metrics/{model_name}/{timestamp}"
214     os.makedirs(folder, exist_ok=True)
215
216     # JSON с историей
217     with open(f"{folder}/history.json", 'w') as f:
218         json.dump(self.history, f, indent=2)
219
220     # График потерь
221     plt.figure()
222     plt.plot(self.history['epoch'], self.history['train_loss'], label='Train Loss')
223     plt.plot(self.history['epoch'], self.history['val_loss'], label='Val Loss')
224     plt.xlabel('Epoch')
225     plt.ylabel('Loss')
226     plt.title(f'{model_name} Learning Curve')
227     plt.legend()
228     plt.savefig(f"{folder}/loss.png")
229     plt.close()
230
231     # График точности
232     plt.figure()
233     plt.plot(self.history['epoch'], self.history['val_acc'], label='Val Accuracy')
234     plt.xlabel('Epoch')
235     plt.ylabel('Accuracy')
236     plt.title(f'{model_name} Validation Accuracy')
237     plt.legend()
238     plt.savefig(f"{folder}/accuracy.png")
239     plt.close()
240
241     # Матрица ошибок и отчёт по классификации (только если есть разметка)

```

```

242         if self.y_true is not None and self.y_pred is not None and self.
            class_names is not None:
243             cm = confusion_matrix(self.y_true, self.y_pred)
244             plt.figure(figsize=(8, 6))
245             try:
246                 import seaborn as sns
247                 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                             xticklabels=self.class_names, yticklabels=self.class_names
                             )
248             except ImportError:
249                 plt.imshow(cm, interpolation='nearest', cmap='Blues')
250                 plt.colorbar()
251                 plt.title(f'{model_name} Confusion Matrix (Validation)')
252                 plt.xlabel('Predicted')
253                 plt.ylabel('True')
254                 plt.savefig(f'{folder}/confusion_matrix.png')
255                 plt.close()
256
257             # Classification report
258             report = classification_report(self.y_true, self.y_pred,
                                             target_names=self.class_names, output_dict=True)
259             with open(f'{folder}/classification_report.json', 'w') as f:
260                 json.dump(report, f, indent=2)
261
262         if self.signals:
263             self.signals.log.emit(f"Метрики сохранены в {folder}")
264
265     def stop(self):
266         self.stop_event.set()

```

database.py

```

1  # database.py
2  import sqlite3
3  from datetime import datetime
4
5  DB_NAME = "aps.db"
6
7  def init_db():
8      conn = sqlite3.connect(DB_NAME)
9      cur = conn.cursor()
10     cur.execute("""
11         CREATE TABLE IF NOT EXISTS orders (
12             id TEXT PRIMARY KEY,
13             due_date TEXT NOT NULL,
14             priority_weight REAL NOT NULL,
15             name TEXT DEFAULT ''
16         )
17     """)
18     cur.execute("PRAGMA table_info(orders)")
19     cols = [c[1] for c in cur.fetchall()]
20     if 'name' not in cols:
21         cur.execute("ALTER TABLE orders ADD COLUMN name TEXT DEFAULT ''")
22
23     cur.execute("""

```

```

24     CREATE TABLE IF NOT EXISTS operations (
25         id TEXT PRIMARY KEY,
26         order_id TEXT NOT NULL,
27         item TEXT,
28         op_number INTEGER,
29         norm_duration REAL,
30         type_id TEXT DEFAULT '',
31         predecessors TEXT DEFAULT '',
32         quantity INTEGER DEFAULT 1,
33         urgency REAL DEFAULT 1.0,
34         slack_hours REAL DEFAULT 0,
35         FOREIGN KEY (order_id) REFERENCES orders(id)
36     )
37     """
38     cur.execute("PRAGMA table_info(operations)")
39     cols = [c[1] for c in cur.fetchall()]
40     if 'predecessors' not in cols:
41         cur.execute("ALTER TABLE operations ADD COLUMN predecessors TEXT
42             DEFAULT ''")
43     if 'type_id' not in cols:
44         cur.execute("ALTER TABLE operations ADD COLUMN type_id TEXT DEFAULT
45             ''")
46     if 'quantity' not in cols:
47         cur.execute("ALTER TABLE operations ADD COLUMN quantity INTEGER
48             DEFAULT 1")
49     if 'urgency' not in cols:
50         cur.execute("ALTER TABLE operations ADD COLUMN urgency REAL DEFAULT
51             1.0")
52     if 'slack_hours' not in cols:
53         cur.execute("ALTER TABLE operations ADD COLUMN slack_hours REAL
54             DEFAULT 0")
55
56     cur.execute("""
57     CREATE TABLE IF NOT EXISTS resources (
58         id TEXT PRIMARY KEY,
59         name TEXT,
60         section TEXT DEFAULT 'Основной участок',
61         status TEXT DEFAULT 'Работает',
62         operator_name TEXT DEFAULT '',
63         load_minutes REAL DEFAULT 0,
64         downtime_minutes REAL DEFAULT 0,
65         work_hours INTEGER DEFAULT 24,
66         repair INTEGER DEFAULT 0,
67         reliability REAL DEFAULT 1.0
68     )
69     """)
70     cur.execute("PRAGMA table_info(resources)")
71     cols = [c[1] for c in cur.fetchall()]
72     if 'section' not in cols:
73         cur.execute("ALTER TABLE resources ADD COLUMN section TEXT DEFAULT '
74             Основной участок'")
75     if 'status' not in cols:
76         cur.execute("ALTER TABLE resources ADD COLUMN status TEXT DEFAULT '
77             Работает'")

```

```

71 if 'operator_name' not in cols:
72     cur.execute("ALTER TABLE resources ADD COLUMN operator_name TEXT
73                 DEFAULT '')
74 if 'load_minutes' not in cols:
75     cur.execute("ALTER TABLE resources ADD COLUMN load_minutes REAL
76                 DEFAULT 0")
77 if 'downtime_minutes' not in cols:
78     cur.execute("ALTER TABLE resources ADD COLUMN downtime_minutes REAL
79                 DEFAULT 0")
80 if 'work_hours' not in cols:
81     cur.execute("ALTER TABLE resources ADD COLUMN work_hours INTEGER
82                 DEFAULT 24")
83 if 'repair' not in cols:
84     cur.execute("ALTER TABLE resources ADD COLUMN repair INTEGER DEFAULT
85                 0")
86 if 'reliability' not in cols:
87     cur.execute("ALTER TABLE resources ADD COLUMN reliability REAL
88                 DEFAULT 1.0")
89
90 cur.execute("""
91     CREATE TABLE IF NOT EXISTS operation_types (
92         id TEXT PRIMARY KEY,
93         name TEXT
94     )
95 """)
96 cur.execute("""
97     CREATE TABLE IF NOT EXISTS resource_operation_types (
98         resource_id TEXT NOT NULL,
99         operation_type_id TEXT NOT NULL,
100         PRIMARY KEY (resource_id, operation_type_id),
101         FOREIGN KEY (resource_id) REFERENCES resources(id),
102         FOREIGN KEY (operation_type_id) REFERENCES operation_types(id)
103     )
104 """)
105 cur.execute("""
106     CREATE TABLE IF NOT EXISTS schedule (
107         operation_id TEXT PRIMARY KEY,
108         resource_id TEXT NOT NULL,
109         start TEXT,
110         end TEXT,
111         fixed INTEGER DEFAULT 0,
112         status TEXT DEFAULT 'planned',
113         FOREIGN KEY (operation_id) REFERENCES operations(id),
114         FOREIGN KEY (resource_id) REFERENCES resources(id)
115     )
116 """)
117 cur.execute("PRAGMA table_info(schedule)")
118 cols = [c[1] for c in cur.fetchall()]
119 if 'status' not in cols:
120     cur.execute("ALTER TABLE schedule ADD COLUMN status TEXT DEFAULT '
121                 planned'")
122     cur.execute("UPDATE schedule SET status='fixed' WHERE fixed=1")
123     cur.execute("UPDATE schedule SET status='planned' WHERE fixed=0")

```



```

118
119 cur.execute("""
120     CREATE TABLE IF NOT EXISTS task_queue (
121         id INTEGER PRIMARY KEY AUTOINCREMENT,
122         resource_id TEXT NOT NULL,
123         operation_id TEXT UNIQUE NOT NULL,
124         position INTEGER NOT NULL DEFAULT 0,
125         status TEXT NOT NULL DEFAULT 'pending' CHECK(status IN ('pending
            ', 'active', 'completed'))),
126         FOREIGN KEY (resource_id) REFERENCES resources(id),
127         FOREIGN KEY (operation_id) REFERENCES operations(id)
128     )
129 """)
130
131 cur.execute("""
132     CREATE TABLE IF NOT EXISTS state_types (
133         id TEXT PRIMARY KEY,
134         name TEXT,
135         color TEXT
136     )
137 """)
138 cur.execute("""
139     CREATE TABLE IF NOT EXISTS downtimes (
140         resource_id TEXT NOT NULL,
141         start TEXT NOT NULL,
142         end TEXT,
143         state_id TEXT,
144         FOREIGN KEY (resource_id) REFERENCES resources(id),
145         FOREIGN KEY (state_id) REFERENCES state_types(id)
146     )
147 """)
148
149 cur.execute("""
150     CREATE TABLE IF NOT EXISTS app_settings (
151         key TEXT PRIMARY KEY,
152         value TEXT,
153         updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
154     )
155 """)
156
157 cur.execute("""
158     CREATE TABLE IF NOT EXISTS resource_state_log (
159         id INTEGER PRIMARY KEY AUTOINCREMENT,
160         resource_id TEXT NOT NULL,
161         state_id TEXT NOT NULL,
162         start TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
163         end TIMESTAMP,
164         source TEXT DEFAULT 'manual' CHECK(source IN ('manual', 'mes')),
165         FOREIGN KEY (resource_id) REFERENCES resources(id),
166         FOREIGN KEY (state_id) REFERENCES state_types(id)
167     )
168 """)
169
170 cur.execute("""

```

```

171     CREATE TABLE IF NOT EXISTS operation_state_log (
172         id INTEGER PRIMARY KEY AUTOINCREMENT,
173         operation_id TEXT NOT NULL,
174         status TEXT NOT NULL CHECK(status IN ('pending','active','
175             completed','interrupted')),
176         resource_id TEXT,
177         state_id TEXT,
178         timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
179         source TEXT DEFAULT 'manual' CHECK(source IN ('manual','mes')),
180         FOREIGN KEY (operation_id) REFERENCES operations(id),
181         FOREIGN KEY (resource_id) REFERENCES resources(id),
182         FOREIGN KEY (state_id) REFERENCES state_types(id)
183     )
184     """
185 cur.execute("""
186     CREATE TABLE IF NOT EXISTS proposed_changes (
187         id INTEGER PRIMARY KEY AUTOINCREMENT,
188         operation_id TEXT NOT NULL,
189         new_resource_id TEXT NOT NULL,
190         new_start TEXT NOT NULL,
191         new_end TEXT NOT NULL,
192         accepted INTEGER DEFAULT 0,
193         FOREIGN KEY (operation_id) REFERENCES operations(id),
194         FOREIGN KEY (new_resource_id) REFERENCES resources(id)
195     )
196     """
197
198 # столбец status у заказов
199 cur.execute("PRAGMA table_info(orders)")
200 cols = [c[1] for c in cur.fetchall()]
201 if 'status' not in cols:
202     cur.execute("ALTER TABLE orders ADD COLUMN status TEXT DEFAULT '
203         active'")
204     cur.execute("UPDATE orders SET status='active' WHERE status IS NULL")
205
206 conn.commit()
207 conn.close()
208
209 # ----- Orders -----
210 def save_orders_to_db(orders):
211     conn = sqlite3.connect(DB_NAME)
212     cur = conn.cursor()
213     for order in orders:
214         cur.execute("INSERT OR REPLACE INTO orders (id, due_date,
215             priority_weight, name, status) VALUES (?, ?, ?, ?, 'active')",
216             (order.id, order.due_date.isoformat(), order.
217                 priority_weight, order.name))
218     for op in order.ops:
219         preds = ",".join(op.predecessors) if op.predecessors else ""
220         type_id = getattr(op, 'type_id', '')
221         quantity = getattr(op, 'quantity', 1)
222         urgency = getattr(op, 'urgency', 1.0)
223         slack = getattr(op, 'slack_hours', 0.0)

```

```

221         cur.execute("INSERT OR REPLACE INTO operations (id, order_id,
                    item, op_number, norm_duration, type_id, predecessors,
                    quantity, urgency, slack_hours) VALUES (?, ?, ?, ?, ?, ?, ?,
                    ?, ?, ?)",
222                     (op.id, order.id, op.item, op.op_number, op.
                        norm_duration, type_id, preds, quantity, urgency,
                        slack))

223     conn.commit()
224     conn.close()
225
226     def load_orders_from_db(resources=None):
227         from datamodels import Order, Operation
228         conn = sqlite3.connect(DB_NAME)
229         cur = conn.cursor()
230         cur.execute("PRAGMA table_info(operations)")
231         cols = [c[1] for c in cur.fetchall()]
232         has_preds = 'predecessors' in cols
233         has_type = 'type_id' in cols
234         has_quantity = 'quantity' in cols
235         has_urgency = 'urgency' in cols
236         has_slack = 'slack_hours' in cols
237         # Загружаем только активные заказы
238         cur.execute("SELECT * FROM orders WHERE status='active'")
239         orders = []
240         for row in cur:
241             order = Order(id=row[0], due_date=datetime.fromisoformat(row[1]),
                           priority_weight=row[2])
242             if len(row) > 3:
243                 order.name = row[3] if row[3] else ""
244             cur2 = conn.execute("SELECT * FROM operations WHERE order_id=?", (
                order.id,))
245             for op_row in cur2:
246                 preds = []
247                 type_id = ''
248                 quantity = 1
249                 urgency = 1.0
250                 slack = 0.0
251                 if has_preds and len(op_row) >= 7:
252                     preds_str = op_row[6] if op_row[6] else ''
253                     preds = preds_str.split(",") if preds_str else []
254                 if has_type and len(op_row) >= 8:
255                     type_id = op_row[7] if op_row[7] else ''
256                 if has_quantity and len(op_row) >= 9:
257                     try:
258                         quantity = int(float(op_row[8])) if op_row[8] else 1
259                     except (ValueError, TypeError):
260                         quantity = 1
261                 if has_urgency and len(op_row) >= 10:
262                     urgency = op_row[9] if op_row[9] else 1.0
263                 if has_slack and len(op_row) >= 11:
264                     slack = op_row[10] if op_row[10] else 0.0
265                 op = Operation(
266                     id=op_row[0], order_id=op_row[1], item=op_row[2],
267                     op_number=op_row[3], norm_duration=op_row[4],

```

```

268         predecessors=preds
269     )
270     if type_id:
271         op.type_id = type_id
272         op.quantity = quantity
273         op.urgency = urgency
274         op.slack_hours = slack
275         order.ops.append(op)
276     orders.append(order)
277 conn.close()
278 return orders
279
280 def delete_order_from_db(order_id):
281     conn = sqlite3.connect(DB_NAME)
282     cur = conn.cursor()
283     cur.execute("DELETE FROM orders WHERE id=?", (order_id,))
284     cur.execute("DELETE FROM operations WHERE order_id=?", (order_id,))
285     conn.commit()
286     conn.close()
287
288 def delete_operation_from_db(op_id):
289     conn = sqlite3.connect(DB_NAME)
290     cur = conn.cursor()
291     cur.execute("DELETE FROM operations WHERE id=?", (op_id,))
292     conn.commit()
293     conn.close()
294
295 def set_order_status(order_id, status):
296     conn = sqlite3.connect(DB_NAME)
297     cur = conn.cursor()
298     cur.execute("UPDATE orders SET status=? WHERE id=?", (status, order_id))
299     conn.commit()
300     conn.close()
301
302 # ----- Resources -----
303 def save_resources(resources):
304     conn = sqlite3.connect(DB_NAME)
305     cur = conn.cursor()
306     cur.execute("DELETE FROM resources")
307     for r in resources:
308         cur.execute("""
309             INSERT INTO resources (id, name, section, status, operator_name,
310                                   load_minutes, downtime_minutes, work_hours
311                                   , repair, reliability)
312             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
313             """, (r.id, r.name, r.section, r.status, r.operator_name,
314                 r.load_minutes, r.downtime_minutes,
315                 getattr(r, 'work_hours', 24),
316                 int(getattr(r, 'repair', 0)),
317                 float(getattr(r, 'reliability', 1.0))))
318     conn.commit()
319     conn.close()
320 def load_resources():

```

```

321 from datamodels import Resource
322 conn = sqlite3.connect(DB_NAME)
323 cur = conn.cursor()
324
325 # Гарантируем наличие нужных столбцов (если база старая)
326 cur.execute("PRAGMA table_info(resources)")
327 cols = [c[1] for c in cur.fetchall()]
328 if 'repair' not in cols:
329     cur.execute("ALTER TABLE resources ADD COLUMN repair INTEGER DEFAULT
330                 0")
331 if 'reliability' not in cols:
332     cur.execute("ALTER TABLE resources ADD COLUMN reliability REAL
333                 DEFAULT 1.0")
334 conn.commit()
335
336 cur.execute("""
337     SELECT id, name, section, status, operator_name,
338            load_minutes, downtime_minutes, work_hours, repair,
339            reliability
340     FROM resources
341 """)
342 resources = []
343 for row in cur:
344     r = Resource(id=row[0], name=row[1])
345     r.section = row[2] if row[2] else "Основной участок"
346     r.status = row[3] if row[3] else "Работает"
347     r.operator_name = row[4] if row[4] else ""
348     r.load_minutes = float(row[5]) if row[5] else 0.0
349     r.downtime_minutes = float(row[6]) if row[6] else 0.0
350     r.work_hours = int(row[7]) if row[7] else 24
351     r.repair = int(row[8]) if row[8] is not None else 0
352     r.reliability = float(row[9]) if row[9] is not None else 1.0
353     resources.append(r)
354 conn.close()
355 return resources
356
357 # ----- Остальные функции (сохранены из предыдущей версии) -----
358 def delete_resource(resource_id):
359     conn = sqlite3.connect(DB_NAME)
360     cur = conn.cursor()
361     cur.execute("DELETE FROM resources WHERE id=?", (resource_id,))
362     conn.commit()
363     conn.close()
364
365 def add_operation_type(name):
366     import uuid
367     tid = str(uuid.uuid4())[:8]
368     conn = sqlite3.connect(DB_NAME)
369     cur = conn.cursor()
370     cur.execute("INSERT INTO operation_types (id, name) VALUES (?, ?)", (tid,
371                                     name))
372     conn.commit()
373     conn.close()
374     return tid

```

```

371
372 def get_operation_types():
373     conn = sqlite3.connect(DB_NAME)
374     cur = conn.cursor()
375     cur.execute("SELECT id, name FROM operation_types")
376     types = {row[0]: row[1] for row in cur.fetchall()}
377     conn.close()
378     return types
379
380 def delete_operation_type(type_id):
381     conn = sqlite3.connect(DB_NAME)
382     cur = conn.cursor()
383     cur.execute("DELETE FROM operation_types WHERE id=?", (type_id,))
384     conn.commit()
385     conn.close()
386
387 def set_resource_operation_types(resource_id, type_ids):
388     conn = sqlite3.connect(DB_NAME)
389     cur = conn.cursor()
390     cur.execute("DELETE FROM resource_operation_types WHERE resource_id=?", (
391         resource_id,))
392     for tid in type_ids:
393         cur.execute("INSERT INTO resource_operation_types (resource_id,
394             operation_type_id) VALUES (?, ?)", (resource_id, tid))
395     conn.commit()
396     conn.close()
397
398 def get_resource_operation_types():
399     conn = sqlite3.connect(DB_NAME)
400     cur = conn.cursor()
401     cur.execute("SELECT resource_id, operation_type_id FROM
402         resource_operation_types")
403     res_types = {}
404     for row in cur:
405         res_id, tid = row
406         res_types.setdefault(res_id, []).append(tid)
407     conn.close()
408     return res_types
409
410 # алиас для совместимости
411 get_resource_types = get_resource_operation_types
412
413 def update_schedule(operation_id, resource_id, start, end, fixed=1, status='
414     fixed'):
415     conn = sqlite3.connect(DB_NAME)
416     cur = conn.cursor()
417     if resource_id is None:
418         cur.execute("DELETE FROM schedule WHERE operation_id=?", (
419             operation_id,))
420     else:
421         cur.execute("INSERT OR REPLACE INTO schedule (operation_id,
422             resource_id, start, end, fixed, status) VALUES (?, ?, ?, ?, ?, ?)"
423             ,

```

```

417         (operation_id, resource_id, start.isoformat(), end.
            isoformat(), fixed, status))
418     conn.commit()
419     conn.close()
420
421     def get_fixed_operations():
422         conn = sqlite3.connect(DB_NAME)
423         cur = conn.cursor()
424         cur.execute("SELECT operation_id, resource_id, start, end FROM schedule
            WHERE status='fixed'")
425         fixed = {}
426         for row in cur:
427             fixed[row[0]] = {'resource_id': row[1], 'start': datetime.
                fromisoformat(row[2]), 'end': datetime.fromisoformat(row[3])}
428         conn.close()
429         return fixed
430
431     def get_planned_operations():
432         conn = sqlite3.connect(DB_NAME)
433         cur = conn.cursor()
434         cur.execute("SELECT operation_id, resource_id, start, end FROM schedule
            WHERE status='planned'")
435         planned = {}
436         for row in cur:
437             planned[row[0]] = {'resource_id': row[1], 'start': datetime.
                fromisoformat(row[2]) if row[2] else None,
438                               'end': datetime.fromisoformat(row[3]) if row[3]
                else None}
439         conn.close()
440         return planned
441
442     def add_task_to_queue(resource_id, operation_id, position=None):
443         conn = sqlite3.connect(DB_NAME)
444         cur = conn.cursor()
445         if position is None:
446             cur.execute("SELECT COALESCE(MAX(position),0)+1 FROM task_queue WHERE
                resource_id=? AND status='pending'", (resource_id,))
447             position = cur.fetchone()[0]
448             cur.execute("INSERT OR REPLACE INTO task_queue (resource_id, operation_id
                , position, status) VALUES (?, ?, ?, 'pending')",
                (resource_id, operation_id, position))
449         conn.commit()
450         conn.close()
451
452     def update_task_status(operation_id, new_status):
453         conn = sqlite3.connect(DB_NAME)
454         cur = conn.cursor()
455         cur.execute("UPDATE task_queue SET status=? WHERE operation_id=?", (
            new_status, operation_id))
456         conn.commit()
457         conn.close()
458
459     def get_completed_tasks():
460         conn = sqlite3.connect(DB_NAME)

```

```

462 cur = conn.cursor()
463 cur.execute("""
464     SELECT
465         o.operation_id, op.order_id, op.item, op.type_id, o.resource_id,
466         r.name as resource_name
467     FROM operation_state_log o
468     JOIN operations op ON o.operation_id = op.id
469     JOIN resources r ON o.resource_id = r.id
470     WHERE o.status = 'completed'
471 """)
472 tasks = []
473 for row in cur:
474     tasks.append({
475         'operation_id': row[0],
476         'order_id': row[1],
477         'item': row[2],
478         'type_id': row[3],
479         'resource_id': row[4],
480         'resource_name': row[5]
481     })
482 conn.close()
483 return tasks
484
485 def get_tasks_for_resource(resource_id):
486     conn = sqlite3.connect(DB_NAME)
487     cur = conn.cursor()
488     # Возвращаем только задачи в статусе 'pending' и 'active'
489     cur.execute("SELECT operation_id, position, status FROM task_queue WHERE
490         resource_id=? AND status IN ('pending', 'active') ORDER BY position",
491         (resource_id,))
492     tasks = [{ 'operation_id': row[0], 'position': row[1], 'status': row[2]}
493         for row in cur.fetchall()]
494     conn.close()
495     return tasks
496
497 def delete_task_from_queue(operation_id):
498     conn = sqlite3.connect(DB_NAME)
499     cur = conn.cursor()
500     cur.execute("DELETE FROM task_queue WHERE operation_id=?", (operation_id
501         ,))
502     conn.commit()
503     conn.close()
504
505 def add_state_type(name, color):
506     import uuid
507     sid = str(uuid.uuid4())[:8]
508     conn = sqlite3.connect(DB_NAME)
509     cur = conn.cursor()
510     cur.execute("INSERT INTO state_types (id, name, color) VALUES (?, ?, ?)",
511         (sid, name, color))
512     conn.commit()
513     conn.close()
514     return sid
515

```



```

510 def get_state_types():
511     conn = sqlite3.connect(DB_NAME)
512     cur = conn.cursor()
513     cur.execute("SELECT id, name, color FROM state_types")
514     return {row[0]: (row[1], row[2]) for row in cur.fetchall()}
515
516 def delete_state_type(sid):
517     conn = sqlite3.connect(DB_NAME)
518     cur = conn.cursor()
519     cur.execute("DELETE FROM state_types WHERE id=?", (sid,))
520     conn.commit()
521     conn.close()
522
523 def save_downtime(resource_id, start, end, state_id=""):
524     conn = sqlite3.connect(DB_NAME)
525     cur = conn.cursor()
526     cur.execute("INSERT INTO downtimes (resource_id, start, end, state_id)
527                 VALUES (?, ?, ?, ?)",
528                 (resource_id, start.isoformat(), end.isoformat(), state_id))
529     conn.commit()
530     conn.close()
531
532 def load_downtimes():
533     conn = sqlite3.connect(DB_NAME)
534     cur = conn.cursor()
535     cur.execute("SELECT resource_id, start, end, state_id FROM downtimes")
536     downtimes = {}
537     for row in cur:
538         res_id, start_str, end_str, state_id = row
539         downtimes.setdefault(res_id, []).append((datetime.fromisoformat(
540             start_str), datetime.fromisoformat(end_str), state_id))
541     conn.close()
542     return downtimes
543
544 def delete_downtime(resource_id, start, end):
545     conn = sqlite3.connect(DB_NAME)
546     cur = conn.cursor()
547     cur.execute("DELETE FROM downtimes WHERE resource_id=? AND start=? AND
548                 end=?",
549                 (resource_id, start.isoformat(), end.isoformat()))
550     conn.commit()
551     conn.close()
552
553 def get_setting(key, default=None):
554     conn = sqlite3.connect(DB_NAME)
555     cur = conn.cursor()
556     cur.execute("SELECT value FROM app_settings WHERE key=?", (key,))
557     row = cur.fetchone()
558     conn.close()
559     return row[0] if row else default
560
561 def set_setting(key, value):
562     conn = sqlite3.connect(DB_NAME)
563     cur = conn.cursor()

```

```

561 cur.execute("INSERT OR REPLACE INTO app_settings (key, value, updated_at)
           VALUES (?, ?, CURRENT_TIMESTAMP)", (key, str(value)))
562 conn.commit()
563 conn.close()
564
565 def get_all_settings():
566     conn = sqlite3.connect(DB_NAME)
567     cur = conn.cursor()
568     cur.execute("SELECT key, value FROM app_settings")
569     settings = {row[0]: row[1] for row in cur.fetchall()}
570     conn.close()
571     return settings
572
573 def log_resource_state(resource_id, state_id, source='manual'):
574     conn = sqlite3.connect(DB_NAME)
575     cur = conn.cursor()
576     cur.execute("UPDATE resource_state_log SET end=CURRENT_TIMESTAMP WHERE
           resource_id=? AND end IS NULL", (resource_id,))
577     cur.execute("INSERT INTO resource_state_log (resource_id, state_id,
           source) VALUES (?, ?, ?)", (resource_id, state_id, source))
578     conn.commit()
579     conn.close()
580
581 def log_operation_state(operation_id, status, resource_id=None, state_id=None
, source='manual'):
582     conn = sqlite3.connect(DB_NAME)
583     cur = conn.cursor()
584     cur.execute("INSERT INTO operation_state_log (operation_id, status,
           resource_id, state_id, source) VALUES (?, ?, ?, ?, ?)",
           (operation_id, status, resource_id, state_id, source))
585     conn.commit()
586     conn.close()
587
588 def get_execution_data():
589     conn = sqlite3.connect(DB_NAME)
590     cur = conn.cursor()
591     cur.execute("""
592         SELECT o.operation_id, o.resource_id, op.norm_duration,
593                (julianday(o.end) - julianday(o.start)) * 1440
594         FROM (
595             SELECT operation_id, resource_id,
596                    MIN(timestamp) as start,
597                    MAX(timestamp) as end
598             FROM operation_state_log
599             WHERE status IN ('active', 'completed')
600             GROUP BY operation_id
601         ) o
602         JOIN operations op ON o.operation_id = op.id
603     """)
604     data = []
605     for row in cur:
606         data.append({'operation_id': row[0], 'resource_id': row[1], '
           norm_duration': row[2], 'actual_duration': row[3]})
607     conn.close()

```

```

609     return data
610
611 def add_proposed_change(operation_id, new_resource_id, new_start, new_end):
612     conn = sqlite3.connect(DB_NAME)
613     cur = conn.cursor()
614     cur.execute("INSERT INTO proposed_changes (operation_id, new_resource_id,
615             new_start, new_end) VALUES (?, ?, ?, ?)",
616             (operation_id, new_resource_id, new_start.isoformat(),
617             new_end.isoformat()))
618
619     conn.commit()
620     conn.close()
621
622 def get_proposed_changes():
623     conn = sqlite3.connect(DB_NAME)
624     cur = conn.cursor()
625     cur.execute("SELECT id, operation_id, new_resource_id, new_start, new_end
626             FROM proposed_changes WHERE accepted=0")
627     changes = []
628     for row in cur:
629         changes.append({
630             'id': row[0], 'operation_id': row[1], 'new_resource_id': row[2],
631             'new_start': datetime.fromisoformat(row[3]), 'new_end': datetime.
632             fromisoformat(row[4])
633         })
634     conn.close()
635     return changes
636
637 def accept_change(change_id):
638     conn = sqlite3.connect(DB_NAME)
639     cur = conn.cursor()
640     cur.execute("UPDATE proposed_changes SET accepted=1 WHERE id=?", (
641         change_id,))
642     conn.commit()
643     conn.close()
644
645 def apply_accepted_changes():
646     conn = sqlite3.connect(DB_NAME)
647     cur = conn.cursor()
648     cur.execute("SELECT operation_id, new_resource_id, new_start, new_end
649             FROM proposed_changes WHERE accepted=1")
650     for row in cur:
651         op_id, res, start, end = row
652         cur.execute("UPDATE schedule SET resource_id=?, start=?, end=?,
653             status='fixed' WHERE operation_id=?",
654             (res, start, end, op_id))
655     conn.commit()
656     conn.close()

```

Место для диска